



ParticleZoo

ParticleZoo Reference Manual

v1.1.2

July 9, 2026

1 ParticleZoo	1
1.1 Overview	2
1.2 Supported Formats	3
1.3 Architecture	4
1.4 Building and Installation	6
1.5 Using the Library	9
1.6 Command-Line Tools	11
1.7 Examples	13
1.8 Extending the Library	14
1.9 ROOT Format Support (Optional)	15
1.10 Python Bindings	17
1.11 Performance Considerations	18
1.12 Troubleshooting	19
2 License	21
3 Deprecated List	23
4 Hierarchical Index	25
4.1 Class Hierarchy	26
5 Class Index	27
5.1 Class List	28
6 Namespace Documentation	31
6.1 ParticleZoo Namespace Reference	32
7 Class Documentation	41
7.1 ParticleZoo::ByteBuffer Class Reference	42
7.2 ParticleZoo::EGSphspFile::Reader Class Reference	55
7.3 ParticleZoo::EGSphspFile::Writer Class Reference	64
7.4 ParticleZoo::FixedValues Struct Reference	73
7.5 ParticleZoo::FormatRegistry Class Reference	75
7.6 ParticleZoo::HistoryBalancedParallelReader Class Reference	83
7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference	91
7.8 ParticleZoo::IAEAphspFile::Reader Class Reference	126
7.9 ParticleZoo::IAEAphspFile::Writer Class Reference	134

7.10 ParticleZoo::Particle Class Reference	144
7.11 ParticleZoo::ParticleBalancedParallelReader Class Reference	164
7.12 ParticleZoo::penEasyphspFile::Reader Class Reference	173
7.13 ParticleZoo::penEasyphspFile::Writer Class Reference	180
7.14 ParticleZoo::PhaseSpaceFileReader Class Reference	186
7.15 ParticleZoo::PhaseSpaceFileWriter Class Reference	210
7.16 ParticleZoo::ROOT::BranchInfo Struct Reference	234
7.17 ParticleZoo::ROOT::Reader Class Reference	235
7.18 ParticleZoo::ROOT::Writer Class Reference	242
7.19 ParticleZoo::SupportedFormat Struct Reference	248
7.20 ParticleZoo::TOPASphspFile::Header Class Reference	249
7.21 ParticleZoo::TOPASphspFile::Header::DataColumn Struct Reference	258
7.22 ParticleZoo::TOPASphspFile::Header::ParticleStats Struct Reference	263
7.23 ParticleZoo::TOPASphspFile::Reader Class Reference	264
7.24 ParticleZoo::TOPASphspFile::Writer Class Reference	274
7.25 ParticleZoo::Version Struct Reference	284
8 Python API Reference	287
8.1 Installation	287
8.2 Quick Start	287
8.3 Classes	288
8.4 Functions	304

Chapter 1

ParticleZoo

A high-performance C++20 library for reading, writing, and manipulating particle phase space files across multiple Monte Carlo simulation ecosystems. [ParticleZoo](#) provides a unified API that abstracts away format-specific details, enabling seamless interoperability between different simulation codes and workflows.

1.1 Overview

[ParticleZoo](#) serves as a universal translator and processor for particle phase space data, which represents the position, momentum, energy, and other properties of particles at specific locations in Monte Carlo simulations. The library is designed around a common `Particle` data model that can represent particles from any supported format, with automatic format detection and conversion capabilities.

Key Features

- **Unified API:** Single interface to work with multiple phase space formats
- **Format Transparency:** Automatic format detection with explicit override options
- **High Performance:** Efficient binary I/O with configurable buffering
- **Parallel Processing:** Built-in support for multi-threaded reading with history or particle-balanced partitioning
- **Random Access:** Seek to specific particles within phase space files
- **Extensible Architecture:** Plugin-style registry system for adding new formats
- **Unit Consistency:** Comprehensive unit system ensures proper dimensional handling
- **Memory Efficient:** Streaming interfaces for processing large files
- **Cross-Platform:** Windows, Linux and macOS support with standard build tools
- **Python Bindings:** Optional Python package for scripting and rapid prototyping

1.2 Supported Formats

The library includes built-in support for major Monte Carlo simulation formats:

- **EGS** (EGSnrc): `.egsphsp` files in MODE0 and MODE2, including suffixed variants (`.egsphsp1`, etc.)
- **IAEA**: `.IAEAphsp` International Atomic Energy Agency format with header files
- **TOPAS**: `.phsp` files in Binary, ASCII, and Limited variants
- **penEasy**: `.dat` ASCII format from the PENELOPE simulation code
- **ROOT** (optional): `.root` files generated with the CERN ROOT framework. Includes built-in templates for TOPAS and OpenGATE generated files. Also supports custom branch mappings

Additional formats can be added through the extensible registry system without modifying core library code.

1.3 Architecture

Core Components

****Particle Class****: The central data model representing a particle with position, momentum, energy, weight, and type information. Supports both standard properties and format-specific extensions through a flexible property system.

****PhaseSpaceFileReader****: Abstract base class for reading phase space files. Implementations handle format-specific parsing while presenting a common streaming interface.

****PhaseSpaceFileWriter****: Abstract base class for writing phase space files. Handles format-specific serialization with proper history counting.

****FormatRegistry****: Plugin-style system for registering and creating readers/writers. Enables runtime format discovery and automatic format detection from file extensions.

****ByteBuffer****: High-performance binary I/O buffer for efficient reading of large files with configurable buffering strategies.

****HistoryBalancedParallelReader****: Multi-threaded reader that partitions phase space files by history count for parallel processing. Each thread receives an approximately equal share of histories.

****ParticleBalancedParallelReader****: Multi-threaded reader that partitions phase space files by particle count for parallel processing. Each thread receives an approximately equal share of particles (with history boundaries respected).

Data Model

The `Particle` class provides access to:

- **Spatial coordinates**: Position (x, y, z) in configurable units
- **Momentum**: Direction cosines and kinetic energy
- **Particle properties**: PDG particle codes, statistical weight
- **History tracking**: Original history numbers and incremental counters
- **Generation tracking**: Primary/secondary status and generation number (where available)
- **Format-specific data**: Extensible property system for specialized information

1.3 Architecture

Unit System

[ParticleZoo](#) includes a comprehensive unit system that ensures dimensional consistency across different formats. The unit system covers length, area, volume, energy, mass, time, frequency, force, pressure, charge, density, dose, and angle:

```
// Length units: nm, um, mm, cm, m, km, in, ft
float x_in_cm = particle.getX() / cm;
float y_in_mm = particle.getY() / mm;

// Energy units: eV, keV, MeV, GeV, TeV, J
float energy_MeV = particle.getKineticEnergy() / MeV;

// Dose units: Gy, cGy, rad, Sv, mSv, rem
float dose_cGy = absorbed_dose / cGy;

// Angle units: radian, deg
float angle_deg = angle / deg;
```

1.4 Building and Installation

Prerequisites

- **Operating System:** Linux, macOS, or Windows
- **Compiler / Toolchain:**
 - Linux/macOS: C++20 compatible compiler (GCC 10+, Clang 13+)
 - Windows: Visual Studio 2019 or later with C++ development tools (MSVC)
- **Build Tools:**
 - Linux/macOS: GNU Make
 - Windows: Windows Command Prompt or PowerShell (uses `build.bat`)
- **Optional Dependencies:**
 - CERN ROOT (for ROOT format support)
 - * Requires `root-config` in PATH on all platforms

Build Process

Linux and macOS

```
# Configure the build system (auto-detects compiler and dependencies)
./configure [--prefix=/your/installation/prefix]

# Build the library and tools
make          # Release build (default)
make debug    # Debug build with symbols
make release  # Explicitly build release version

# Install (optional)
make install  # defaults to /usr/local for the install PREFIX
make install PREFIX=/usr/local
make install-debug # Install debug binaries and library instead
make uninstall  # Remove an installation

# Python bindings (optional)
make install-python # Standard installation
make install-python-dev # Editable/development installation
make uninstall-python # Remove Python bindings
```

Windows

```
# Configure, build, and optionally install
build.bat [--prefix=C:\path\to\install] [debug|release]
build.bat install [--prefix=C:\path\to\install] [debug|release]
```

If GNU Make is available on Windows, the standard make targets (`make`, `make debug`, `make install`, `make clean`) also work and forward to `build.bat` automatically. The `PREFIX`, `USE_ROOT=0`, and `JOBS` make variables map to the corresponding `build.bat` flags.

Build Outputs

The build system creates the following artifacts:

Release build

- Linux/macOS: `build/gcc/release/`
- Windows: `build/msvc/release/`
 - Static library:
 - * Linux/macOS: `libparticlezoo.a`
 - * Windows: `libparticlezoo.lib`
 - Shared/dynamic library:
 - * Linux: `libparticlezoo.so`
 - * macOS: `libparticlezoo.dylib`
 - * Windows: `build/msvc/release/bin/particlezoo.dll`
 - Executables:
 - * Linux/macOS: `PHSPConvert`, `PHSPCombine`, `PHSPImage`, `PHSPSplit`
 - * Windows: `PHSPConvert.exe`, `PHSPCombine.exe`, `PHSPImage.exe`, `PHSPSplit.exe`

Debug build

- Linux/macOS: `build/gcc/debug/`
- Windows: `build/msvc/debug/`
 - Static library with debug symbols:
 - * Linux/macOS: `libparticlezoo.a`
 - * Windows: `libparticlezoo.lib`
 - Shared/dynamic library:
 - * Linux: `libparticlezoo.so`
 - * macOS: `libparticlezoo.dylib`
 - * Windows: `build/msvc/debug/bin/particlezoo.dll`
 - Debug executables:
 - * Linux/macOS: `PHSPConvert`, *etc.*
 - * Windows: `PHSPConvert.exe`, *etc.*

Installation (optional)

- Linux/macOS (with `make install`):

- Headers: `$PREFIX/include/particlezoo/`
- Libraries: `$PREFIX/lib/libparticlezoo.a` and `$PREFIX/lib/libparticlezoo.so` (`.dylib` on macOS)
- Executables: `$PREFIX/bin/PHSPConvert`, etc.
- Windows (with `build.bat` install):
 - Headers: `PREFIX%\include\particlezoo\`
 - Static Library: `PREFIX%\lib\libparticlezoo.lib`
 - Executables and DLL: `PREFIX%\bin\PHSPConvert.exe`, etc.

Configuration Options

The `configure` script (Linux/macOS) accepts the following options:

- `--prefix=PATH` - Installation prefix (default: `/usr/local`)
- `--no-root` - Disable ROOT support even if available

The `build.bat` script (Windows) accepts the following options:

- `--prefix=PATH` - Installation prefix (default: `LOCALAPPDATA%\particlezoo`)
- `--no-root` - Disable ROOT support even if available
- `-j N` or `--jobs=N` - Number of parallel compilation jobs

1.5 Using the Library

Basic Usage

Here's a simple example showing how to read from one format and write to another:

```
#include <particlezoo/PhaseSpaceFileReader.h>
#include <particlezoo/PhaseSpaceFileWriter.h>
#include <particlezoo/utilities/formats.h>

using namespace ParticleZoo;

int main() {
    // Create readers and writers - format auto-detected from extension
    auto reader = FormatRegistry::CreateReader("input.IAEaphsp");
    auto writer = FormatRegistry::CreateWriter("output.egsphsp");

    // Process all particles
    while (reader->hasMoreParticles()) {
        Particle particle = reader->getNextParticle();

        // Optionally modify particle properties
        // particle.setWeight(particle.getWeight() * 2.0);

        writer->writeParticle(particle);
    }

    // Clean up
    writer->close();
    reader->close();

    return 0;
}
```

Advanced Usage with Options

Many format readers and writers accept configuration options:

```
// Create options map for custom behavior
// Requires: #include <particlezoo/ROOT/ROOTTphsp.h>
UserOptions options;
// Example: select predefined ROOT template when reading ROOT files
options[ParticleZoo::ROOT::ROOTFormatCommand] = { std::string("TOPAS") };

// Create reader with our user options
auto reader = FormatRegistry::CreateReader("ROOT", "simulation.root", options);
```

Advanced Usage with Fixed Values

Many formats support holding certain values (e.g. X, Y, Z) constant across all particles to reduce file sizes.

```
// Create default options map
UserOptions options;

// Create the flags for the fixed values and set the Z value to be constant at 100 cm
FixedValues fixedValues;
fixedValues.zIsConstant = true;
fixedValues.constantZ = 100 * cm;

// Create writer with explicit format, options, and a fixed Z value
auto writer = FormatRegistry::CreateWriter("IAEA", "simulation.IAEaphsp", options, fixedValues);
```

Working with Particles

The `Particle` class provides extensive access to particle properties:

```
Particle p = reader->getNextParticle();

// Basic properties
float x = p.getX() / cm;           // Position in cm
float y = p.getY() / cm;
float z = p.getZ() / cm;

float energy = p.getKineticEnergy() / MeV; // Energy in MeV
float weight = p.getWeight();             // Statistical weight

// Direction (unit vector)
float dx = p.getDirectionalCosineX();
float dy = p.getDirectionalCosineY();
float dz = p.getDirectionalCosineZ();

// Generation information
bool isPrimary = p.isPrimary();
if (p.hasIntProperty(IntPropertyType::GENERATION)) {
    int generation = p.getIntProperty(IntPropertyType::GENERATION);
}
```

Format-Specific Features

Different formats support different features. The library provides access to format-specific properties:

```
// EGS-specific latch information
if (p.hasIntProperty(IntPropertyType::EGS_LATCH)) {
    int latch = p.getIntProperty(IntPropertyType::EGS_LATCH);
}

// PENELOPE-specific interaction flags
if (p.hasIntProperty(IntPropertyType::PENELOPE_ILB1)) {
    int ilb1 = p.getIntProperty(IntPropertyType::PENELOPE_ILB1);
}
```

Error Handling

The library uses standard C++ exception handling:

```
try {
    auto reader = FormatRegistry::CreateReader("nonexistent.phsp");
    // ... process particles
} catch (const std::runtime_error& e) {
    std::cerr << "Error: " << e.what() << std::endl;
} catch (const std::exception& e) {
    std::cerr << "Unexpected error: " << e.what() << std::endl;
}
```

1.6 Command-Line Tools

[ParticleZoo](#) includes several command-line utilities that demonstrate the library's capabilities:

PHSPConvert - Format Conversion

Converts phase space files between different formats with optional filtering and transformations:

```
# Auto-detect formats from file extensions
PHSPConvert input.egsphp output.IEAphsp

# Explicitly specify formats
PHSPConvert --inputFormat EGS --outputFormat IEA input.file output.file

# Limit particle count
PHSPConvert --maxParticles 1000000 input.IEAphsp output.phsp

# Project particles along their direction to a plane during conversion
# (--projectToX and --projectToY are also available)
PHSPConvert --projectToZ 100.0 input.phsp output.IEAphsp

# Filter by particle type
PHSPConvert --photonsOnly input.egsphp photons_only.IEAphsp
PHSPConvert --electronsOnly input.egsphp electrons_only.IEAphsp
PHSPConvert --filterByPDG 2212 input.phsp protons_only.phsp

# Filter by energy range (in MeV)
PHSPConvert --minEnergy 1.0 --maxEnergy 10.0 input.phsp filtered.phsp

# Filter by position (in cm)
PHSPConvert --minX -5.0 --maxX 5.0 --minY -5.0 --maxY 5.0 input.phsp central.phsp

# Filter by radial distance from central axis (in cm)
PHSPConvert --maxRadius 10.0 input.phsp within_radius.phsp

# Filter by generation (primary, secondary, etc.)
PHSPConvert --primariesOnly input.phsp primaries.phsp
PHSPConvert --excludePrimaries input.phsp secondaries.phsp
PHSPConvert --generations 1 2 input.phsp first_two_generations.phsp

# Treat warnings as errors in the exit code (also supported by PHSPImage)
PHSPConvert --errorOnWarning input.phsp output.IEAphsp
```

PHSPCombine - File Merging

Combines multiple phase space files into a single output:

```
# Combine multiple files
PHSPCombine --outputFile combined.IEAphsp file1.egsphp file2.egsphp file3.egsphp

# Mix formats during combination
PHSPCombine --outputFile result.phsp input1.IEAphsp input2.egsphp

# Preserve constant values in the output file if all input files have the same constant values
PHSPCombine --preserveConstants --outputFile result.IEAphsp input1.IEAphsp input2.IEAphsp
```

PHSPImage - Visualization and Third Party Analysis

Creates 2D particle fluence or energy fluence images from phase space data. Can output either a detailed TIFF image with raw fluence data stored in 32-bit floats (default) which can be analyzed directly in third party tools like ImageJ, or a simple bitmap BMP image with automatic contrast for easy visualization:

```
# Generate a flattened XY plane image (default)
PHSPImage beam.egsphsp fluence_map.tiff

# Project particles to a specific XY plane (e.g. 100 cm or isocenter)
PHSPImage --projectTo 100 beam.egsphsp projection.tiff

# Custom plane and energy weighting, particles are not relocated, only particles located at
# Y = 5 cm +- a default margin of 0.25 cm will be counted (margin for XZ plane can be changed
# with the --tolerance parameter)
PHSPImage --outputFormat BMP --projectionType none --plane XZ --planeLocation 5.0 --energyWeighted
simulation.IAEAphsp dose_profile.bmp

# Score different quantities (count, energy, xDir, yDir, zDir)
PHSPImage --score energy input.phsp energy_fluence.tiff

# Filter by generation
PHSPImage --primariesOnly input.phsp primaries_fluence.tiff
PHSPImage --generations 2 3 input.phsp secondaries_fluence.tiff

# Custom image dimensions and spatial boundaries
PHSPImage --imageWidth 2048 --imageHeight 2048 --square 20.0 input.phsp high_res.tiff
PHSPImage --minX -10 --maxX 10 --minY -10 --maxY 10 input.phsp custom_bounds.tiff

# EGS LATCH filtering (for EGS format files)
PHSPImage --EGS-latch-filter 0x00000001 input.egsphsp latch_filtered.tiff

# Normalize by particle count instead of history count
PHSPImage --normalizeByParticles input.phsp per_particle.tiff

# Print detailed information about the parameters being used
PHSPImage --showDetails input.phsp fluence.tiff
```

PHSPSplit - File Splitting

Splits a single phase space file into multiple (roughly) equally sized output files. History boundaries are respected, so individual files may differ slightly in size.

```
# Split a file into multiple parts
PHSPSplit --splitNumber 10 input.egsphsp

# Use short flag and specify output format
PHSPSplit -n 5 --outputFormat IAEA input.egsphsp
```

Common Writer Options

Any tool that writes phase space files also accepts these options:

```
# Hold a value constant for all written particles (X, Y, Z positions in cm;
# Px, Py, Pz directional cosines; W statistical weight)
PHSPConvert --constantZ 100.0 input.egsphsp output.IAEAphsp

# Flip directional cosines for all written particles
PHSPConvert --flipZ input.IAEAphsp flipped.IAEAphsp
```

Available options: --constantX, --constantY, --constantZ, --constantPx, --constantPy, --constantPz, --constantWeight, --flipX, --flipY, --flipZ

1.7 Examples

The `examples/` directory contains reference implementations showing how to integrate [ParticleZoo](#) into external simulation frameworks and scripting workflows.

Python Scripts (`examples/python/`)

Ready-to-run Python scripts demonstrating core [ParticleZoo](#) functionality:

- `**read_phase_space.py**` — Lists supported formats and reads particles from any phase space file. Supports all command-line options (format overrides, particle limits, ROOT branch mappings, etc.).
- `**write_phase_space.py**` — Creates a synthetic phase space file by generating sample particles programmatically. Demonstrates writer creation, particle construction, and format-specific options.

```
# List supported formats
python examples/python/read_phase_space.py --formats-only

# Read a phase space file
python examples/python/read_phase_space.py beam.IAEaphsp --limit 10

# Write a synthetic phase space file
python examples/python/write_phase_space.py output.egsphsp
```

Geant4 Phase Space Source (`examples/geant4/`)

`G4PHSPSourceAction` is a ready-to-use `G4VUserPrimaryGeneratorAction` that reads particles from any [ParticleZoo](#)-supported phase space file and injects them as primary vertices into a Geant4 simulation. Key features include:

- **Multi-threaded support** via a shared `ParticleBalancedParallelReader`, with each Geant4 worker thread processing its own partition of the file
- **Incremental history handling** to correctly reproduce the original history structure
- **Particle recycling** with automatic weight adjustment
- **Spatial transformations** — apply global translations and rotations (with configurable center of rotation) to model gantry angles, collimator rotations, etc.

Typical setup in your `ActionInitialization`:

```
// Master thread: create a shared parallel reader
auto reader = std::make_shared<ParticleZoo::ParticleBalancedParallelReader>(
    "beam.IAEaphsp", UserOptions{}, numberOfThreads);

// Worker threads: create one G4PHSPSourceAction per thread
void MyActionInitialization::Build() const {
    auto action = new ParticleZoo::G4PHSPSourceAction(reader, G4Threading::G4GetThreadId());
    action->SetTranslation(G4ThreeVector(0, 0, -100*cm));
    action->SetRecycleNumber(5);
    SetUserAction(action);
}
```

1.8 Extending the Library

Adding New Formats

To add support for a new phase space format:

1. **Implement Reader:** Inherit from `PhaseSpaceFileReader` and implement virtual methods
2. **Implement Writer:** Inherit from `PhaseSpaceFileWriter` and implement virtual methods
3. **Register Format:** Add registration call to connect file extensions with your implementations

Example registration:

```
SupportedFormat myFmt{"MyFormat", "My custom phase space format", ".myext"};
FormatRegistry::RegisterFormat (
    myFmt,
    [] (const std::string& file, const UserOptions& opts) -> std::unique_ptr<PhaseSpaceFileReader> {
        return std::make_unique<MyFormatReader>(file, opts);
    },
    [] (const std::string& file, const UserOptions& opts, const FixedValues & fixedValues) ->
        std::unique_ptr<PhaseSpaceFileWriter> {
            return std::make_unique<MyFormatWriter>(file, opts, fixedValues);
        }
);
```

Custom Particle Properties

The `Particle` class supports custom properties through the property system:

```
// Add custom integer property
particle.setIntProperty(IntPropertyType::CUSTOM, 42);

// Add custom float property
particle.setFloatProperty(FloatPropertyType::CUSTOM, 3.14f);

// Add custom boolean property
particle.setBoolProperty(BoolPropertyType::CUSTOM, true);
```

1.9 ROOT Format Support (Optional)

When compiled with ROOT support, [ParticleZoo](#) can read and write ROOT-based phase space files using predefined templates or custom branch mappings.

Predefined Templates

```
# Use TOPAS template
PHSPConvert --inputFormat ROOT --ROOT-format TOPAS input.root output.IAEaphsp

# Use OpenGATE template
PHSPConvert --inputFormat ROOT --ROOT-format OpenGATE simulation.root converted.egsphsp
```

Custom Branch Mapping

For ROOT files with non-standard branch names, you can configure custom branch mappings using the `BranchInfo` structure. Each mapping specifies the branch name and its unit conversion factor:

```
#include <particlezoo/ROOT/ROOTphsp.h>

using namespace ParticleZoo;

// Create reader with custom mappings via UserOptions
UserOptions options;
options[ROOT::ROOTTreeNameCommand] = { std::string("MyTree") };
options[ROOT::ROOTEnergyCommand] = { std::string("E_kin") };
options[ROOT::ROOTPositionXCommand] = { std::string("pos_x") };
// ... additional mappings as needed

auto reader = FormatRegistry::CreateReader("ROOT", "custom_file.root", options);
```

The command-line tools also support custom branch mapping via options:

```
PHSPConvert --inputFormat ROOT \
  --ROOT-tree-name MyTree \
  --ROOT-energy E_kin \
  --ROOT-position-x pos_x \
  --ROOT-position-y pos_y \
  --ROOT-position-z pos_z \
  --ROOT-weight stat_weight \
  input.root output.phsp
```

Available branch mapping options:

- `--ROOT-tree-name <name>` - ROOT tree name
- `--ROOT-energy <branch>` - Energy branch
- `--ROOT-weight <branch>` - Statistical weight branch
- `--ROOT-position-x/y/z <branch>` - Position branches
- `--ROOT-cosine-x/y/z <branch>` - Direction cosines
- `--ROOT-cosine-z-sign <branch>` - Boolean flag for Z-direction sign
- `--ROOT-pdg-code <branch>` - Particle type identifier
- `--ROOT-history-number <branch>` - History counter

Random Access

Readers support random access to particles within phase space files:

```
auto reader = FormatRegistry::CreateReader("simulation.IAEAphsp");

// Move to a specific particle index (zero-based)
reader->moveToParticle(1000000);

// Continue reading from that position
while (reader->hasMoreParticles()) {
    Particle p = reader->getNextParticle();
    // ...
}
```

Parallel Processing

For large-scale processing, use the parallel readers to distribute work across multiple threads:

```
#include <particlezoo/parallel/ParticleBalancedParallelReader.h>
#include <thread>
#include <vector>

// Create a parallel reader with 8 threads
const size_t numThreads = 8;
ParticleBalancedParallelReader parallelReader("large_file.IAEAphsp", {}, numThreads);

// Each thread processes its partition independently
std::vector<std::thread> threads;
for (size_t threadId = 0; threadId < numThreads; threadId++) {
    threads.emplace_back([&parallelReader, threadId]() {
        while (parallelReader.hasMoreParticles(threadId)) {
            Particle p = parallelReader.getNextParticle(threadId);
            // Process particle...
        }
    });
}

// Wait for all threads to complete
for (auto& t : threads) {
    t.join();
}
```

1.10 Python Bindings

[ParticleZoo](#) includes optional Python bindings for scripting and rapid prototyping.

Installation

Linux/macOS (using Makefile)

```
# Standard installation (uses virtual env if active, otherwise user site-packages)
make install-python

# Development/editable installation
make install-python-dev

# Uninstall
make uninstall-python
```

Windows (using pip)

```
# From the repository root
python -m pip install python          # Standard install
python -m pip install -e python      # Editable install
```

See the [python/README.md](#) for detailed installation options and full API documentation.

```
import particlezoo as pz

# Create a particle programmatically
p = pz.Particle(pz.ParticleType.Electron, 6.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0)
print(f"Created particle with energy: {p.kinetic_energy} MeV")

# Read particles from one format and write to another
reader = pz.create_reader("/path/to/input.IAEaphsp")
writer = pz.create_writer("/path/to/output.egsphsp")

for i, particle in enumerate(reader):
    # Optionally modify the particle
    # particle.weight *= 2.0

    # Write to output file
    writer.write_particle(particle)

    if i % 100000 == 0:
        print(f"Processed {i} particles...")

writer.close()
print("Conversion complete!")
```

1.11 Performance Considerations

Optimization Tips

- Use binary formats when possible for faster I/O
- Consider particle limits (`--maxParticles`) for testing and prototyping
- Enable compiler optimizations (`make` or `make release`) for production use (do not use `make debug`)
- Use parallel readers for multi-threaded processing of large files

1.12 Troubleshooting

Common Issues

Build Problems:

- `"config.status not found"` → Run `./configure` before `make`
- `"The C++ standard in this build does not match ROOT configuration"` → The ROOT installation on your system was compiled with a different C++ standard than [ParticleZoo](#). It may still work, but it is not supported. Either rebuild both with the same C++ standard or accept the risk.
- `"ROOT support: no"` → Ensure `root-config` is in PATH and re-run `./configure`
- `"checking whether g++ accepts -std=c++20... no"` → Update compiler (GCC 10+ or Clang 13+)

Runtime Issues:

- `"Unknown format"` → Use `--inputFormat` to explicitly specify format
- `"File not found"` → Check file paths and permissions

Getting Help

For additional support:

1. Use `--help` option to see a list of available commands
2. Use `--formats` option to verify supported formats at runtime
3. Consult the reference manual and test summary report PDFs in the `docs/` directory
4. Raise an issue on GitHub or post questions in the GitHub discussions

Chapter 2

License

MIT License

Copyright (c) 2025 Daniel O'Brien

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Chapter 3

Deprecated List

Member [ParticleZoo::IS_SECONDARY_PARTICLE](#)

Since v1.1.0. Use [IntPropertyType::GENERATION](#) instead.

Chapter 4

Hierarchical Index

4.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

ParticleZoo::ByteBuffer	42
ParticleZoo::FixedValues	73
ParticleZoo::FormatRegistry	75
ParticleZoo::HistoryBalancedParallelReader	83
ParticleZoo::IAEAphspFile::IAEAHeader	91
ParticleZoo::Particle	144
ParticleZoo::ParticleBalancedParallelReader	164
ParticleZoo::PhaseSpaceFileReader	186
ParticleZoo::EGSphspFile::Reader	55
ParticleZoo::IAEAphspFile::Reader	126
ParticleZoo::ROOT::Reader	235
ParticleZoo::TOPASphspFile::Reader	264
ParticleZoo::penEasyphspFile::Reader	173
ParticleZoo::PhaseSpaceFileWriter	210
ParticleZoo::EGSphspFile::Writer	64
ParticleZoo::IAEAphspFile::Writer	134
ParticleZoo::ROOT::Writer	242
ParticleZoo::TOPASphspFile::Writer	274
ParticleZoo::penEasyphspFile::Writer	180
ParticleZoo::ROOT::BranchInfo	234
ParticleZoo::SupportedFormat	248
ParticleZoo::TOPASphspFile::Header	249
ParticleZoo::TOPASphspFile::Header::DataColumn	258
ParticleZoo::TOPASphspFile::Header::ParticleStats	263
ParticleZoo::Version	284

Chapter 5

Class Index

5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

ParticleZoo::ByteBuffer	Byte buffer used to improve I/O performance for reading and writing binary and text data	42
ParticleZoo::EGSphspFile::Reader	Reader class for EGS phase space files	55
ParticleZoo::EGSphspFile::Writer	Writer class for EGS phase space files	64
ParticleZoo::FixedValues	Structure defining constant (fixed) values for particle properties	73
ParticleZoo::FormatRegistry	Singleton registry for managing phase space file format readers and writers	75
ParticleZoo::HistoryBalancedParallelReader	Multi-threaded phase space file reader for parallel particle processing	83
ParticleZoo::IAEAphspFile::IAEAHeader	Header manager for IAEA phase space files	91
ParticleZoo::IAEAphspFile::Reader	Reader for IAEA format phase space files	126
ParticleZoo::IAEAphspFile::Writer	Writer for IAEA format phase space files	134
ParticleZoo::Particle	Represents a particle in phase space	144
ParticleZoo::ParticleBalancedParallelReader	Multi-threaded phase space file reader for parallel particle processing	164
ParticleZoo::penEasyphspFile::Reader	Reader for penEasy format phase space files	173
ParticleZoo::penEasyphspFile::Writer	Writer for penEasy format phase space files	180
ParticleZoo::PhaseSpaceFileReader	Base class for reading phase space files	186
ParticleZoo::PhaseSpaceFileWriter	Base class for writing phase space files	210
ParticleZoo::ROOT::BranchInfo	Configuration for ROOT TTree branch mapping	234
ParticleZoo::ROOT::Reader	ROOT format phase space file reader	235
ParticleZoo::ROOT::Writer	ROOT format phase space file writer	242
ParticleZoo::SupportedFormat	Structure describing a supported phase space file format	248
ParticleZoo::TOPASphspFile::Header	Header for TOPAS phase space files	249
ParticleZoo::TOPASphspFile::Header::DataColumn	Column definition for TOPAS phase space files	258

5.1 Class List

ParticleZoo::TOPASphspFile::Header::ParticleStats	
Statistics tracking for individual particle types for TOPAS phase space files	263
ParticleZoo::TOPASphspFile::Reader	
Reader for TOPAS phase space files	264
ParticleZoo::TOPASphspFile::Writer	
Writer for TOPAS phase space files	274
ParticleZoo::Version	
Version information and metadata for the ParticleZoo library	284

Chapter 6

Namespace Documentation

6.1 ParticleZoo Namespace Reference

Main namespace for the [ParticleZoo](#) phase space file processing library.

Classes

- class [ByteBuffer](#)
Byte buffer used to improve I/O performance for reading and writing binary and text data.
- struct [FixedValues](#)
Structure defining constant (fixed) values for particle properties.
- class [FormatRegistry](#)
Singleton registry for managing phase space file format readers and writers.
- class [HistoryBalancedParallelReader](#)
Multi-threaded phase space file reader for parallel particle processing.
- class [Particle](#)
Represents a particle in phase space.
- class [ParticleBalancedParallelReader](#)
Multi-threaded phase space file reader for parallel particle processing.
- class [PhaseSpaceFileReader](#)
Base class for reading phase space files.
- class [PhaseSpaceFileWriter](#)
Base class for writing phase space files.
- struct [SupportedFormat](#)
Structure describing a supported phase space file format.
- struct [Version](#)
Version information and metadata for the [ParticleZoo](#) library.

Typedefs

- using **byte** = unsigned char
Type alias for unsigned byte (8 bits)
- using **signed_byte** = char
Type alias for signed byte (8 bits)

6.1 ParticleZoo Namespace Reference

Enumerations

- enum class [BoolPropertyType](#) { [INVALID](#) , [IS_MULTIPLE_CROSSER](#) , [IS_SECONDARY_PARTICLE](#) , [CUSTOM](#) }
Enumeration of boolean property types for particles.
- enum class [ByteOrder](#) { [LittleEndian](#) = 1234 , [BigEndian](#) = 4321 , [PDPEndian](#) = 3412 }
Enumeration of byte ordering schemes for multi-byte data types.
- enum class [FloatPropertyType](#) { [INVALID](#) , [XLAST](#) , [YLAST](#) , [ZLAST](#) , [CUSTOM](#) }
Enumeration of floating-point property types for particles.
- enum class [FormatType](#) { [BINARY](#) , [ASCII](#) , [NONE](#) }
Enumeration of file format types.
- enum class [IntPropertyType](#) { [INVALID](#) , [INCREMENTAL_HISTORY_NUMBER](#) , [EGS_LATCH](#) , [PENELOPE_ILB1](#) , [PENELOPE_ILB2](#) , [PENELOPE_ILB3](#) , [PENELOPE_ILB4](#) , [PENELOPE_ILB5](#) , [GENERATION](#) , [CUSTOM](#) }
Enumeration of integer property types for particles.
- enum class [ParticleType](#) : std::int32_t { }
Strongly-typed enumeration of particle types with PDG codes.

Functions

- const std::vector< std::pair< std::string_view, [ParticleType](#) > > & [getAllParticleTypes](#) ()
Get a complete list of particle names and types.
- [ParticleType](#) [getParticleTypeFromPDGID](#) (std::int32_t pdg) noexcept
Convert PDG identification code to ParticleType enumeration.
- constexpr std::string_view [getParticleTypeName](#) ([ParticleType](#) t)
Get human-readable name for a particle type.
- std::int32_t [getPDGIDFromParticleType](#) ([ParticleType](#) type) noexcept
Convert ParticleType enumeration to PDG identification code.
- std::ostream & [operator<<](#) (std::ostream &os, const [ByteBuffer](#) &buffer)

Variables

- constexpr std::size_t [DEFAULT_BUFFER_SIZE](#) = 1048576
Default buffer size (1MiB)
- constexpr [ByteOrder](#) [HOST_BYTE_ORDER](#)
The byte order of the host system.

6.1.1 Detailed Description

Main namespace for the [ParticleZoo](#) phase space file processing library.

[ParticleZoo](#) is a comprehensive C++ library for reading, writing, and manipulating phase space files from various Monte Carlo radiation transport codes. The library provides a unified interface for working with different file formats while preserving format-specific features and metadata.

The namespace contains all classes, functions, and utilities, including:

- Phase space file I/O operations
- [Particle](#) data structures and property management
- Universal interface for reading/writing to/from different phase space formats
- Format-specific readers and writers

Supported formats include EGS, IAEA, TOPAS, and others.

6.1.2 Enumeration Type Documentation

6.1.2.1 BoolPropertyType

```
enum class ParticleZoo::BoolPropertyType [strong]
```

Enumeration of boolean property types for particles.

Defines standardized boolean flags that can be associated with particles from different Monte Carlo simulation codes.

Enumerator

INVALID	Invalid property type.
IS_MULTIPLE_CROSSER	Flag indicating that the particle crossed the phase space plane multiple times (assuming the phase space is planar)
IS_SECONDARY_PARTICLE	Flag indicating that the particle is a secondary. Deprecated Since v1.1.0. Use IntPropertyType::GENERATION instead.
CUSTOM	Custom boolean property type, can be used for any user-defined purpose.

6.1 ParticleZoo Namespace Reference

6.1.2.2 ByteOrder

```
enum class ParticleZoo::ByteOrder [strong]
```

Enumeration of byte ordering schemes for multi-byte data types.

Defines the different ways multi-byte values can be stored in memory, for cross-platform compatibility when reading/writing binary data files.

Enumerator

LittleEndian	Least significant byte first.
BigEndian	Most significant byte first.
PDPEndian	Mixed endian.

6.1.2.3 FloatPropertyType

```
enum class ParticleZoo::FloatPropertyType [strong]
```

Enumeration of floating-point property types for particles.

Defines standardized float properties that can be associated with particles from different Monte Carlo simulation codes.

Enumerator

INVALID	Invalid property type, used for error checking.
XLAST	EGS-specific XLAST variable, for photons it is the X position of the last interaction, for electrons/positrons it is the X position it (or it's ancestor) was created at by a photon.
YLAST	EGS-specific YLAST variable, for photons it is the Y position of the last interaction, for electrons/positrons it is the Y position it (or it's ancestor) was created at by a photon.
ZLAST	EGS-specific ZLAST variable, for photons it is the Z position of the last interaction, for electrons/positrons it is the Z position it (or it's ancestor) was created at by a photon.
CUSTOM	Custom float property type, can be used for any user-defined purpose.

6.1.2.4 FormatType

```
enum class ParticleZoo::FormatType [strong]
```

Enumeration of file format types.

Enumerator

BINARY	Binary format.
ASCII	ASCII text format.
NONE	Used for when ParticleZoo will not be responsible for reading/writing (e.g. ROOT)

6.1.2.5 IntPropertyType

```
enum class ParticleZoo::IntPropertyType [strong]
```

Enumeration of integer property types for particles.

Defines standardized integer properties that can be associated with particles from different Monte Carlo simulation codes.

Enumerator

INVALID	Invalid property type, used for error checking.
INCREMENTAL_HISTORY_NUMBER	Sequential history number for tracking, tracks the number of new histories since the last particle was recorded.
EGS_LATCH	EGS-specific latch variable (see BEAMnrc User Manual, Chapter 8 for details)
PENELOPE_ILB1	PENELOPE ILB array value 1, corresponds to the generation of the particle (1 for primary, 2 for secondary, etc.)
PENELOPE_ILB2	PENELOPE ILB array value 2, corresponds to the particle type of the particle's parent (applies only if ILB1 > 1)
PENELOPE_ILB3	PENELOPE ILB array value 3, corresponds to the interaction type that created the particle (applies only if ILB1 > 1)
PENELOPE_ILB4	PENELOPE ILB array value 4, is non-zero if the particle is created by atomic relaxation and corresponds to the atomic transistion that created the particle.
PENELOPE_ILB5	PENELOPE ILB array value 5, a user-defined value which is passed on to all descendant particles created by this particle.
GENERATION	Generation of the particle (1 for primary, 2 for secondary, etc.)
CUSTOM	Custom integer property type, can be used for any user-defined purpose.

6.1.2.6 ParticleType

```
enum class ParticleZoo::ParticleType : std::int32_t [strong]
```


6.1 ParticleZoo Namespace Reference

Strongly-typed enumeration of particle types with PDG codes.

This enum class provides type-safe access to PDG particle codes while maintaining the standardized integer values. Each enumerator corresponds to a specific particle type with its official PDG identification number.

Special values internal to [ParticleZoo](#) (these codes should not be written to files):

- Unsupported (99): For particle types that are not supported by [ParticleZoo](#)
- PseudoParticle (98): For pseudo-particles containing simulation metadata

The enum uses `std::int32_t` as the underlying type to match PDG code specifications and handle the full range of positive and negative values.

Note

Generated automatically from `PARTICLE_LIST` macro

Enumerator

Unsupported	Unknown or non-standard particle type.
PseudoParticle	Simulation-specific pseudo-particle.

6.1.3 Function Documentation

6.1.3.1 `getAllParticleTypes()`

```
const std::vector< std::pair< std::string_view, ParticleType > > & ParticleZoo::getAllParticleTypes ( ) [inline]
```

Get a complete list of particle names and types.

Exposes all `ParticleType` values generated by `PARTICLE_LIST` for use in bindings. Returns a stable reference to a static vector of pairs (name, `ParticleType`).

6.1.3.2 `getParticleTypeFromPDGID()`

```
ParticleType ParticleZoo::getParticleTypeFromPDGID (
    std::int32_t pdg ) [inline], [noexcept]
```

Convert PDG identification code to `ParticleType` enumeration.

Performs efficient lookup from standardized PDG integer codes to the corresponding strongly-typed `ParticleType` enumeration value. This function provides the primary interface for particle identification in Monte Carlo simulation data processing.

Parameters

<i>pdg</i>	The PDG identification code (standardized integer)
------------	--

Returns

ParticleType enumeration value, or [ParticleType::Unsupported](#) for unknown codes

Note

Uses compile-time generated switch statement for O(1) lookup performance

6.1.3.3 getParticleTypeName()

```
constexpr std::string_view ParticleZoo::getParticleTypeName (
    ParticleType t ) [constexpr]
```

Get human-readable name for a particle type.

Returns the string representation of a ParticleType enumeration value, providing descriptive names for particles in logging, debugging, and user interface contexts.

Parameters

<i>t</i>	The ParticleType enumeration value
----------	------------------------------------

Returns

std::string_view containing the particle name (compile-time constant)

Note

Marked constexpr for compile-time evaluation

Returns string_view for efficiency

6.1.3.4 getPDGIDFromParticleType()

```
std::int32_t ParticleZoo::getPDGIDFromParticleType (
    ParticleType type ) [inline], [noexcept]
```

6.1 ParticleZoo Namespace Reference

Convert ParticleType enumeration to PDG identification code.

Extracts the standardized PDG integer code from a ParticleType enumeration value. This provides the inverse operation to [getParticleTypeFromPDGID\(\)](#), enabling conversion from strongly-typed enums back to the integer codes required by some phase space formats.

Parameters

<i>type</i>	The ParticleType enumeration value
-------------	------------------------------------

Returns

std::int32_t PDG identification code

6.1.3.5 operator<<()

```
std::ostream & ParticleZoo::operator<< (  
    std::ostream & os,  
    const ByteBuffer & buffer ) [inline]
```

Parameters

<i>os</i>	The output stream to write to
<i>buffer</i>	The ByteBuffer to write from

Returns

std::ostream& Reference to the output stream for chaining

6.1.4 Variable Documentation

6.1.4.1 HOST_BYTE_ORDER

```
constexpr ByteOrder ParticleZoo::HOST_BYTE_ORDER [constexpr]
```

Initial value:

```
=  
(std::endian::native == std::endian::little) ? ByteOrder::LittleEndian :  
(std::endian::native == std::endian::big)   ? ByteOrder::BigEndian :  
                                              ByteOrder::PDP\_ENDIAN
```

The byte order of the host system.

Automatically determined at compile time based on the system's native byte order.

Chapter 7

Class Documentation

7.1 ParticleZoo::ByteBuffer Class Reference

Byte buffer used to improve I/O performance for reading and writing binary and text data.

```
#include <particlezoo/ByteBuffer.h>
```

Public Member Functions

- [ByteBuffer](#) (const std::span< const [byte](#) > [data](#), [ByteOrder](#) [byteOrder](#)=HOST_BYTE_ORDER)
Create a [ByteBuffer](#) from a span of existing data.
- [ByteBuffer](#) (std::size_t [bufferSize](#)=DEFAULT_BUFFER_SIZE, [ByteOrder](#) [byteOrder](#)=HOST_BYTE_ORDER)
Create an empty [ByteBuffer](#) with a fixed capacity.
- std::size_t [appendData](#) ([ByteBuffer](#) &[buffer](#), bool [ignoreOffset](#)=false)
Append data from another [ByteBuffer](#) to this buffer.
- std::size_t [appendData](#) (std::istream &[stream](#))
Append data from an input stream to the existing buffer content.
- std::size_t [capacity](#) () const
Get the total capacity of the buffer.
- void [clear](#) ()
Reset the buffer, resetting the offset and length to 0.
- void [compact](#) ()
Compact the buffer by moving unread data to the beginning.
- const [byte](#) * [data](#) () const
Get a pointer to the raw buffer data.
- void [expand](#) ()
Expand the buffer to its full capacity, filling unused space with zeros.
- [ByteOrder](#) [getByteOrder](#) () const
Get the current byte order setting.
- std::size_t [length](#) () const
Get the length of valid data in the buffer.
- void [moveTo](#) (std::size_t [offset](#))
Move the read/write offset to a specific position in the buffer.
- std::span< const [byte](#) > [peekBytes](#) (std::size_t [len](#))
Peek at a span of bytes from the buffer without advancing the offset.
- template<typename T >
T [read](#) ()
Read a primitive type T from the buffer with automatic byte order conversion.
- std::span< const [byte](#) > [readBytes](#) (std::size_t [len](#))
Read a span of bytes from the buffer.
- std::string [readLine](#) (bool [allowMissingTerminator](#)=false)
Read a line of ASCII text from the buffer.

7.1 ParticleZoo::ByteBuffer Class Reference

- `std::string readString ()`
Read a null-terminated string from the buffer.
- `std::string readString (std::size_t stringLength)`
Read a string of specified length from the buffer.
- `std::size_t remainingToRead () const`
Get the number of bytes remaining to be read from current offset.
- `std::size_t remainingToWrite () const`
Get the number of bytes available for writing.
- `void setByteOrder (ByteOrder byteOrder)`
Set the byte order for interpreting multi-byte data types.
- `std::size_t setData (std::istream &stream)`
Initialize the buffer with data from an input stream.
- `std::size_t setData (std::span< const byte > data)`
Initialize the buffer with data from a span.
- `template<typename T >`
`void write (const T &value)`
Write a primitive type T to the buffer with automatic byte order conversion.
- `void writeBytes (std::span< const byte > data)`
Write a span of bytes to the buffer.
- `void writeString (const std::string &str, bool includeNullTerminator=false)`
Write a string to the buffer.

7.1.1 Detailed Description

Byte buffer used to improve I/O performance for reading and writing binary and text data.

`ByteBuffer` provides efficient buffered I/O operations with automatic byte order conversion for cross-platform compatibility. It supports both reading from and writing to the buffer, with automatic capacity management and various data type read/write operations.

The buffer maintains both a current offset (read/write position) and a length (amount of valid data), allowing for flexible positioning and partial reads/writes.

7.1.2 Constructor & Destructor Documentation

7.1.2.1 ByteBuffer() [1/2]

```
ParticleZoo::ByteBuffer::ByteBuffer (  
    std::size_t bufferSize = DEFAULT_BUFFER_SIZE,  
    ByteOrder byteOrder = HOST_BYTE_ORDER ) [inline]
```

Create an empty `ByteBuffer` with a fixed capacity.

Note

The capacity must be greater than zero.

Parameters

<i>bufferSize</i>	The maximum capacity of the buffer in bytes (default: DEFAULT_BUFFER_SIZE)
<i>byteOrder</i>	The byte order for multi-byte data types (default: HOST_BYTE_ORDER)

Exceptions

<code>std::runtime_error</code>	if bufferSize is zero
---------------------------------	-----------------------

7.1.2.2 ByteBuffer() [2/2]

```
ParticleZoo::ByteBuffer::ByteBuffer (
    const std::span< const byte > data,
    ByteOrder byteOrder = HOST_BYTE_ORDER ) [inline]
```

Create a [ByteBuffer](#) from a span of existing data.

The buffer is initialized with a copy of the provided data.

Parameters

<i>data</i>	A span containing the initial data to copy into the buffer
<i>byteOrder</i>	The byte order for multi-byte data types (default: HOST_BYTE_ORDER)

7.1.3 Member Function Documentation

7.1.3.1 appendData() [1/2]

```
std::size_t ParticleZoo::ByteBuffer::appendData (
    ByteBuffer & buffer,
    bool ignoreOffset = false ) [inline]
```

Append data from another [ByteBuffer](#) to this buffer.

Copies data from the source buffer and appends it after the current data.

Parameters

<i>buffer</i>	The source ByteBuffer to copy data from
<i>ignoreOffset</i>	If true, copies all data from source; if false, only unread data

7.1 ParticleZoo::ByteBuffer Class Reference

Returns

`std::size_t` The number of bytes appended

Exceptions

<code>std::runtime_error</code>	if the combined data exceeds buffer capacity
---------------------------------	--

7.1.3.2 appendData() [2/2]

```
std::size_t ParticleZoo::ByteBuffer::appendData (  
    std::istream & stream ) [inline]
```

Append data from an input stream to the existing buffer content.

Reads additional data from the stream and appends it after the current data. Does not modify the current offset.

Parameters

<code><i>stream</i></code>	The input stream to read from
----------------------------	-------------------------------

Returns

`std::size_t` The number of bytes appended from the stream

Exceptions

<code>std::runtime_error</code>	if buffer is full or no data could be read
---------------------------------	--

7.1.3.3 capacity()

```
std::size_t ParticleZoo::ByteBuffer::capacity ( ) const [inline]
```

Get the total capacity of the buffer.

Returns

`std::size_t` The maximum number of bytes the buffer can hold

7.1.3.4 compact()

```
void ParticleZoo::ByteBuffer::compact ( ) [inline]
```

Compact the buffer by moving unread data to the beginning.

Shifts any unread data (from current offset to end) to the start of the buffer and updates the length and offset accordingly. Useful for reclaiming space after partial reads.

7.1.3.5 data()

```
const byte * ParticleZoo::ByteBuffer::data ( ) const [inline]
```

Get a pointer to the raw buffer data.

Returns

const byte* Pointer to the beginning of the buffer data

7.1.3.6 expand()

```
void ParticleZoo::ByteBuffer::expand ( ) [inline]
```

Expand the buffer to its full capacity, filling unused space with zeros.

Extends the data length to the full buffer capacity by filling the remaining space with zero-ed bytes.

7.1.3.7 getByteOrder()

```
ByteOrder ParticleZoo::ByteBuffer::getByteOrder ( ) const [inline]
```

Get the current byte order setting.

Returns

ByteOrder The byte order used for multi-byte data types

7.1 ParticleZoo::ByteBuffer Class Reference

7.1.3.8 length()

```
std::size_t ParticleZoo::ByteBuffer::length ( ) const [inline]
```

Get the length of valid data in the buffer.

Returns

std::size_t The number of bytes of valid data

7.1.3.9 moveTo()

```
void ParticleZoo::ByteBuffer::moveTo (
    std::size_t offset ) [inline]
```

Move the read/write offset to a specific position in the buffer.

Parameters

<i>offset</i>	The new offset position (must be $\leq$ current data length)
---------------	--

Exceptions

<i>std::runtime_error</i>	if offset exceeds the current data length
---------------------------	---

7.1.3.10 peekBytes()

```
std::span< const byte > ParticleZoo::ByteBuffer::peekBytes (
    std::size_t len ) [inline]
```

Peek at a span of bytes from the buffer without advancing the offset.

Returns a view of the requested bytes without copying. Does not modify the current offset.

Parameters

<i>len</i>	The number of bytes to peek at
------------	--------------------------------

Returns

`std::span<const byte>` A span view of the requested bytes

Exceptions

<code>std::runtime_error</code>	if insufficient data is available
---------------------------------	-----------------------------------

7.1.3.11 read()

```
template<typename T >
T ParticleZoo::ByteBuffer::read ( ) [inline]
```

Read a primitive type `T` from the buffer with automatic byte order conversion.

Reads `sizeof(T)` bytes from the current offset and converts the byte order if necessary. Advances the offset by `sizeof(T)`.

Template Parameters

<code>T</code>	The primitive type to read (must be trivially copyable)
----------------	---

Returns

`T` The value read from the buffer

Exceptions

<code>std::runtime_error</code>	if insufficient data is available
---------------------------------	-----------------------------------

7.1.3.12 readBytes()

```
std::span< const byte > ParticleZoo::ByteBuffer::readBytes (
    std::size_t len ) [inline]
```

Read a span of bytes from the buffer.

Returns a view of the requested bytes without copying. Advances the offset by the requested length.

7.1 ParticleZoo::ByteBuffer Class Reference

Parameters

<i>len</i>	The number of bytes to read
------------	-----------------------------

Returns

`std::span<const byte>` A span view of the requested bytes

Exceptions

<i>std::runtime_error</i>	if insufficient data is available
---------------------------	-----------------------------------

7.1.3.13 readLine()

```
std::string ParticleZoo::ByteBuffer::readLine (  
    bool allowMissingTerminator = false ) [inline]
```

Read a line of ASCII text from the buffer.

Reads characters until a newline ('
') is found. Automatically handles Windows-style line endings by removing trailing '\r'. Advances the offset past the newline.

Parameters

<i>allowMissingTerminator</i>	If true and no newline is found, the remaining data is returned as the final line instead of throwing
-------------------------------	---

Returns

`std::string` The line read from the buffer (without newline characters)

Exceptions

<i>std::runtime_error</i>	if newline is not found (unless <code>allowMissingTerminator</code> is true) or no data is available
---------------------------	--

7.1.3.14 readString() [1/2]

```
std::string ParticleZoo::ByteBuffer::readString ( ) [inline]
```

Read a null-terminated string from the buffer.

Reads characters until a null terminator ('\0') is found. Advances the offset past the null terminator.

Returns

std::string The string read from the buffer (without null terminator)

Exceptions

<i>std::runtime_error</i>	if null terminator is not found or insufficient data
---------------------------	--

7.1.3.15 readString() [2/2]

```
std::string ParticleZoo::ByteBuffer::readString (
    std::size_t stringLength ) [inline]
```

Read a string of specified length from the buffer.

Reads exactly the specified number of characters. Advances the offset by the string length.

Parameters

<i>stringLength</i>	The number of characters to read
---------------------	----------------------------------

Returns

std::string The string read from the buffer

Exceptions

<i>std::runtime_error</i>	if insufficient data is available
---------------------------	-----------------------------------

7.1 ParticleZoo::ByteBuffer Class Reference

7.1.3.16 remainingToRead()

```
std::size_t ParticleZoo::ByteBuffer::remainingToRead ( ) const [inline]
```

Get the number of bytes remaining to be read from current offset.

Returns

std::size_t The number of unread bytes (length - offset)

7.1.3.17 remainingToWrite()

```
std::size_t ParticleZoo::ByteBuffer::remainingToWrite ( ) const [inline]
```

Get the number of bytes available for writing.

Returns

std::size_t The remaining capacity (capacity - length)

7.1.3.18 setByteOrder()

```
void ParticleZoo::ByteBuffer::setByteOrder (
    ByteOrder byteOrder ) [inline]
```

Set the byte order for interpreting multi-byte data types.

Parameters

<i>byteOrder</i>	The byte order to use for subsequent read/write operations
------------------	--

7.1.3.19 setData() [1/2]

```
std::size_t ParticleZoo::ByteBuffer::setData (
    std::istream & stream ) [inline]
```

Initialize the buffer with data from an input stream.

Reads up to the buffer's capacity from the stream. Replaces any existing data. Resets the offset to 0.

Parameters

<i>stream</i>	The input stream to read from
---------------	-------------------------------

Returns

`std::size_t` The number of bytes read from the stream

Exceptions

<i>std::runtime_error</i>	if no data could be read from the stream
---------------------------	--

7.1.3.20 setData() [2/2]

```
std::size_t ParticleZoo::ByteBuffer::setData (
    std::span< const byte > data ) [inline]
```

Initialize the buffer with data from a span.

Replaces any existing data in the buffer. Resets the offset to 0.

Parameters

<i>data</i>	A span containing the data to copy into the buffer
-------------	--

Returns

`std::size_t` The number of bytes copied

Exceptions

<i>std::runtime_error</i>	if data size exceeds buffer capacity
---------------------------	--------------------------------------

7.1.3.21 write()

```
template<typename T >
void ParticleZoo::ByteBuffer::write (
    const T & value ) [inline]
```


7.1 ParticleZoo::ByteBuffer Class Reference

Write a primitive type T to the buffer with automatic byte order conversion.

Converts the value to the buffer's byte order if necessary and writes sizeof(T) bytes. Advances the offset by sizeof(T) and updates the length if necessary.

Template Parameters

<i>T</i>	The primitive type to write (must be trivially copyable)
----------	--

Parameters

<i>value</i>	The value to write to the buffer
--------------	----------------------------------

Exceptions

<i>std::runtime_error</i>	if insufficient space is available
---------------------------	------------------------------------

7.1.3.22 writeBytes()

```
void ParticleZoo::ByteBuffer::writeBytes (
    std::span< const byte > data ) [inline]
```

Write a span of bytes to the buffer.

Copies the bytes from the span to the buffer. Advances the offset by the data size and updates the length if necessary.

Parameters

<i>data</i>	A span containing the bytes to write
-------------	--------------------------------------

Exceptions

<i>std::runtime_error</i>	if insufficient space is available
---------------------------	------------------------------------

7.1.3.23 writeString()

```
void ParticleZoo::ByteBuffer::writeString (
    const std::string & str,
    bool includeNullTerminator = false ) [inline]
```

Write a string to the buffer.

Writes the string's characters with or without (default) a null terminator. Advances the offset by the string length and updates the length if necessary.

Parameters

<i>str</i>	The string to write to the buffer
<i>includeNullTerminator</i>	If true, appends a null terminator after the string

Exceptions

<i>std::runtime_error</i>	if insufficient space is available
---------------------------	------------------------------------

The documentation for this class was generated from the following file:

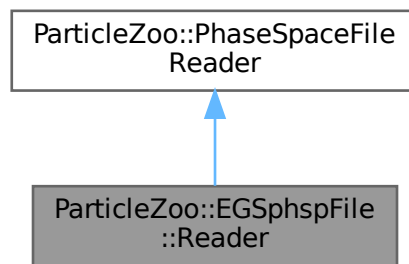
- include/particlezoo/ByteBuffer.h

7.2 ParticleZoo::EGSphspFile::Reader Class Reference

[Reader](#) class for EGS phase space files.

```
#include <particlezoo/egs/egsphspFile.h>
```

Inheritance diagram for ParticleZoo::EGSphspFile::Reader:



Public Member Functions

- [Reader](#) (const std::string &fileName, const UserOptions &options=UserOptions{})
Construct a new EGS phase space file reader.
- EGSLATCHOPTION [getLATCHOption](#) () const
Get the LATCH interpretation option.
- float [getMaxKineticEnergy](#) () const
Get the maximum kinetic energy of particles in the file.
- float [getMinElectronEnergy](#) () const
Get the minimum electron energy threshold used in the simulation.
- EGSMODE [getMode](#) () const
Get the EGS file mode (MODE0 or MODE2).
- std::uint64_t [getNumberOfOriginalHistories](#) () const override
Get the number of original Monte Carlo histories.
- std::uint64_t [getNumberOfParticles](#) () const override
Get the total number of particles in the phase space file.
- unsigned int [getNumberOfPhotons](#) () const
Get the number of photons in the phase space.

Public Member Functions inherited from [ParticleZoo::PhaseSpaceFileReader](#)

- [PhaseSpaceFileReader](#) (const std::string &phspFormat, const std::string &fileName, const UserOptions &userOptions, [FormatType](#) formatType=[FormatType::BINARY](#), const [FixedValues](#) fixedValues=[FixedValues\(\)](#), unsigned int bufferSize=[DEFAULT_BUFFER_SIZE](#))
Construct a new Phase Space File Reader object.
- virtual [~PhaseSpaceFileReader](#) ()
Destroy the Phase Space File Reader object.
- void [close](#) ()
Close the phase space file and clean up resources.
- float [getConstantPx](#) () const
Get the constant X-component of the direction unit vector (if constant).
- float [getConstantPy](#) () const
Get the constant Y-component of the direction unit vector (if constant).
- float [getConstantPz](#) () const
Get the constant Z-component of the direction unit vector (if constant).
- float [getConstantWeight](#) () const
Get the constant statistical weight value (if constant).
- float [getConstantX](#) () const
Get the constant X coordinate value (if constant).
- float [getConstantY](#) () const
Get the constant Y coordinate value (if constant).
- float [getConstantZ](#) () const
Get the constant Z coordinate value (if constant).
- const std::string [getFileName](#) () const
Get the filename of the phase space file being read.
- std::uint64_t [getFileSize](#) () const
Get the size of the phase space file in bytes.
- const [FixedValues](#) [getFixedValues](#) () const
Get the fixed values configuration.
- virtual std::uint64_t [getHistoriesRead](#) ()
Get the number of Monte Carlo histories that have been read so far. If the end of the file has been reached, this will return the total number of original histories unless more histories than expected have already been read - in which case it returns the actual count.
- [Particle](#) [getNextParticle](#) ()
Get the next particle from the phase space file.
- virtual std::uint64_t [getNumberOfRepresentedHistories](#) () const
Get the number of histories actually represented in this phase space.
- virtual std::uint64_t [getParticlesRead](#) ()
Get the number of particles that have been read so far.
- const std::string [getPHSPFormat](#) () const
Get the phase space file format identifier.

7.2 ParticleZoo::EGSphspFile::Reader Class Reference

- virtual bool [hasMoreParticles](#) ()
Check if there are more particles to read in the file.
- virtual bool [hasNativeIncrementalHistoryCounters](#) () const
Check if this format directly stores incremental history numbers per-particle.
- virtual bool [hasNativeRepresentedHistoryCount](#) () const
Check if this format can provide the number of represented histories without scanning the file.
- bool [isPxConstant](#) () const
Check if the X-component of momentum is constant for all particles.
- bool [isPyConstant](#) () const
Check if the Y-component of momentum is constant for all particles.
- bool [isPzConstant](#) () const
Check if the Z-component of momentum is constant for all particles.
- bool [isWeightConstant](#) () const
Check if the statistical weight is constant for all particles.
- bool [isXConstant](#) () const
Check if the X coordinate is constant for all particles.
- bool [isYConstant](#) () const
Check if the Y coordinate is constant for all particles.
- bool [isZConstant](#) () const
Check if the Z coordinate is constant for all particles.
- void [moveToParticle](#) (std::uint64_t particleIndex)
Move the file position to a specific particle index.
- [Particle](#) [peekNextParticle](#) ()
Peek at the next particle without advancing the file position.
- void [setCommentMarkers](#) (const std::vector< std::string > &commentMarkers)
Set comment markers for ASCII format files.

Static Public Member Functions

- static std::vector< CLICommand > [getFormatSpecificCLICommands](#) ()
Get the list of EGS-specific command line interface commands.

Static Public Member Functions inherited from [ParticleZoo::PhaseSpaceFileReader](#)

- static std::vector< CLICommand > [getCLICommands](#) ()
Get command line interface commands supported by this reader.

Protected Member Functions

- `std::size_t` [getParticleRecordLength](#) () const override
Get the length of each particle record in bytes.
- `std::size_t` [getParticleRecordStartOffset](#) () const override
Get the byte offset where particle records start.
- [Particle](#) [readBinaryParticle](#) ([ByteBuffer](#) &buffer) override
Read a single particle in EGS binary format.

Protected Member Functions inherited from [ParticleZoo::PhaseSpaceFileReader](#)

- `double` [calcThirdUnitComponent](#) (`double` &u, `double` &v) const
Calculate the third component of a unit vector from two components (double precision).
- `float` [calcThirdUnitComponent](#) (`float` &u, `float` &v) const
Calculate the third component of a unit vector from two components (float precision).
- `const ByteBuffer` [getHeaderData](#) ()
Get the file header data as a byte buffer.
- `const ByteBuffer` [getHeaderData](#) (`std::size_t` headerSize)
Get a specific amount of header data as a byte buffer.
- `virtual std::size_t` [getMaximumASCIILineLength](#) () const
Get the maximum line length for ASCII format files.
- [Particle](#) [getNextParticle](#) (`bool` countParticleInStatistics)
Get the next particle with optional statistics counting control.
- `std::size_t` [getNumberOfEntriesInFile](#) () const
Get the number of particle records that fit in the file.
- `virtual std::uint64_t` [getParticlesRead](#) (`bool` includeSkippedParticles)
Get the number of particles read with optional inclusion of skipped particles (including pseudo-particles).
- `const UserOptions` & [getUserOptions](#) () const
Get the user options that were passed to the constructor.
- `virtual Particle` [peekParticleManually](#) ()
Peek at a particle manually (for formats requiring third-party I/O).
- `virtual Particle` [readASCIIParticle](#) (`const std::string` &line)
Read a particle from ASCII data.
- `virtual Particle` [readParticleManually](#) ()
Read a particle manually (for formats requiring third-party I/O).
- `void` [setByteOrder](#) ([ByteOrder](#) byteOrder)
Set the byte order for binary data interpretation.
- `void` [setConstantPx](#) (`float` Px)
Set a constant X-component of the direction unit vector for all particles.
- `void` [setConstantPy](#) (`float` Py)
Set a constant Y-component of the direction unit vector for all particles.

7.2 ParticleZoo::EGSphspFile::Reader Class Reference

- void [setConstantPz](#) (float Pz)
Set a constant Z-component of the direction unit vector for all particles.
- void [setConstantWeight](#) (float weight)
Set a constant statistical weight for all particles.
- void [setConstantX](#) (float X)
Set a constant X coordinate value for all particles.
- void [setConstantY](#) (float Y)
Set a constant Y coordinate value for all particles.
- void [setConstantZ](#) (float Z)
Set a constant Z coordinate value for all particles.

7.2.1 Detailed Description

[Reader](#) class for EGS phase space files.

Provides functionality to read EGS phase space files created by EGSnrc Monte Carlo simulations (or it's variants BEAMnrc, DOSXYZnrc, etc.). Supports both MODE0 and MODE2 formats.

7.2.2 Constructor & Destructor Documentation

7.2.2.1 Reader()

```
ParticleZoo::EGSphspFile::Reader::Reader (
    const std::string & fileName,
    const UserOptions & options = UserOptions{} )
```

Construct a new EGS phase space file reader.

Parameters

<i>fileName</i>	Path to the EGS phase space file to read
<i>options</i>	User options including user-specific configuration

Exceptions

<i>std::runtime_error</i>	if file format is invalid or unsupported mode
---------------------------	---

7.2.3 Member Function Documentation

7.2.3.1 getFormatSpecificCLICommands()

```
std::vector< CLICommand > ParticleZoo::EGSphspFile::Reader::getFormatSpecificCLICommands ( )  
[static]
```

Get the list of EGS-specific command line interface commands.

Returns

std::vector<CLICommand> Vector of EGS-specific CLI commands

7.2.3.2 getLATCHOption()

```
EGSLATCHOPTION ParticleZoo::EGSphspFile::Reader::getLATCHOption ( ) const [inline]
```

Get the LATCH interpretation option.

Returns

EGSLATCHOPTION The LATCH option used for reading particles

7.2.3.3 getMaxKineticEnergy()

```
float ParticleZoo::EGSphspFile::Reader::getMaxKineticEnergy ( ) const [inline]
```

Get the maximum kinetic energy of particles in the file.

Returns

float Maximum kinetic energy

7.2.3.4 getMinElectronEnergy()

```
float ParticleZoo::EGSphspFile::Reader::getMinElectronEnergy ( ) const [inline]
```

Get the minimum electron energy threshold used in the simulation.

Returns

float Minimum electron energy

7.2 ParticleZoo::EGSphspFile::Reader Class Reference

7.2.3.5 getMode()

```
EGSMODE ParticleZoo::EGSphspFile::Reader::getMode ( ) const [inline]
```

Get the EGS file mode (MODE0 or MODE2).

Returns

EGSMODE The detected file mode

7.2.3.6 getNumberOfOriginalHistories()

```
std::uint64_t ParticleZoo::EGSphspFile::Reader::getNumberOfOriginalHistories ( ) const [inline],  
[override], [virtual]
```

Get the number of original Monte Carlo histories.

Returns

std::uint64_t Number of original histories that generated this phase space

Implements [ParticleZoo::PhaseSpaceFileReader](#).

7.2.3.7 getNumberOfParticles()

```
std::uint64_t ParticleZoo::EGSphspFile::Reader::getNumberOfParticles ( ) const [inline], [override],  
[virtual]
```

Get the total number of particles in the phase space file.

Returns

std::uint64_t Number of particles (from header or calculated from file size)

Implements [ParticleZoo::PhaseSpaceFileReader](#).

7.2.3.8 getNumberOfPhotons()

```
unsigned int ParticleZoo::EGSphspFile::Reader::getNumberOfPhotons ( ) const [inline]
```

Get the number of photons in the phase space.

Returns

unsigned int Number of photon particles

7.2.3.9 getParticleRecordLength()

```
std::size_t ParticleZoo::EGSphspFile::Reader::getParticleRecordLength ( ) const [inline], [override],  
[protected], [virtual]
```

Get the length of each particle record in bytes.

Returns

std::size_t Record length (28 for MODE0, 32 for MODE2)

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

7.2.3.10 getParticleRecordStartOffset()

```
std::size_t ParticleZoo::EGSphspFile::Reader::getParticleRecordStartOffset ( ) const [inline],  
[override], [protected], [virtual]
```

Get the byte offset where particle records start.

Returns

std::size_t Offset from the beginning of the file (same as record length for EGS files)

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

7.2.3.11 readBinaryParticle()

```
Particle ParticleZoo::EGSphspFile::Reader::readBinaryParticle (  
    ByteBuffer & buffer ) [override], [protected], [virtual]
```

Read a single particle in EGS binary format.

Parses EGS binary format including LATCH bits and ZLAST if present.

7.2 ParticleZoo::EGSphspFile::Reader Class Reference

Parameters

<i>buffer</i>	The byte buffer containing particle data
---------------	--

Returns

[Particle](#) The parsed particle object with EGS-specific properties

Exceptions

<i>std::runtime_error</i>	if particle data is invalid
---------------------------	-----------------------------

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

The documentation for this class was generated from the following files:

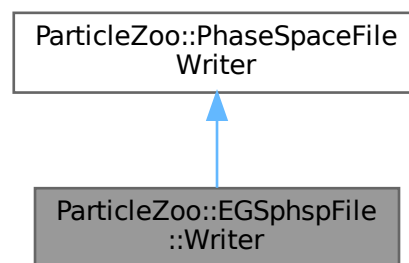
- include/particlezoo/egs/egsphspFile.h
- src/egs/egsphspFile.cc

7.3 ParticleZoo::EGSphspFile::Writer Class Reference

[Writer](#) class for EGS phase space files.

```
#include <particlezoo/egs/egsphspFile.h>
```

Inheritance diagram for ParticleZoo::EGSphspFile::Writer:



Public Member Functions

- [Writer](#) (const std::string &fileName, const UserOptions &options=UserOptions{})
Construct a new EGS phase space file writer.
- EGSLATCHOPTION [getLATCHOption](#) () const
Get the LATCH interpretation option.
- std::uint64_t [getMaximumSupportedParticles](#) () const override
Get the maximum number of particles this format can support.
- EGSMODE [getMode](#) () const
Get the EGS file mode (MODE0 or MODE2).
- void [setNumberOfOriginalHistories](#) (unsigned int numberOfOriginalHistories)
Manually set the number of original Monte Carlo histories.

Public Member Functions inherited from ParticleZoo::PhaseSpaceFileWriter

- [PhaseSpaceFileWriter](#) (const std::string &phspFormat, const std::string &fileName, const UserOptions &userOptions, [FormatType](#) formatType=[FormatType::BINARY](#), const [FixedValues](#) fixedValues=[FixedValues\(\)](#), unsigned int bufferSize=[DEFAULT_BUFFER_SIZE](#))
Construct a new Phase Space File Writer object.
- virtual [~PhaseSpaceFileWriter](#) ()
Destroy the Phase Space File Writer object.
- void [addAdditionalHistories](#) (std::uint64_t additionalHistories)
Add additional Monte Carlo histories to the count.
- void [close](#) ()
Close the phase space file and finalize writing.
- [ByteOrder](#) [getByteOrder](#) () const
Get the byte order used for binary data writing.
- float [getConstantPx](#) () const
Get the constant X-component of the direction unit vector (if constant).
- float [getConstantPy](#) () const
Get the constant Y-component of the direction unit vector (if constant).
- float [getConstantPz](#) () const
Get the constant Z-component of the direction unit vector (if constant).
- float [getConstantWeight](#) () const
Get the constant statistical weight value (if constant).
- float [getConstantX](#) () const
Get the constant X coordinate value (if constant).
- float [getConstantY](#) () const
Get the constant Y coordinate value (if constant).
- float [getConstantZ](#) () const
Get the constant Z coordinate value (if constant).
- const std::string [getFileName](#) () const
Get the filename where the phase space file is being written.
- const [FixedValues](#) [getFixedValues](#) () const
Get the fixed values configuration.
- virtual std::uint64_t [getHistoriesWritten](#) () const
Get the number of Monte Carlo histories that have been written.
- std::uint64_t [getParticlesWritten](#) () const
Get the number of particles that have been written to the file.
- const std::string [getPHSPFormat](#) () const
Get the phase space file format identifier.
- bool [isPxConstant](#) () const
Check if the X-component of the direction unit vector is set to a constant value for all particles.
- bool [isPyConstant](#) () const
Check if the Y-component of the direction unit vector is set to a constant value for all particles.

- bool [isPzConstant](#) () const
Check if the Z-component of the direction unit vector is set to a constant value for all particles.
- bool [isWeightConstant](#) () const
Check if the statistical weight is set to a constant value for all particles.
- bool [isXConstant](#) () const
Check if the X coordinate is set to a constant value for all particles.
- bool [isYConstant](#) () const
Check if the Y coordinate is set to a constant value for all particles.
- bool [isZConstant](#) () const
Check if the Z coordinate is set to a constant value for all particles.
- virtual void [writeParticle](#) ([Particle](#) particle)
Write a particle to the phase space file.

Static Public Member Functions

- static std::vector< [CLICommand](#) > [getFormatSpecificCLICommands](#) ()
Get the list of EGS-specific command line interface commands.

Static Public Member Functions inherited from [ParticleZoo::PhaseSpaceFileWriter](#)

- static std::vector< [CLICommand](#) > [getCLICommands](#) ()
Get command line interface commands supported by this writer.

Protected Member Functions

- std::size_t [getParticleRecordLength](#) () const override
Get the length of each particle record in bytes.
- std::size_t [getParticleRecordStartOffset](#) () const override
Get the byte offset where particle records start.
- virtual void [writeBinaryParticle](#) ([ByteBuffer](#) &buffer, [Particle](#) &particle) override
Write a single particle in EGS binary format.
- virtual void [writeHeaderData](#) ([ByteBuffer](#) &buffer) override
Write the EGS file header with current statistics.

Protected Member Functions inherited from ParticleZoo::PhaseSpaceFileWriter

- virtual bool [accountForAdditionalHistories](#) (std::uint64_t additionalHistories)
Handle accounting for simulation histories that produced no particles.
- virtual bool [canHaveConstantPx](#) () const
Check if this format supports constant X-component of the direction unit vector.
- virtual bool [canHaveConstantPy](#) () const
Check if this format supports constant Y-component of the direction unit vector.
- virtual bool [canHaveConstantPz](#) () const
Check if this format supports constant Z-component of the direction unit vector.
- virtual bool [canHaveConstantWeight](#) () const
Check if this format supports constant statistical weights.
- virtual bool [canHaveConstantX](#) () const
Check if this format supports constant X coordinates.
- virtual bool [canHaveConstantY](#) () const
Check if this format supports constant Y coordinates.
- virtual bool [canHaveConstantZ](#) () const
Check if this format supports constant Z coordinates.
- virtual bool [canWritePseudoParticlesExplicitly](#) () const
Check if this format can write pseudo-particles explicitly.
- virtual void [fixedValuesHaveChanged](#) ()
Called when fixed values have been changed.
- virtual size_t [getMaximumASCIILineLength](#) () const
Get the maximum line length for ASCII format files.
- virtual std::uint64_t [getPendingHistories](#) () const
Get the number of pending histories to account for.
- const UserOptions & [getUserOptions](#) () const
Get the user options that were passed to the constructor.
- void [setByteOrder](#) (ByteOrder byteOrder)
Set the byte order for binary data writing.
- void [setConstantPx](#) (float Px)
Set a constant X-component of the direction unit vector for all particles.
- void [setConstantPy](#) (float Py)
Set a constant Y-component of the direction unit vector for all particles.
- void [setConstantPz](#) (float Pz)
Set a constant Z-component of the direction unit vector for all particles.
- void [setConstantWeight](#) (float weight)
Set a constant statistical weight for all particles.
- void [setConstantX](#) (float X)
Set a constant X coordinate value for all particles.
- void [setConstantY](#) (float Y)
Set a constant Y coordinate value for all particles.

- void `setConstantZ` (float Z)
Set a constant Z coordinate value for all particles.
- virtual const std::string `writeASCIIParticle` (Particle &particle)
Write a particle in ASCII format as a string.
- virtual void `writeParticleManually` (Particle &particle)
Write a particle manually (for formats requiring third-party I/O).

7.3.1 Detailed Description

`Writer` class for EGS phase space files.

Provides functionality to write EGS phase space files compatible with EGSnrc Monte Carlo simulations (or it's variants BEAMnrc, DOSXYZnrc, etc.). Supports both MODE0 and MODE2 formats.

7.3.2 Constructor & Destructor Documentation

7.3.2.1 Writer()

```
ParticleZoo::EGSphspFile::Writer::Writer (
    const std::string & fileName,
    const UserOptions & options = UserOptions{} )
```

Construct a new EGS phase space file writer.

Parameters

<i>fileName</i>	Path where the EGS phase space file will be written
<i>options</i>	User options including EGS-specific configuration (e.g., mode selection)

Exceptions

<i>std::runtime_error</i>	if specified mode is unsupported
---------------------------	----------------------------------

7.3.3 Member Function Documentation

7.3.3.1 getFormatSpecificCLICommands()

```
std::vector< CLICommand > ParticleZoo::EGSphspFile::Writer::getFormatSpecificCLICommands ( )  
[static]
```

Get the list of EGS-specific command line interface commands.

Returns

std::vector<CLICommand> Vector of EGS-specific CLI commands for writers

7.3.3.2 getLATCHOption()

```
EGSLATCHOPTION ParticleZoo::EGSphspFile::Writer::getLATCHOption ( ) const [inline]
```

Get the LATCH interpretation option.

Returns

EGSLATCHOPTION The LATCH option used for writing particles

7.3.3.3 getMaximumSupportedParticles()

```
std::uint64_t ParticleZoo::EGSphspFile::Writer::getMaximumSupportedParticles ( ) const [inline],  
[override], [virtual]
```

Get the maximum number of particles this format can support.

Returns

std::uint64_t Maximum particle count (limited by 32-bit unsigned integer)

Implements [ParticleZoo::PhaseSpaceFileWriter](#).

7.3.3.4 `getMode()`

```
EGSMODE ParticleZoo::EGSphspFile::Writer::getMode ( ) const [inline]
```

Get the EGS file mode (MODE0 or MODE2).

Returns

EGSMODE The file mode

7.3.3.5 `getParticleRecordLength()`

```
std::size_t ParticleZoo::EGSphspFile::Writer::getParticleRecordLength ( ) const [inline], [override],  
[protected], [virtual]
```

Get the length of each particle record in bytes.

Returns

std::size_t Record length (28 for MODE0, 32 for MODE2)

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.3.3.6 `getParticleRecordStartOffset()`

```
std::size_t ParticleZoo::EGSphspFile::Writer::getParticleRecordStartOffset ( ) const [inline],  
[override], [protected], [virtual]
```

Get the byte offset where particle records start.

Returns

std::size_t Offset from the beginning of the file (same as record length for EGS files)

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.3.3.7 `setNumberOfOriginalHistories()`

```
void ParticleZoo::EGSphspFile::Writer::setNumberOfOriginalHistories (  
    unsigned int numberOfOriginalHistories ) [inline]
```

Manually set the number of original Monte Carlo histories.

Allows explicit specification of the history count instead of automatic tracking.

7.3 ParticleZoo::EGSphspFile::Writer Class Reference

Parameters

<i>numberOfOriginalHistories</i>	The number of original histories to record in the header
----------------------------------	--

7.3.3.8 writeBinaryParticle()

```
void ParticleZoo::EGSphspFile::Writer::writeBinaryParticle (
    ByteBuffer & buffer,
    Particle & particle ) [override], [protected], [virtual]
```

Write a single particle in EGS binary format.

Converts particle data to EGS format including LATCH encoding and ZLAST if in MODE2.

Parameters

<i>buffer</i>	The byte buffer to write particle data into
<i>particle</i>	The particle to write

Exceptions

<i>std::runtime_error</i>	if particle type is unsupported or required properties are missing
---------------------------	--

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.3.3.9 writeHeaderData()

```
void ParticleZoo::EGSphspFile::Writer::writeHeaderData (
    ByteBuffer & buffer ) [override], [protected], [virtual]
```

Write the EGS file header with current statistics.

Writes the header containing mode string, particle counts, energy ranges, and history information based on accumulated statistics (or specified histories).

Parameters

<i>buffer</i>	The byte buffer to write header data into
---------------	---

Exceptions

<code>std::runtime_error</code>	if mode is unsupported
---------------------------------	------------------------

Implements [ParticleZoo::PhaseSpaceFileWriter](#).

The documentation for this class was generated from the following files:

- `include/particlezoo/egs/egsphspFile.h`
- `src/egs/egsphspFile.cc`

7.4 ParticleZoo::FixedValues Struct Reference

Structure defining constant (fixed) values for particle properties.

```
#include <particlezoo/Particle.h>
```

Public Member Functions

- bool **operator==** (const [FixedValues](#) &other) const
Equality comparison operator for [FixedValues](#).

Public Attributes

- float **constantPx** {0}
Constant X directional cosine value (when pxIsConstant is true)
- float **constantPy** {0}
Constant Y directional cosine value (when pyIsConstant is true)
- float **constantPz** {0}
Constant Z directional cosine value (when pzIsConstant is true)
- float **constantWeight** {1}
Constant statistical weight value (when weightIsConstant is true)
- float **constantX** {0}
Constant X coordinate value (when xIsConstant is true)
- float **constantY** {0}
Constant Y coordinate value (when yIsConstant is true)
- float **constantZ** {0}
Constant Z coordinate value (when zIsConstant is true)
- bool **pxIsConstant** {false}
True if X directional cosine is constant for all particles.
- bool **pyIsConstant** {false}
True if Y directional cosine is constant for all particles.
- bool **pzIsConstant** {false}
True if Z directional cosine is constant for all particles.
- bool **weightIsConstant** {false}
True if statistical weight is constant for all particles.
- bool **xIsConstant** {false}
True if X coordinate is constant for all particles.
- bool **yIsConstant** {false}
True if Y coordinate is constant for all particles.
- bool **zIsConstant** {false}
True if Z coordinate is constant for all particles.

7.4.1 Detailed Description

Structure defining constant (fixed) values for particle properties.

Used to optimize phase space files by storing constant values once rather than repeating them for every particle. Useful when all particles share certain properties (e.g., all particles start from the same position).

7.4.2 Member Function Documentation

7.4.2.1 operator==()

```
bool ParticleZoo::FixedValues::operator== (
    const FixedValues & other ) const [inline]
```

Equality comparison operator for [FixedValues](#).

Parameters

<i>other</i>	The other FixedValues object to compare with
--------------	--

Returns

true if all members are equal
false if any members differ

The documentation for this struct was generated from the following file:

- include/particlezoo/Particle.h

7.5 ParticleZoo::FormatRegistry Class Reference

Singleton registry for managing phase space file format readers and writers.

```
#include <particlezoo/utilities/formats.h>
```

Public Types

- using [ReaderFactoryFn](#) = std::function< std::unique_ptr< [PhaseSpaceFileReader](#) >(const std::string &filename, const UserOptions &options)>
Function type for creating phase space file readers.
- using [WriterFactoryFn](#) = std::function< std::unique_ptr< [PhaseSpaceFileWriter](#) >(const std::string &filename, const UserOptions &options, const [FixedValues](#) &fixedValues)>
Function type for creating phase space file writers.

Static Public Member Functions

- static std::unique_ptr< [PhaseSpaceFileReader](#) > [CreateReader](#) (const std::string &filename, const UserOptions &options={})
Create a reader for a file using automatic format detection.
- static std::unique_ptr< [PhaseSpaceFileReader](#) > [CreateReader](#) (const std::string &formatName, const std::string &filename, const UserOptions &options={})
Create a reader for a specific format and file.
- static std::unique_ptr< [PhaseSpaceFileWriter](#) > [CreateWriter](#) (const std::string &filename, const UserOptions &options={}, const [FixedValues](#) &fixedValues={})
Create a writer for a file using automatic format detection.
- static std::unique_ptr< [PhaseSpaceFileWriter](#) > [CreateWriter](#) (const std::string &formatName, const std::string &filename, const UserOptions &options={}, const [FixedValues](#) &fixedValues={})
Create a writer for a specific format and file.
- static const std::string [ExtensionForFormat](#) (const std::string &formatName)
Get the standard file extension for a specific format.
- static std::vector< [SupportedFormat](#) > [FormatsForExtension](#) (const std::string &extension)
Find all formats that support a given file extension.
- static void [PrintSupportedFormats](#) ()
Print a list of all supported formats to standard output.
- static void [RegisterFormat](#) (const [SupportedFormat](#) &fmt, [ReaderFactoryFn](#) reader, [WriterFactoryFn](#) writer)
Register a new phase space file format with reader and writer factories.
- static void [RegisterStandardFormats](#) ()
Register all standard built-in phase space file formats.
- static const std::vector< [SupportedFormat](#) > [SupportedFormats](#) ()
Get a list of all registered formats.

Static Public Attributes

- static constexpr bool [FILE_EXTENSION_CAN_HAVE_SUFFIX](#) = true
Constant indicating that a file extension can have numeric suffixes.

7.5.1 Detailed Description

Singleton registry for managing phase space file format readers and writers.

The [FormatRegistry](#) provides a centralized thread-safe system for registering, discovering, and creating phase space file readers and writers for different simulation formats. It supports automatic format detection based on file extensions and provides factory methods for creating appropriate reader/writer instances.

The registry is typically populated during application startup by calling [RegisterStandardFormats\(\)](#), which registers all built-in format handlers.

7.5.2 Member Typedef Documentation

7.5.2.1 ReaderFactoryFn

```
using ParticleZoo::FormatRegistry::ReaderFactoryFn = std::function<std::unique_ptr<PhaseSpaceFileReader>(const std::string& filename, const UserOptions& options)>
```

Function type for creating phase space file readers.

Parameters

<i>filename</i>	The path to the file to read
<i>options</i>	User options for configuring the reader

Returns

`std::unique_ptr<PhaseSpaceFileReader>` A unique pointer to the created reader

7.5.2.2 WriterFactoryFn

```
using ParticleZoo::FormatRegistry::WriterFactoryFn = std::function<std::unique_ptr<PhaseSpaceFileWriter>(const std::string& filename, const UserOptions& options, const FixedValues& fixedValues)>
```

Function type for creating phase space file writers.

7.5 ParticleZoo::FormatRegistry Class Reference

Parameters

<i>filename</i>	The path to the file to write
<i>options</i>	User options for configuring the writer
<i>fixedValues</i>	Fixed values for constant particle properties

Returns

`std::unique_ptr<PhaseSpaceFileWriter>` A unique pointer to the created writer

7.5.3 Member Function Documentation

7.5.3.1 CreateReader() [1/2]

```
std::unique_ptr< PhaseSpaceFileReader > ParticleZoo::FormatRegistry::CreateReader (
    const std::string & filename,
    const UserOptions & options = {} ) [static]
```

Create a reader for a file using automatic format detection.

Determines the appropriate format based on the file extension and creates a reader instance. Requires a unique format match for the extension.

Parameters

<i>filename</i>	The path to the file to read (must have a recognized extension)
<i>options</i>	User options for configuring the reader (default: empty)

Returns

`std::unique_ptr<PhaseSpaceFileReader>` A unique pointer to the created reader

Exceptions

<i>std::runtime_error</i>	if no extension found, no format matches, or multiple formats match
---------------------------	---

7.5.3.2 CreateReader() [2/2]

```
std::unique_ptr< PhaseSpaceFileReader > ParticleZoo::FormatRegistry::CreateReader (
    const std::string & formatName,
    const std::string & filename,
    const UserOptions & options = {} ) [static]
```

Create a reader for a specific format and file.

Creates a reader instance for the specified format, bypassing automatic detection.

Parameters

<i>formatName</i>	The name of the format to use (must be registered)
<i>filename</i>	The path to the file to read
<i>options</i>	User options for configuring the reader (default: empty)

Returns

`std::unique_ptr<PhaseSpaceFileReader>` A unique pointer to the created reader

Exceptions

<code>std::runtime_error</code>	if the format is not registered
---------------------------------	---------------------------------

7.5.3.3 CreateWriter() [1/2]

```
std::unique_ptr< PhaseSpaceFileWriter > ParticleZoo::FormatRegistry::CreateWriter (
    const std::string & filename,
    const UserOptions & options = {},
    const FixedValues & fixedValues = {} ) [static]
```

Create a writer for a file using automatic format detection.

Determines the appropriate format based on the file extension and creates a writer instance. Requires a unique format match for the extension.

Parameters

<i>filename</i>	The path to the file to write (must have a recognized extension)
<i>options</i>	User options for configuring the writer (default: empty)
<i>fixedValues</i>	Fixed values for constant particle properties (default: empty)

7.5 ParticleZoo::FormatRegistry Class Reference

Returns

std::unique_ptr<PhaseSpaceFileWriter> A unique pointer to the created writer

Exceptions

<code>std::runtime_error</code>	if no extension found, no format matches, or multiple formats match
---------------------------------	---

7.5.3.4 CreateWriter() [2/2]

```
std::unique_ptr< PhaseSpaceFileWriter > ParticleZoo::FormatRegistry::CreateWriter (
    const std::string & formatName,
    const std::string & filename,
    const UserOptions & options = {},
    const FixedValues & fixedValues = {} ) [static]
```

Create a writer for a specific format and file.

Creates a writer instance for the specified format, bypassing automatic detection.

Parameters

<i>formatName</i>	The name of the format to use (must be registered)
<i>filename</i>	The path to the file to write
<i>options</i>	User options for configuring the writer (default: empty)
<i>fixedValues</i>	Fixed values for constant particle properties (default: empty)

Returns

std::unique_ptr<PhaseSpaceFileWriter> A unique pointer to the created writer

Exceptions

<code>std::runtime_error</code>	if the format is not registered
---------------------------------	---------------------------------

7.5.3.5 ExtensionForFormat()

```
const std::string ParticleZoo::FormatRegistry::ExtensionForFormat (
    const std::string & formatName ) [static]
```

Get the standard file extension for a specific format.

Parameters

<i>formatName</i>	The name of the format to query
-------------------	---------------------------------

Returns

const std::string The standard file extension for the format

Exceptions

<i>std::runtime_error</i>	if the format is not registered
---------------------------	---------------------------------

7.5.3.6 FormatsForExtension()

```
std::vector< SupportedFormat > ParticleZoo::FormatRegistry::FormatsForExtension (
    const std::string & extension ) [static]
```

Find all formats that support a given file extension.

Performs case-insensitive matching of file extensions. Also handles formats that support extensions with numeric suffixes (e.g., ".egsphsp1" matching ".egsphsp").

Parameters

<i>extension</i>	The file extension to search for (including the dot)
------------------	--

Returns

std::vector<SupportedFormat> Vector of formats supporting the extension

7.5.3.7 PrintSupportedFormats()

```
void ParticleZoo::FormatRegistry::PrintSupportedFormats ( ) [static]
```

Print a list of all supported formats to standard output.

Displays format names, descriptions, and file extensions in a human-readable format.

7.5 ParticleZoo::FormatRegistry Class Reference

7.5.3.8 RegisterFormat()

```
void ParticleZoo::FormatRegistry::RegisterFormat (
    const SupportedFormat & fmt,
    ReaderFactoryFn reader,
    WriterFactoryFn writer ) [static]
```

Register a new phase space file format with reader and writer factories.

Registers a format with the global registry, making it available for automatic detection and creation. The format name must be unique.

Parameters

<i>fmt</i>	The format metadata (name, description, extension, etc.)
<i>reader</i>	Factory function for creating readers of this format
<i>writer</i>	Factory function for creating writers of this format

Exceptions

<i>std::invalid_argument</i>	if format parameters are invalid
<i>std::runtime_error</i>	if format name is already registered

7.5.3.9 RegisterStandardFormats()

```
void ParticleZoo::FormatRegistry::RegisterStandardFormats ( ) [static]
```

Register all standard built-in phase space file formats.

This method registers the standard formats supported by [ParticleZoo](#) including:

- IAEA format (.IAEAphsp)
- TOPAS format (.phsp)
- penEasy format (.dat)
- EGS format (.egsphsp with suffixes)
- ROOT format (.root) - if compiled with ROOT support

This method is safe to call multiple times (uses internal flag to prevent duplicate registration). Should be called during application initialization before using the registry.

7.5.3.10 SupportedFormats()

```
const std::vector< SupportedFormat > ParticleZoo::FormatRegistry::SupportedFormats ( ) [static]
```

Get a list of all registered formats.

Returns a copy of all currently registered format metadata.

Returns

const std::vector<SupportedFormat> Vector of all supported formats

7.5.4 Member Data Documentation

7.5.4.1 FILE_EXTENSION_CAN_HAVE_SUFFIX

```
constexpr bool ParticleZoo::FormatRegistry::FILE_EXTENSION_CAN_HAVE_SUFFIX = true [static], [constexpr]
```

Constant indicating that a file extension can have numeric suffixes.

Used when registering formats that support numbered file extensions (e.g., ".egsphsp1", ".egsphsp2").

The documentation for this class was generated from the following files:

- include/particlezoo/utilities/formats.h
- src/utilities/formats.cc

7.6 ParticleZoo::HistoryBalancedParallelReader Class Reference

Multi-threaded phase space file reader for parallel particle processing.

```
#include <particlezoo/parallel/HistoryBalancedParallelReader.h>
```

Public Member Functions

- [HistoryBalancedParallelReader](#) (const std::string &filename, const UserOptions &options={}, size_t numThreads=1)
Constructs a multi-threaded phase space reader.
- [~HistoryBalancedParallelReader](#) ()
Destructor that closes all underlying readers.
- void [close](#) ()
Closes all underlying phase space file readers.
- std::uint64_t [getHistoriesRead](#) (size_t threadIndex) const
Gets the total number of histories processed by a specific thread.
- [Particle](#) [getNextParticle](#) (size_t threadIndex)
Retrieves the next particle for a specific thread.
- std::uint64_t [getNumberOfOriginalHistories](#) () const
Gets the number of original histories in the phase space file.
- std::uint64_t [getNumberOfParticles](#) () const
Gets the total number of particles in the phase space file.
- std::uint64_t [getNumberOfRepresentedHistories](#) () const
Gets the number of represented histories in the file.
- std::size_t [getNumberOfThreads](#) () const
Gets the number of threads used by this reader.
- std::uint64_t [getParticlesRead](#) (size_t threadIndex) const
Gets the total number of particles processed by a specific thread.
- std::uint64_t [getTotalHistoriesRead](#) () const
Gets the total number of original histories read across all threads.
- std::uint64_t [getTotalParticlesRead](#) () const
Gets the total number of particles read across all threads.
- bool [hasMoreParticles](#) (size_t threadIndex)
Checks if more particles are available for a specific thread.
- bool [hasNativeIncrementalHistoryCounters](#) () const
Checks if the underlying phase space format provides native incremental history counters.
- bool [hasNativeRepresentedHistoryCount](#) () const
Checks if the underlying phase space format provides native represented history count.
- [Particle](#) [peekNextParticle](#) (size_t threadIndex)
Peeks at the next particle for a specific thread without consuming it.

7.6.1 Detailed Description

Multi-threaded phase space file reader for parallel particle processing.

[HistoryBalancedParallelReader](#) provides thread-safe parallel reading of phase space files by creating multiple independent [PhaseSpaceFileReader](#) instances and partitioning the histories across threads. Each thread maintains its own reader positioned at a specific section of the phase space file.

The important property of this class is that it distributes histories evenly across threads, it does not necessarily distribute particles evenly. This is for use-cases where histories are the fundamental unit of work. If load balancing by particle count is desired, consider using a different approach. Although on average load balancing by history should effectively balance particle counts as well, there is significant overhead associated with dynamically distributing histories to threads so a dedicated particle-based load balancing mechanism would be preferable in that scenario.

The reader automatically handles:

- History partitioning across threads with proper boundary alignment
- Empty history gap distribution for files with empty histories
- Thread-safe particle retrieval without locking (each thread has its own reader)

Note

Each thread must use its assigned thread index when calling methods.

Incremental history counts are derived from represented histories only and do not reflect empty histories counts directly recorded in the phase space file if present.

[Particle](#) order within a history is preserved, but histories are processed in parallel across threads.

7.6.2 Constructor & Destructor Documentation

7.6.2.1 HistoryBalancedParallelReader()

```
ParticleZoo::HistoryBalancedParallelReader::HistoryBalancedParallelReader (
    const std::string & filename,
    const UserOptions & options = {},
    size_t numThreads = 1 )
```

Constructs a multi-threaded phase space reader.

Creates multiple [PhaseSpaceFileReader](#) instances (one per thread) and positions each at its designated starting history. The constructor partitions the total represented histories evenly across threads, with remainder histories distributed to the first N threads.

7.6 ParticleZoo::HistoryBalancedParallelReader Class Reference

Parameters

<i>filename</i>	Path to the phase space file to read
<i>options</i>	User options for configuring the reader (format-specific settings)
<i>numThreads</i>	Number of parallel threads that will read from this file

Exceptions

<i>std::runtime_error</i>	If the file cannot be opened or contains zero histories
<i>std::runtime_error</i>	If a PhaseSpaceFileReader cannot be created

7.6.3 Member Function Documentation

7.6.3.1 close()

```
void ParticleZoo::HistoryBalancedParallelReader::close ( )
```

Closes all underlying phase space file readers.

Should be called when done reading to free file handles and resources. Automatically called by the destructor.

7.6.3.2 getHistoriesRead()

```
std::uint64_t ParticleZoo::HistoryBalancedParallelReader::getHistoriesRead (
    size_t threadIndex ) const
```

Gets the total number of histories processed by a specific thread.

Returns the cumulative count of histories (including empty histories) that have been processed by this thread. This represents the contribution to the total original history count.

Parameters

<i>threadIndex</i>	The index of the thread (0 to numThreads-1)
--------------------	---

Returns

Total number of original histories processed (including empty ones)

Exceptions

<code>std::out_of_range</code>	If <code>threadIndex</code> is invalid
--------------------------------	--

7.6.3.3 getNextParticle()

```
Particle ParticleZoo::HistoryBalancedParallelReader::getNextParticle (
    size_t threadIndex )
```

Retrieves the next particle for a specific thread.

Reads and returns the next particle from the phase space file for the given thread. Updates internal counters for history tracking and automatically distributes empty history gaps when present.

Parameters

<code>threadIndex</code>	The index of the calling thread (0 to <code>numThreads-1</code>)
--------------------------	---

Returns

The next particle for this thread

Exceptions

<code>std::out_of_range</code>	If <code>threadIndex</code> is invalid
<code>std::runtime_error</code>	If no more particles are available for this thread

Note

Must call [hasMoreParticles\(\)](#) first to verify particles are available

Sets the `INCREMENTAL_HISTORY_NUMBER` property on particles

7.6.3.4 getNumberOfOriginalHistories()

```
std::uint64_t ParticleZoo::HistoryBalancedParallelReader::getNumberOfOriginalHistories ( ) const
[inline]
```

Gets the number of original histories in the phase space file.

Returns the total number of histories that are in the phase space file, including those that have no particles (empty histories).

7.6 ParticleZoo::HistoryBalancedParallelReader Class Reference

Returns

Number of original histories (including empty ones)

7.6.3.5 getNumberOfParticles()

```
std::uint64_t ParticleZoo::HistoryBalancedParallelReader::getNumberOfParticles ( ) const [inline]
```

Gets the total number of particles in the phase space file.

Returns

Total particle count in the file

7.6.3.6 getNumberOfRepresentedHistories()

```
std::uint64_t ParticleZoo::HistoryBalancedParallelReader::getNumberOfRepresentedHistories ( )  
const [inline]
```

Gets the number of represented histories in the file.

Returns the count of histories that actually produced particles and are represented in the phase space file. This is always less than or equal to the number of original histories.

Returns

Number of histories with at least one particle

7.6.3.7 getNumberOfThreads()

```
std::size_t ParticleZoo::HistoryBalancedParallelReader::getNumberOfThreads ( ) const [inline]
```

Gets the number of threads used by this reader.

Returns

Number of parallel threads

7.6.3.8 getParticlesRead()

```
std::uint64_t ParticleZoo::HistoryBalancedParallelReader::getParticlesRead (  
    size_t threadIndex ) const
```

Gets the total number of particles processed by a specific thread.

Returns the cumulative count of particles that have been read by this thread.

Parameters

<i>threadIndex</i>	The index of the thread (0 to numThreads-1)
--------------------	---

Returns

Total number of particles processed

Exceptions

<i>std::out_of_range</i>	If threadIndex is invalid
--------------------------	---------------------------

7.6.3.9 getTotalHistoriesRead()

```
std::uint64_t ParticleZoo::HistoryBalancedParallelReader::getTotalHistoriesRead ( ) const
```

Gets the total number of original histories read across all threads.

Returns

Total number of original histories read

7.6.3.10 getTotalParticlesRead()

```
std::uint64_t ParticleZoo::HistoryBalancedParallelReader::getTotalParticlesRead ( ) const
```

Gets the total number of particles read across all threads.

Returns

Total number of particles read

7.6.3.11 hasMoreParticles()

```
bool ParticleZoo::HistoryBalancedParallelReader::hasMoreParticles (
    size_t threadIndex )
```

Checks if more particles are available for a specific thread.

Determines whether the thread has reached its partition boundary or the end of the file. Uses caching to avoid repeated checks on consecutive calls.

7.6 ParticleZoo::HistoryBalancedParallelReader Class Reference

Parameters

<i>threadIndex</i>	The index of the calling thread (0 to numThreads-1)
--------------------	---

Returns

true if more particles are available, false otherwise

Exceptions

<i>std::out_of_range</i>	If threadIndex is invalid
--------------------------	---------------------------

Note

Always call this before [getNextParticle\(\)](#) to avoid exceptions

7.6.3.12 hasNativeIncrementalHistoryCounters()

```
bool ParticleZoo::HistoryBalancedParallelReader::hasNativeIncrementalHistoryCounters ( ) const
```

Checks if the underlying phase space format provides native incremental history counters.

Returns

True if native incremental history counters are available, false otherwise

7.6.3.13 hasNativeRepresentedHistoryCount()

```
bool ParticleZoo::HistoryBalancedParallelReader::hasNativeRepresentedHistoryCount ( ) const
```

Checks if the underlying phase space format provides native represented history count.

Returns

True if native represented history count is available, false otherwise

7.6.3.14 peekNextParticle()

```
Particle ParticleZoo::HistoryBalancedParallelReader::peekNextParticle (
    size_t threadIndex )
```

Peeks at the next particle for a specific thread without consuming it.

Allows inspection of the next particle that would be returned by [getNextParticle\(\)](#) without advancing the reader's position. Useful for checking history boundaries.

Parameters

<i>threadIndex</i>	The index of the calling thread (0 to numThreads-1)
--------------------	---

Returns

The next particle for this thread

Exceptions

<i>std::out_of_range</i>	If threadIndex is invalid
<i>std::runtime_error</i>	If no more particles are available for this thread

Note

Must call [hasMoreParticles\(\)](#) first to verify particles are available

The documentation for this class was generated from the following files:

- include/particlezoo/parallel/HistoryBalancedParallelReader.h
- src/parallel/HistoryBalancedParallelReader.cc

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

Header manager for IAEA phase space files.

```
#include <particlezoo/IAEA/IAEAHeader.h>
```

Public Types

- enum class [EXTRA_FLOAT_TYPE](#) { [USER_DEFINED_GENERIC_TYPE](#) = 0 , [XLAST](#) = 1 , [YLAST](#) = 2 , [ZLAST](#) = 3 }

Extra float data types for IAEA format.

- enum class [EXTRA_LONG_TYPE](#) { [USER_DEFINED_GENERIC_TYPE](#) = 0 , [INCREMENTAL_HISTORY_NUMBER](#) = 1 , [EGS_LATCH](#) = 2 , [PENELOPE_ILB5](#) = 3 , [PENELOPE_ILB4](#) = 4 , [PENELOPE_ILB3](#) = 5 , [PENELOPE_ILB2](#) = 6 , [PENELOPE_ILB1](#) = 7 }

Extra integer data types for IAEA format.

- enum class [FileType](#) { [PHSP_FILE](#) = 0 , [PHSP_GENERATOR](#) = 1 }

File type classification for IAEA phase space files.

- enum class [SECTION](#) { [IAEA_INDEX](#) , [TITLE](#) , [FILE_TYPE](#) , [CHECKSUM](#) , [RECORD_CONTENTS](#) , [RECORD_CONSTANT](#) , [RECORD_LENGTH](#) , [BYTE_ORDER](#) , [ORIGINAL_HISTORIES](#) , [PARTICLES](#) , [PHOTONS](#) , [ELECTRONS](#) , [POSITRONS](#) , [NEUTRONS](#) , [PROTONS](#) , [TRANSPORT_PARAMETERS](#) , [MACHINE_TYPE](#) , [MONTE_CARLO_CODE_VERSION](#) , [GLOBAL_PHOTON_ENERGY_CUTOFF](#) , [GLOBAL_PARTICLE_ENERGY](#) , [COORDINATE_SYSTEM_DESCRIPTION](#) , [BEAM_NAME](#) , [FIELD_SIZE](#) , [NOMINAL_SSD](#) , [MC_INPUT_FILENAME](#) , [VARIANCE_REDUCTION_TECHNIQUES](#) , [INITIAL_SOURCE_DESCRIPTION](#) , [PUBLISHED_REFERENCE](#) , [AUTHORS](#) , [INSTITUTION](#) , [LINK_VALIDATION](#) , [ADDITIONAL_NOTES](#) , [STATISTICAL_INFORMATION_PARTICLES](#) , [STATISTICAL_INFORMATION_GEOMETRY](#) , [CUSTOM_SECTION](#) }

Header section identifiers for IAEA format.

Public Member Functions

- [IAEAHeader](#) (const [IAEAHeader](#) &other, const std::string &newFilePath)

Copy constructor with new file path.

- [IAEAHeader](#) (const std::string &filePath, bool newFile=false)

Construct header from existing IAEA header file.

- void [addExtraFloat](#) ([EXTRA_FLOAT_TYPE](#) type)

Add an extra float data type to the record format.

- void [addExtraLong](#) ([EXTRA_LONG_TYPE](#) type)

- Add an extra integer data type to the record format.
- bool `checksumIsValid ()` const

Validate the data integrity checksum This check is strict. Not only does it verify that the checksum matches the file size, but it also checks that it equals the record length multiplied by the number of particles.
- void `countParticleStats (const Particle &particle)`

Update particle statistics with a new particle.
- `ByteOrder` `getByteOrder ()` const

Get the byte order for binary data (endianness)
- `std::uint64_t` `getChecksum ()` const

Get the data integrity checksum.
- float `getConstantU ()` const

Get the constant U direction cosine value (when not stored per particle)
- float `getConstantV ()` const

Get the constant V direction cosine value (when not stored per particle)
- float `getConstantW ()` const

Get the constant W direction cosine value (when not implicitly stored per particle)
- float `getConstantWeight ()` const

Get the constant particle weight value (when not stored per particle)
- float `getConstantX ()` const

Get the constant X coordinate value (when not stored per particle)
- float `getConstantY ()` const

Get the constant Y coordinate value (when not stored per particle)
- float `getConstantZ ()` const

Get the constant Z coordinate value (when not stored per particle)
- `std::string` `getDataFilePath ()` const

Get the path to the associated data file.
- `EXTRA_FLOAT_TYPE` `getExtraFloatType (unsigned int index)` const

Get the type of the extra float value at the specified index.
- `EXTRA_LONG_TYPE` `getExtraLongType (unsigned int index)` const

Get the type of the extra integer value at the specified index.
- `FileType` `getFileType ()` const

Get the file type classification.
- `std::string` `getHeaderFilePath ()` const

Get the path to the header file.
- `std::string` `getIAEAIndex ()` const

Get the IAEA index string.
- float `getMaxEnergy (ParticleType particleType)` const

Get the maximum energy for particles of a specific type.
- float `getMaxWeight (ParticleType particleType)` const

Get the maximum weight for particles of a specific type.
- float `getMaxX ()` const

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

- Get the maximum X coordinate across all particles.*
- float [getMaxY](#) () const
- Get the maximum Y coordinate across all particles.*
- float [getMaxZ](#) () const
- Get the maximum Z coordinate across all particles.*
- float [getMeanEnergy](#) (ParticleType particleType) const
- Get the mean energy for particles of a specific type.*
- float [getMeanWeight](#) (ParticleType particleType) const
- Get the mean weight for particles of a specific type.*
- float [getMinEnergy](#) (ParticleType particleType) const
- Get the minimum energy for particles of a specific type.*
- float [getMinWeight](#) (ParticleType particleType) const
- Get the minimum weight for particles of a specific type.*
- float [getMinX](#) () const
- Get the minimum X coordinate across all particles.*
- float [getMinY](#) () const
- Get the minimum Y coordinate across all particles.*
- float [getMinZ](#) () const
- Get the minimum Z coordinate across all particles.*
- unsigned int [getNumberOfExtraFloats](#) () const
- Get the number of extra float values per record.*
- unsigned int [getNumberOfExtraLongs](#) () const
- Get the number of extra integer values per record.*
- std::uint64_t [getNumberOfParticles](#) () const
- Get the total number of particles in the phase space.*
- std::uint64_t [getNumberOfParticles](#) (ParticleType particleType) const
- Get the number of particles of a specific type.*
- std::uint64_t [getOriginalHistories](#) () const
- Get the number of original simulation histories.*
- std::size_t [getRecordLength](#) () const
- Get the length of each particle record in bytes.*
- const std::string [getSection](#) (const std::string §ionName) const
- Get a header section value by name.*
- const std::string [getSection](#) (SECTION section) const
- Get a header section value by enum.*
- const std::string & [getTitle](#) () const
- Get the phase space file title.*
- float [getTotalWeight](#) (ParticleType particleType) const
- Get the total weight for particles of a specific type.*
- bool [hasExtraFloat](#) (EXTRA_FLOAT_TYPE type) const
- Check if a specific extra float type is included.*

- bool `hasExtraLong` (`EXTRA_LONG_TYPE` type) const
Check if a specific extra integer type is included.
- void `setChecksum` (std::uint64_t checksum)
Set the data integrity checksum.
- void `setConstantU` (float u)
Set the constant U direction cosine value.
- void `setConstantV` (float v)
Set the constant V direction cosine value.
- void `setConstantW` (float w)
Set the constant W direction cosine value.
- void `setConstantWeight` (float weight)
Set the constant particle weight value.
- void `setConstantX` (float x)
Set the constant X coordinate value.
- void `setConstantY` (float y)
Set the constant Y coordinate value.
- void `setConstantZ` (float z)
Set the constant Z coordinate value.
- void `setFilePath` (const std::string &filePath)
Set the file path for the header.
- void `setFileType` (FileType fileType)
Set the file type classification.
- void `setIAEAIndex` (const std::string &index)
Set the IAEA index string.
- void `setMaxEnergy` (ParticleType particleType, float maxEnergy)
Set the maximum energy for particles of a specific type.
- void `setMaxWeight` (ParticleType particleType, float maxWeight)
Set the maximum weight for particles of a specific type.
- void `setMaxX` (float maxX)
Set the maximum X coordinate boundary.
- void `setMaxY` (float maxY)
Set the maximum Y coordinate boundary.
- void `setMaxZ` (float maxZ)
Set the maximum Z coordinate boundary.
- void `setMeanEnergy` (ParticleType particleType, float meanEnergy)
Set the mean energy for particles of a specific type.
- void `setMinEnergy` (ParticleType particleType, float minEnergy)
Set the minimum energy for particles of a specific type.
- void `setMinWeight` (ParticleType particleType, float minWeight)
Set the minimum weight for particles of a specific type.
- void `setMinX` (float minX)

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

- Set the minimum X coordinate boundary.*
- void [setMinY](#) (float minY)
- Set the minimum Y coordinate boundary.*
- void [setMinZ](#) (float minZ)
- Set the minimum Z coordinate boundary.*
- void [setNumberOfParticles](#) ([ParticleType](#) particleType, std::uint64_t numberOfParticles)
- Set the number of particles for a specific type.*
- void [setNumberOfParticles](#) (std::uint64_t numberOfParticles)
- Set the total number of particles.*
- void [setOriginalHistories](#) (std::uint64_t originalHistories)
- Set the number of original simulation histories.*
- void [setRecordLength](#) (std::size_t length)
- Set the particle record length in bytes.*
- void [setSection](#) (const std::string §ionName, const std::string §ionValue)
- Set a header section value by name.*
- void [setSection](#) ([SECTION](#) section, const std::string §ionValue)
- Set a header section value using the explicit enum type.*
- void [setTitle](#) (const std::string &title)
- Set the phase space file title.*
- void [setTotalWeight](#) ([ParticleType](#) particleType, float totalWeight)
- Set the total weight for particles of a specific type.*
- bool [ulsStored](#) () const
- Check if U direction cosines are stored in records.*
- bool [vlsStored](#) () const
- Check if V direction cosines are stored in records.*
- bool [weightsStored](#) () const
- Check if particle weights are stored in records.*
- bool [wlsStored](#) () const
- Check if W direction cosines are stored in records.*
- void [writeHeader](#) ()
- Write header information to file.*
- bool [xlsStored](#) () const
- Check if X coordinates are stored in records.*
- bool [ylsStored](#) () const
- Check if Y coordinates are stored in records.*
- bool [zlsStored](#) () const
- Check if Z coordinates are stored in records.*

Static Public Member Functions

- static const std::string [DeterminePathToHeaderFile](#) (const std::string &filename)
Determine the header file path from a data file name.
- static constexpr [FloatPropertyType](#) [translateExtraFloatType](#) ([IAEAHeader::EXTRA_FLOAT_TYPE](#) type)
Convert IAEA extra float type to [ParticleZoo](#) property type.
- static constexpr [IntPropertyType](#) [translateExtraLongType](#) ([IAEAHeader::EXTRA_LONG_TYPE](#) type)
Convert IAEA extra 'long' type to [ParticleZoo](#) integer property type.

7.7.1 Detailed Description

Header manager for IAEA phase space files.

This class handles reading, writing, and manipulating the header information for IAEA format phase space files. It manages file metadata, particle statistics, data layout specifications, and validation checksums.

7.7.2 Member Enumeration Documentation

7.7.2.1 EXTRA_FLOAT_TYPE

```
enum class ParticleZoo::IAEAphspFile::IAEAHeader::EXTRA\_FLOAT\_TYPE [strong]
```

Extra float data types for IAEA format.

Defines the types of additional floating-point data that can be stored with each particle record beyond the standard IAEA format.

Enumerator

USER_DEFINED_GENERIC_TYPE	Generic user-defined float.
XLAST	Last X position.
YLAST	Last Y position.
ZLAST	Last Z position.

7.7.2.2 EXTRA_LONG_TYPE

```
enum class ParticleZoo::IAEAphspFile::IAEAHeader::EXTRA\_LONG\_TYPE [strong]
```

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

Extra integer data types for IAEA format.

Defines the types of additional integer data that can be stored with each particle record beyond the standard IAEA format. Referred to as "long" in the original IAEA documentation, however it is always a 32-bit integer on both 32-bit and 64-bit systems.

Enumerator

USER_DEFINED_GENERIC_TYPE	Generic user-defined integer.
INCREMENTAL_HISTORY_NUMBER	Sequential history number for tracking, tracks the number of new histories since the last particle was recorded.
EGS_LATCH	EGS-specific latch variable (see BEAMnrc User Manual for details)
PENELOPE_ILB5	PENELOPE ILB array value 1, corresponds to the generation of the particle (1 for primary, 2 for secondary, etc.)
PENELOPE_ILB4	PENELOPE ILB array value 2, corresponds to the particle type of the particle's parent (applies only if ILB1 > 1)
PENELOPE_ILB3	PENELOPE ILB array value 3, corresponds to the interaction type that created the particle (applies only if ILB1 > 1)
PENELOPE_ILB2	PENELOPE ILB array value 4, is non-zero if the particle is created by atomic relaxation and corresponds to the atomic transistion that created the particle.
PENELOPE_ILB1	PENELOPE ILB array value 5, a user-defined value which is passed on to all descendant particles created by this particle.

7.7.2.3 FileType

```
enum class ParticleZoo::IAEAphspFile::IAEAHeader::FileType [strong]
```

File type classification for IAEA phase space files.

Enumerator

PHSP_FILE	Standard phase space file.
PHSP_GENERATOR	Phase space generator file (as far as I know this is not used anywhere, but it exists in the original implementation)

7.7.2.4 SECTION

```
enum class ParticleZoo::IAEAphspFile::IAEAHeader::SECTION [strong]
```

Header section identifiers for IAEA format.

Defines all standard sections that can appear in an IAEA header file, used for parsing and generating header content. Includes a CUSTOM_SECTION for user-defined entries.

Enumerator

IAEA_INDEX	IAEA index code.
TITLE	File title/description.
FILE_TYPE	Either PHSP_FILE or PHSP_GENERATOR.
CHECKSUM	Data integrity checksum.
RECORD_CONTENTS	Description of record structure.
RECORD_CONSTANT	Constant values in records.
RECORD_LENGTH	Length of each particle record.
BYTE_ORDER	Byte ordering specification (endianness)
ORIGINAL_HISTORIES	Number of original simulation histories.
PARTICLES	Total particle count.
PHOTONS	Photon count and statistics.
ELECTRONS	Electron count and statistics.
POSITRONS	Positron count and statistics.
NEUTRONS	Neutron count and statistics.
PROTONS	Proton count and statistics.
TRANSPORT_PARAMETERS	Monte Carlo transport settings.
MACHINE_TYPE	Linear accelerator type.
MONTE_CARLO_CODE_VERSION	Monte Carlo code version information.
GLOBAL_PHOTON_ENERGY_CUTOFF	Global photon cutoff energy.
GLOBAL_PARTICLE_ENERGY_CUTOFF	Global particle cutoff energy.
COORDINATE_SYSTEM_DESCRIPTION	Coordinate system definition.
BEAM_NAME	Treatment beam name.
FIELD_SIZE	Radiation field dimensions.
NOMINAL SSD	Source-to-surface distance.
MC_INPUT_FILENAME	Monte Carlo input file name.
VARIANCE_REDUCTION_TECHNIQUES	Variance reduction methods used.
INITIAL_SOURCE_DESCRIPTION	Primary source description.
PUBLISHED_REFERENCE	Publication reference.
AUTHORS	File authors.
INSTITUTION	Institution name.
LINK_VALIDATION	Validation link information.
ADDITIONAL_NOTES	Additional notes.
STATISTICAL_INFORMATION_PARTICLES	Particle statistics summary.
STATISTICAL_INFORMATION_GEOMETRY	Geometric statistics summary.
CUSTOM_SECTION	User-defined section.

7.7.3 Constructor & Destructor Documentation

7.7.3.1 IAEAHeader() [1/2]

```
ParticleZoo::IAEAphspFile::IAEAHeader::IAEAHeader (
    const std::string & filePath,
    bool newFile = false )
```

Construct header from existing IAEA header file.

Parameters

<i>filePath</i>	Path to the IAEA header file (.IAEAheader)
<i>newFile</i>	If true, creates a new header; if false, reads existing file

Exceptions

<i>std::runtime_error</i>	if file cannot be read or is invalid
---------------------------	--------------------------------------

7.7.3.2 IAEAHeader() [2/2]

```
ParticleZoo::IAEAphspFile::IAEAHeader::IAEAHeader (
    const IAEAHeader & other,
    const std::string & newFilePath )
```

Copy constructor with new file path.

Creates a new header based on an existing one but with a different file path. Resets particle counts and statistics to zero.

Parameters

<i>other</i>	Source header to copy from
<i>newFilePath</i>	Path for the new header file

7.7.4 Member Function Documentation

7.7.4.1 addExtraFloat()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::addExtraFloat (
    EXTRA_FLOAT_TYPE type ) [inline]
```

Add an extra float data type to the record format.

Parameters

<i>type</i>	Type of additional floating-point data to include
-------------	---

7.7.4.2 addExtraLong()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::addExtraLong (
    EXTRA_LONG_TYPE type ) [inline]
```

Add an extra integer data type to the record format.

Parameters

<i>type</i>	Type of additional integer data to include
-------------	--

7.7.4.3 checksumIsValid()

```
bool ParticleZoo::IAEAphspFile::IAEAHeader::checksumIsValid ( ) const
```

Validate the data integrity checksum This check is strict. Not only does it verify that the checksum matches the file size, but it also checks that it equals the record length multiplied by the number of particles.

Returns

true if checksum matches expected value based on file size and record length

7.7.4.4 countParticleStats()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::countParticleStats (
    const Particle & particle ) [inline]
```

Update particle statistics with a new particle.

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

Parameters

<i>particle</i>	Particle to include in statistics calculations
-----------------	--

7.7.4.5 DeterminePathToHeaderFile()

```
const std::string ParticleZoo::IAEAphspFile::IAEAHeader::DeterminePathToHeaderFile (
    const std::string & filename ) [static]
```

Determine the header file path from a data file name.

Parameters

<i>filename</i>	Path to the data file (.IAEAphsp)
-----------------	-----------------------------------

Returns

Path to the corresponding header file (.IAEAheader)

7.7.4.6 getByteOrder()

```
ByteOrder ParticleZoo::IAEAphspFile::IAEAHeader::getByteOrder ( ) const [inline]
```

Get the byte order for binary data (endianness)

Returns

ByteOrder specification for data interpretation

7.7.4.7 getChecksum()

```
std::uint64_t ParticleZoo::IAEAphspFile::IAEAHeader::getChecksum ( ) const [inline]
```

Get the data integrity checksum.

Returns

Checksum value for data validation

7.7.4.8 `getConstantU()`

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getConstantU ( ) const [inline]
```

Get the constant U direction cosine value (when not stored per particle)

Returns

U direction cosine when not stored per particle

7.7.4.9 `getConstantV()`

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getConstantV ( ) const [inline]
```

Get the constant V direction cosine value (when not stored per particle)

Returns

V direction cosine when not stored per particle

7.7.4.10 `getConstantW()`

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getConstantW ( ) const [inline]
```

Get the constant W direction cosine value (when not implicitly stored per particle)

Returns

W direction cosine when not stored per particle

7.7.4.11 `getConstantWeight()`

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getConstantWeight ( ) const [inline]
```

Get the constant particle weight value (when not stored per particle)

Returns

Weight when not stored per particle

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

7.7.4.12 getConstantX()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getConstantX ( ) const [inline]
```

Get the constant X coordinate value (when not stored per particle)

Returns

X coordinate when not stored per particle

7.7.4.13 getConstantY()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getConstantY ( ) const [inline]
```

Get the constant Y coordinate value (when not stored per particle)

Returns

Y coordinate when not stored per particle

7.7.4.14 getConstantZ()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getConstantZ ( ) const [inline]
```

Get the constant Z coordinate value (when not stored per particle)

Returns

Z coordinate when not stored per particle

7.7.4.15 getDataFilePath()

```
std::string ParticleZoo::IAEAphspFile::IAEAHeader::getDataFilePath ( ) const
```

Get the path to the associated data file.

Returns

Path to the .IAEAphsp data file

7.7.4.16 getExtraFloatType()

```
IAEAHeader::EXTRA_FLOAT_TYPE ParticleZoo::IAEAphspFile::IAEAHeader::getExtraFloatType (
    unsigned int index ) const [inline]
```

Get the type of the extra float value at the specified index.

Parameters

<i>index</i>	Index of the extra float (0-based)
--------------	------------------------------------

Returns

EXTRA_FLOAT_TYPE describing the data type

Exceptions

<i>std::out_of_range</i>	if index is invalid
--------------------------	---------------------

7.7.4.17 getExtraLongType()

```
IAEAHeader::EXTRA_LONG_TYPE ParticleZoo::IAEAphspFile::IAEAHeader::getExtraLongType (
    unsigned int index ) const [inline]
```

Get the type of the extra integer value at the specified index.

Parameters

<i>index</i>	Index of the extra integer (0-based)
--------------	--------------------------------------

Returns

EXTRA_LONG_TYPE describing the data type

Exceptions

<i>std::out_of_range</i>	if index is invalid
--------------------------	---------------------

7.7.4.18 getFileType()

```
IAEAHeader::FileType ParticleZoo::IAEAphspFile::IAEAHeader::getFileType ( ) const [inline]
```

Get the file type classification.

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

Returns

FileType indicating PHSP_FILE or PHSP_GENERATOR

7.7.4.19 getHeaderFilePath()

```
std::string ParticleZoo::IAEAphspFile::IAEAHeader::getHeaderFilePath ( ) const [inline]
```

Get the path to the header file.

Returns

Path to the .IAEAheader file

7.7.4.20 getIAEAIndex()

```
std::string ParticleZoo::IAEAphspFile::IAEAHeader::getIAEAIndex ( ) const [inline]
```

Get the IAEA index string.

Returns

IAEA index (preserved with leading zeros if present)

7.7.4.21 getMaxEnergy()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getMaxEnergy (
    ParticleType particleType ) const
```

Get the maximum energy for particles of a specific type.

Parameters

<i>particleType</i>	Type of particle to query
---------------------	---------------------------

Returns

Maximum kinetic energy for the particle type

7.7.4.22 getMaxWeight()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getMaxWeight (
    ParticleType particleType ) const
```

Get the maximum weight for particles of a specific type.

Parameters

<i>particleType</i>	Type of particle to query
---------------------	---------------------------

Returns

Maximum weight value for the particle type

7.7.4.23 getMaxX()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getMaxX ( ) const [inline]
```

Get the maximum X coordinate across all particles.

Returns

Maximum X value in the phase space

7.7.4.24 getMaxY()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getMaxY ( ) const [inline]
```

Get the maximum Y coordinate across all particles.

Returns

Maximum Y value in the phase space

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

7.7.4.25 getMaxZ()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getMaxZ ( ) const [inline]
```

Get the maximum Z coordinate across all particles.

Returns

Maximum Z value in the phase space

7.7.4.26 getMeanEnergy()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getMeanEnergy (
    ParticleType particleType ) const
```

Get the mean energy for particles of a specific type.

Parameters

<i>particleType</i>	Type of particle to query
---------------------	---------------------------

Returns

Average kinetic energy for the particle type

7.7.4.27 getMeanWeight()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getMeanWeight (
    ParticleType particleType ) const
```

Get the mean weight for particles of a specific type.

Parameters

<i>particleType</i>	Type of particle to query
---------------------	---------------------------

Returns

Average weight value for the particle type

7.7.4.28 getMinEnergy()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getMinEnergy (
    ParticleType particleType ) const
```

Get the minimum energy for particles of a specific type.

Parameters

<i>particleType</i>	Type of particle to query
---------------------	---------------------------

Returns

Minimum kinetic energy for the particle type

7.7.4.29 getMinWeight()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getMinWeight (
    ParticleType particleType ) const
```

Get the minimum weight for particles of a specific type.

Parameters

<i>particleType</i>	Type of particle to query
---------------------	---------------------------

Returns

Minimum weight value for the particle type

7.7.4.30 getMinX()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getMinX ( ) const [inline]
```

Get the minimum X coordinate across all particles.

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

Returns

Minimum X value in the phase space

7.7.4.31 getMinY()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getMinY ( ) const [inline]
```

Get the minimum Y coordinate across all particles.

Returns

Minimum Y value in the phase space

7.7.4.32 getMinZ()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getMinZ ( ) const [inline]
```

Get the minimum Z coordinate across all particles.

Returns

Minimum Z value in the phase space

7.7.4.33 getNumberOfExtraFloats()

```
unsigned int ParticleZoo::IAEAphspFile::IAEAHeader::getNumberOfExtraFloats ( ) const [inline]
```

Get the number of extra float values per record.

Returns

Count of additional floating-point values

7.7.4.34 getNumberOfExtraLongs()

```
unsigned int ParticleZoo::IAEAphspFile::IAEAHeader::getNumberOfExtraLongs ( ) const [inline]
```

Get the number of extra integer values per record.

Returns

Count of additional integer values

7.7.4.35 getNumberOfParticles() [1/2]

```
std::uint64_t ParticleZoo::IAEAphspFile::IAEAHeader::getNumberOfParticles ( ) const [inline]
```

Get the total number of particles in the phase space.

Returns

Total particle count across all types

7.7.4.36 getNumberOfParticles() [2/2]

```
std::uint64_t ParticleZoo::IAEAphspFile::IAEAHeader::getNumberOfParticles (
    ParticleType particleType ) const
```

Get the number of particles of a specific type.

Parameters

<i>particleType</i>	Type of particle to count
---------------------	---------------------------

Returns

Number of particles of the specified type

7.7.4.37 getOriginalHistories()

```
std::uint64_t ParticleZoo::IAEAphspFile::IAEAHeader::getOriginalHistories ( ) const [inline]
```

Get the number of original simulation histories.

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

Returns

The number of primary histories used to generate the phase space

7.7.4.38 getRecordLength()

```
std::size_t ParticleZoo::IAEAphspFile::IAEAHeader::getRecordLength ( ) const [inline]
```

Get the length of each particle record in bytes.

Returns

Record length in bytes

7.7.4.39 getSection() [1/2]

```
const std::string ParticleZoo::IAEAphspFile::IAEAHeader::getSection (
    const std::string & sectionName ) const
```

Get a header section value by name.

Parameters

<i>sectionName</i>	Name of the section to retrieve
--------------------	---------------------------------

Returns

Section content as string, "UNKNOWN" if not found

7.7.4.40 getSection() [2/2]

```
const std::string ParticleZoo::IAEAphspFile::IAEAHeader::getSection (
    SECTION section ) const
```

Get a header section value by enum.

Parameters

<i>section</i>	Section identifier to retrieve
----------------	--------------------------------

Returns

Section content as string, empty if not found

7.7.4.41 getTitle()

```
const std::string & ParticleZoo::IAEAphspFile::IAEAHeader::getTitle ( ) const [inline]
```

Get the phase space file title.

Returns

Title string describing the phase space file

7.7.4.42 getTotalWeight()

```
float ParticleZoo::IAEAphspFile::IAEAHeader::getTotalWeight (
    ParticleType particleType ) const
```

Get the total weight for particles of a specific type.

Parameters

<i>particleType</i>	Type of particle to query
---------------------	---------------------------

Returns

Sum of all weights for the particle type

7.7.4.43 hasExtraFloat()

```
bool ParticleZoo::IAEAphspFile::IAEAHeader::hasExtraFloat (
    EXTRA_FLOAT_TYPE type ) const [inline]
```

Check if a specific extra float type is included.

Parameters

<i>type</i>	Extra float type to check for
-------------	-------------------------------

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

Returns

true if the type is included in the record format

7.7.4.44 hasExtraLong()

```
bool ParticleZoo::IAEAphspFile::IAEAHeader::hasExtraLong (
    EXTRA_LONG_TYPE type ) const [inline]
```

Check if a specific extra integer type is included.

Parameters

<i>type</i>	Extra integer type to check for
-------------	---------------------------------

Returns

true if the type is included in the record format

7.7.4.45 setChecksum()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setChecksum (
    std::uint64_t checksum ) [inline]
```

Set the data integrity checksum.

Parameters

<i>checksum</i>	New checksum value
-----------------	--------------------

7.7.4.46 setConstantU()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setConstantU (
    float u ) [inline]
```

Set the constant U direction cosine value.

Parameters

<i>u</i>	U direction cosine for all particles
----------	--------------------------------------

7.7.4.47 setConstantV()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setConstantV (
    float v ) [inline]
```

Set the constant V direction cosine value.

Parameters

<i>v</i>	V direction cosine for all particles
----------	--------------------------------------

7.7.4.48 setConstantW()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setConstantW (
    float w ) [inline]
```

Set the constant W direction cosine value.

Parameters

<i>w</i>	W direction cosine for all particles
----------	--------------------------------------

7.7.4.49 setConstantWeight()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setConstantWeight (
    float weight ) [inline]
```

Set the constant particle weight value.

Parameters

<i>weight</i>	Weight for all particles
---------------	--------------------------

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

7.7.4.50 setConstantX()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setConstantX (
    float x ) [inline]
```

Set the constant X coordinate value.

Parameters

x	X coordinate for all particles
---	--------------------------------

7.7.4.51 setConstantY()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setConstantY (
    float y ) [inline]
```

Set the constant Y coordinate value.

Parameters

y	Y coordinate for all particles
---	--------------------------------

7.7.4.52 setConstantZ()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setConstantZ (
    float z ) [inline]
```

Set the constant Z coordinate value.

Parameters

z	Z coordinate for all particles
---	--------------------------------

7.7.4.53 setFilePath()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setFilePath (
    const std::string & filePath ) [inline]
```

Set the file path for the header.

Parameters

<i>filePath</i>	New path to the .IAEAheader file
-----------------	----------------------------------

7.7.4.54 setFileType()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setFileType (
    IAEAHeader::FileType fileType ) [inline]
```

Set the file type classification.

Parameters

<i>fileType</i>	Type specification (PHSP_FILE or PHSP_GENERATOR)
-----------------	--

7.7.4.55 setIAEAIndex()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setIAEAIndex (
    const std::string & index ) [inline]
```

Set the IAEA index string.

Parameters

<i>index</i>	New IAEA index identifier
--------------	---------------------------

7.7.4.56 setMaxEnergy()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setMaxEnergy (
    ParticleType particleType,
    float maxEnergy ) [inline]
```

Set the maximum energy for particles of a specific type.

Parameters

<i>particleType</i>	Type of particle to set statistics for
<i>maxEnergy</i>	Maximum kinetic energy for this particle type

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

7.7.4.57 setMaxWeight()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setMaxWeight (
    ParticleType particleType,
    float maxWeight ) [inline]
```

Set the maximum weight for particles of a specific type.

Parameters

<i>particleType</i>	Type of particle to set statistics for
<i>maxWeight</i>	Maximum weight value for this particle type

7.7.4.58 setMaxX()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setMaxX (
    float maxX ) [inline]
```

Set the maximum X coordinate boundary.

Parameters

<i>maxX</i>	Maximum X value in the phase space
-------------	------------------------------------

7.7.4.59 setMaxY()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setMaxY (
    float maxY ) [inline]
```

Set the maximum Y coordinate boundary.

Parameters

<i>maxY</i>	Maximum Y value in the phase space
-------------	------------------------------------

7.7.4.60 setMaxZ()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setMaxZ (
    float maxZ ) [inline]
```

Set the maximum Z coordinate boundary.

Parameters

<i>maxZ</i>	Maximum Z value in the phase space
-------------	------------------------------------

7.7.4.61 setMeanEnergy()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setMeanEnergy (
    ParticleType particleType,
    float meanEnergy ) [inline]
```

Set the mean energy for particles of a specific type.

Parameters

<i>particleType</i>	Type of particle to set statistics for
<i>meanEnergy</i>	Average kinetic energy for this particle type

7.7.4.62 setMinEnergy()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setMinEnergy (
    ParticleType particleType,
    float minEnergy ) [inline]
```

Set the minimum energy for particles of a specific type.

Parameters

<i>particleType</i>	Type of particle to set statistics for
<i>minEnergy</i>	Minimum kinetic energy for this particle type

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

7.7.4.63 setMinWeight()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setMinWeight (
    ParticleType particleType,
    float minWeight ) [inline]
```

Set the minimum weight for particles of a specific type.

Parameters

<i>particleType</i>	Type of particle to set statistics for
<i>minWeight</i>	Minimum weight value for this particle type

7.7.4.64 setMinX()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setMinX (
    float minX ) [inline]
```

Set the minimum X coordinate boundary.

Parameters

<i>minX</i>	Minimum X value in the phase space
-------------	------------------------------------

7.7.4.65 setMinY()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setMinY (
    float minY ) [inline]
```

Set the minimum Y coordinate boundary.

Parameters

<i>minY</i>	Minimum Y value in the phase space
-------------	------------------------------------

7.7.4.66 `setMinZ()`

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setMinZ (
    float minZ ) [inline]
```

Set the minimum Z coordinate boundary.

Parameters

<i>minZ</i>	Minimum Z value in the phase space
-------------	------------------------------------

7.7.4.67 `setNumberOfParticles()` [1/2]

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setNumberOfParticles (
    ParticleType particleType,
    std::uint64_t numberOfParticles ) [inline]
```

Set the number of particles for a specific type.

Parameters

<i>particleType</i>	Type of particle to set count for
<i>numberOfParticles</i>	Number of particles of this type

7.7.4.68 `setNumberOfParticles()` [2/2]

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setNumberOfParticles (
    std::uint64_t numberOfParticles ) [inline]
```

Set the total number of particles.

Parameters

<i>numberOfParticles</i>	Total particle count across all types
--------------------------	---------------------------------------

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

7.7.4.69 setOriginalHistories()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setOriginalHistories (
    std::uint64_t originalHistories ) [inline]
```

Set the number of original simulation histories.

Parameters

<i>originalHistories</i>	Count of primary histories
--------------------------	----------------------------

7.7.4.70 setRecordLength()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setRecordLength (
    std::size_t length ) [inline]
```

Set the particle record length in bytes.

Parameters

<i>length</i>	New record length for each particle
---------------	-------------------------------------

7.7.4.71 setSection() [1/2]

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setSection (
    const std::string & sectionName,
    const std::string & sectionValue )
```

Set a header section value by name.

Parameters

<i>sectionName</i>	Name of the section to set
<i>sectionValue</i>	Content to store in the section

7.7.4.72 setSection() [2/2]

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setSection (
    SECTION section,
    const std::string & sectionValue )
```

Set a header section value using the explicit enum type.

Parameters

<i>section</i>	Section identifier to set
<i>sectionValue</i>	Content to store in the section

7.7.4.73 setTitle()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setTitle (
    const std::string & title ) [inline]
```

Set the phase space file title.

Parameters

<i>title</i>	New title for the file
--------------	------------------------

7.7.4.74 setTotalWeight()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::setTotalWeight (
    ParticleType particleType,
    float totalWeight ) [inline]
```

Set the total weight for particles of a specific type.

Parameters

<i>particleType</i>	Type of particle to set statistics for
<i>totalWeight</i>	Sum of all weights for this particle type

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

7.7.4.75 translateExtraFloatType()

```
constexpr FloatPropertyType ParticleZoo::IAEAphspFile::IAEAHeader::translateExtraFloatType (
    IAEAHeader::EXTRA_FLOAT_TYPE type ) [static], [constexpr]
```

Convert IAEA extra float type to [ParticleZoo](#) property type.

Parameters

<i>type</i>	IAEA-specific extra float type
-------------	--------------------------------

Returns

Corresponding [ParticleZoo](#) FloatPropertyType

7.7.4.76 translateExtraLongType()

```
constexpr IntPropertyType ParticleZoo::IAEAphspFile::IAEAHeader::translateExtraLongType (
    IAEAHeader::EXTRA_LONG_TYPE type ) [static], [constexpr]
```

Convert IAEA extra 'long' type to [ParticleZoo](#) integer property type.

Parameters

<i>type</i>	IAEA-specific extra 'long' type
-------------	---------------------------------

Returns

Corresponding [ParticleZoo](#) IntPropertyType

7.7.4.77 uIsStored()

```
bool ParticleZoo::IAEAphspFile::IAEAHeader::uIsStored ( ) const [inline]
```

Check if U direction cosines are stored in records.

Returns

true if U values are stored, false if constant

7.7.4.78 vIsStored()

```
bool ParticleZoo::IAEAphspFile::IAEAHeader::vIsStored ( ) const [inline]
```

Check if V direction cosines are stored in records.

Returns

true if V values are stored, false if constant

7.7.4.79 weightIsStored()

```
bool ParticleZoo::IAEAphspFile::IAEAHeader::weightIsStored ( ) const [inline]
```

Check if particle weights are stored in records.

Returns

true if weights are stored, false if constant

7.7.4.80 wIsStored()

```
bool ParticleZoo::IAEAphspFile::IAEAHeader::wIsStored ( ) const [inline]
```

Check if W direction cosines are stored in records.

Note

W being 'stored' means that it is not a constant value recorded in the header. The 'stored' value is only implicitly stored and is actually calculated as needed from U and V.

Returns

true if W values are stored, false if constant

7.7.4.81 writeHeader()

```
void ParticleZoo::IAEAphspFile::IAEAHeader::writeHeader ( )
```

Write header information to file.

Writes the complete header information to the associated .IAEAheader file, including all sections, particle statistics, and metadata.

7.7 ParticleZoo::IAEAphspFile::IAEAHeader Class Reference

Exceptions

<code>std::runtime_error</code>	if file cannot be written
---------------------------------	---------------------------

7.7.4.82 xlsStored()

```
bool ParticleZoo::IAEAphspFile::IAEAHeader::xlsStored ( ) const [inline]
```

Check if X coordinates are stored in records.

Returns

true if X values are stored, false if constant

7.7.4.83 ylsStored()

```
bool ParticleZoo::IAEAphspFile::IAEAHeader::yIsStored ( ) const [inline]
```

Check if Y coordinates are stored in records.

Returns

true if Y values are stored, false if constant

7.7.4.84 zlsStored()

```
bool ParticleZoo::IAEAphspFile::IAEAHeader::zIsStored ( ) const [inline]
```

Check if Z coordinates are stored in records.

Returns

true if Z values are stored, false if constant

The documentation for this class was generated from the following files:

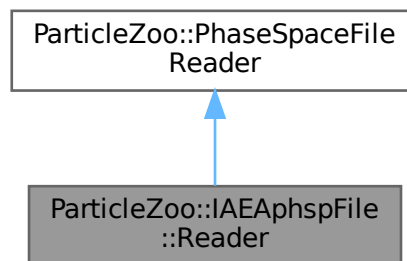
- include/particlezoo/IAEA/IAEAHeader.h
- src/IAEA/IAEAHeader.cc

7.8 ParticleZoo::IAEAphspFile::Reader Class Reference

[Reader](#) for IAEA format phase space files.

```
#include <particlezoo/IAEA/IAEAphspFile.h>
```

Inheritance diagram for ParticleZoo::IAEAphspFile::Reader:



Public Member Functions

- [Reader](#) (const std::string &filename, const UserOptions &options=UserOptions{})
Construct reader for IAEA phase space file.
- const [IAEAHeader](#) & [getHeader](#) () const
Get access to the IAEA header information.
- std::uint64_t [getNumberOfOriginalHistories](#) () const override
Get the number of original simulation histories.
- std::uint64_t [getNumberOfParticles](#) () const override
Get the total number of particles in the phase space.
- std::uint64_t [getNumberOfParticles](#) ([ParticleType](#) particleType) const
Get the number of particles of a specific type.
- bool [hasNativeIncrementalHistoryCounters](#) () const override
Check if this format stores incremental history numbers per-particle.

Public Member Functions inherited from ParticleZoo::PhaseSpaceFileReader

- **PhaseSpaceFileReader** (const std::string &phspFormat, const std::string &fileName, const UserOptions &userOptions, **FormatType** formatType=**FormatType::BINARY**, const **FixedValues** fixedValues=**FixedValues()**, unsigned int bufferSize=**DEFAULT_BUFFER_SIZE**)
Construct a new Phase Space File Reader object.
- virtual **~PhaseSpaceFileReader** ()
Destroy the Phase Space File Reader object.
- void **close** ()
Close the phase space file and clean up resources.
- float **getConstantPx** () const
Get the constant X-component of the direction unit vector (if constant).
- float **getConstantPy** () const
Get the constant Y-component of the direction unit vector (if constant).
- float **getConstantPz** () const
Get the constant Z-component of the direction unit vector (if constant).
- float **getConstantWeight** () const
Get the constant statistical weight value (if constant).
- float **getConstantX** () const
Get the constant X coordinate value (if constant).
- float **getConstantY** () const
Get the constant Y coordinate value (if constant).
- float **getConstantZ** () const
Get the constant Z coordinate value (if constant).
- const std::string **getFileName** () const
Get the filename of the phase space file being read.
- std::uint64_t **getFileSize** () const
Get the size of the phase space file in bytes.
- const **FixedValues** **getFixedValues** () const
Get the fixed values configuration.
- virtual std::uint64_t **getHistoriesRead** ()
Get the number of Monte Carlo histories that have been read so far. If the end of the file has been reached, this will return the total number of original histories unless more histories than expected have already been read - in which case it returns the actual count.
- **Particle** **getNextParticle** ()
Get the next particle from the phase space file.
- virtual std::uint64_t **getNumberOfRepresentedHistories** () const
Get the number of histories actually represented in this phase space.
- virtual std::uint64_t **getParticlesRead** ()
Get the number of particles that have been read so far.
- const std::string **getPHSPFormat** () const
Get the phase space file format identifier.

- virtual bool [hasMoreParticles](#) ()
Check if there are more particles to read in the file.
- virtual bool [hasNativeRepresentedHistoryCount](#) () const
Check if this format can provide the number of represented histories without scanning the file.
- bool [isPxConstant](#) () const
Check if the X-component of momentum is constant for all particles.
- bool [isPyConstant](#) () const
Check if the Y-component of momentum is constant for all particles.
- bool [isPzConstant](#) () const
Check if the Z-component of momentum is constant for all particles.
- bool [isWeightConstant](#) () const
Check if the statistical weight is constant for all particles.
- bool [isXConstant](#) () const
Check if the X coordinate is constant for all particles.
- bool [isYConstant](#) () const
Check if the Y coordinate is constant for all particles.
- bool [isZConstant](#) () const
Check if the Z coordinate is constant for all particles.
- void [moveToParticle](#) (std::uint64_t particleIndex)
Move the file position to a specific particle index.
- [Particle peekNextParticle](#) ()
Peek at the next particle without advancing the file position.
- void [setCommentMarkers](#) (const std::vector< std::string > &commentMarkers)
Set comment markers for ASCII format files.

Static Public Member Functions

- static std::vector< CLICommand > [getFormatSpecificCLICommands](#) ()
Get format-specific command-line options.

Static Public Member Functions inherited from [ParticleZoo::PhaseSpaceFileReader](#)

- static std::vector< CLICommand > [getCLICommands](#) ()
Get command line interface commands supported by this reader.

Protected Member Functions

- std::size_t [getParticleRecordLength](#) () const override
Get the length of each particle record in bytes.
- std::size_t [getParticleRecordStartOffset](#) () const override
Get the byte offset where particle records start.
- [Particle readBinaryParticle](#) (ByteBuffer &buffer) override
Read and decode a single particle from binary data.

Protected Member Functions inherited from ParticleZoo::PhaseSpaceFileReader

- double [calcThirdUnitComponent](#) (double &u, double &v) const
Calculate the third component of a unit vector from two components (double precision).
- float [calcThirdUnitComponent](#) (float &u, float &v) const
Calculate the third component of a unit vector from two components (float precision).
- const [ByteBuffer](#) [getHeaderData](#) ()
Get the file header data as a byte buffer.
- const [ByteBuffer](#) [getHeaderData](#) (std::size_t headerSize)
Get a specific amount of header data as a byte buffer.
- virtual std::size_t [getMaximumASCIILineLength](#) () const
Get the maximum line length for ASCII format files.
- [Particle](#) [getNextParticle](#) (bool countParticleInStatistics)
Get the next particle with optional statistics counting control.
- std::size_t [getNumberOfEntriesInFile](#) () const
Get the number of particle records that fit in the file.
- virtual std::uint64_t [getParticlesRead](#) (bool includeSkippedParticles)
Get the number of particles read with optional inclusion of skipped particles (including pseudo-particles).
- const UserOptions & [getUserOptions](#) () const
Get the user options that were passed to the constructor.
- virtual [Particle](#) [peekParticleManually](#) ()
Peek at a particle manually (for formats requiring third-party I/O).
- virtual [Particle](#) [readASCIIParticle](#) (const std::string &line)
Read a particle from ASCII data.
- virtual [Particle](#) [readParticleManually](#) ()
Read a particle manually (for formats requiring third-party I/O).
- void [setByteOrder](#) ([ByteOrder](#) byteOrder)
Set the byte order for binary data interpretation.
- void [setConstantPx](#) (float Px)
Set a constant X-component of the direction unit vector for all particles.
- void [setConstantPy](#) (float Py)
Set a constant Y-component of the direction unit vector for all particles.
- void [setConstantPz](#) (float Pz)
Set a constant Z-component of the direction unit vector for all particles.
- void [setConstantWeight](#) (float weight)
Set a constant statistical weight for all particles.
- void [setConstantX](#) (float X)
Set a constant X coordinate value for all particles.
- void [setConstantY](#) (float Y)
Set a constant Y coordinate value for all particles.
- void [setConstantZ](#) (float Z)
Set a constant Z coordinate value for all particles.

7.8.1 Detailed Description

[Reader](#) for IAEA format phase space files.

Provides functionality to read phase space data from IAEA format files, handling both header parsing and binary particle data extraction. Supports all standard IAEA features including extra float/long data types.

7.8.2 Constructor & Destructor Documentation

7.8.2.1 Reader()

```
ParticleZoo::IAEAphspFile::Reader::Reader (
    const std::string & filename,
    const UserOptions & options = UserOptions{} )
```

Construct reader for IAEA phase space file.

Parameters

<i>filename</i>	Path to the IAEA phase space data file (.IAEAphsp)
<i>options</i>	User-specified options for reading behavior

Exceptions

<i>std::runtime_error</i>	if file cannot be opened or header is invalid
---------------------------	---

7.8.3 Member Function Documentation

7.8.3.1 getFormatSpecificCLICommands()

```
std::vector< CLICommand > ParticleZoo::IAEAphspFile::Reader::getFormatSpecificCLICommands ( )
[static]
```

Get format-specific command-line options.

Returns

Vector of CLI commands supported by IAEA reader

7.8 ParticleZoo::IAEAphspFile::Reader Class Reference

7.8.3.2 getHeader()

```
const IAEAHeader & ParticleZoo::IAEAphspFile::Reader::getHeader ( ) const [inline]
```

Get access to the IAEA header information.

Returns

Reference to the header containing file metadata

7.8.3.3 getNumberOfOriginalHistories()

```
std::uint64_t ParticleZoo::IAEAphspFile::Reader::getNumberOfOriginalHistories ( ) const [inline],  
[override], [virtual]
```

Get the number of original simulation histories.

Returns

Count of primary histories used in the simulation

Implements [ParticleZoo::PhaseSpaceFileReader](#).

7.8.3.4 getNumberOfParticles() [1/2]

```
std::uint64_t ParticleZoo::IAEAphspFile::Reader::getNumberOfParticles ( ) const [inline], [override],  
[virtual]
```

Get the total number of particles in the phase space.

Returns

Total particle count across all types

Implements [ParticleZoo::PhaseSpaceFileReader](#).

7.8.3.5 getNumberOfParticles() [2/2]

```
std::uint64_t ParticleZoo::IAEAphspFile::Reader::getNumberOfParticles (  
    ParticleType particleType ) const [inline]
```

Get the number of particles of a specific type.

Parameters

<i>particleType</i>	Type of particle to count
---------------------	---------------------------

Returns

Number of particles of the specified type

7.8.3.6 getParticleRecordLength()

```
std::size_t ParticleZoo::IAEAphspFile::Reader::getParticleRecordLength ( ) const [inline], [override],  
[protected], [virtual]
```

Get the length of each particle record in bytes.

Returns

Size of each particle record as defined in header

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

7.8.3.7 getParticleRecordStartOffset()

```
std::size_t ParticleZoo::IAEAphspFile::Reader::getParticleRecordStartOffset ( ) const [inline],  
[override], [protected], [virtual]
```

Get the byte offset where particle records start.

Returns

Starting offset for particle data (0 for IAEA format)

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

7.8 ParticleZoo::IAEAphspFile::Reader Class Reference

7.8.3.8 hasNativeIncrementalHistoryCounters()

```
bool ParticleZoo::IAEAphspFile::Reader::hasNativeIncrementalHistoryCounters ( ) const [inline],  
[override], [virtual]
```

Check if this format stores incremental history numbers per-particle.

Returns

true if the header has an extra long of type INCREMENTAL_HISTORY_NUMBER

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

7.8.3.9 readBinaryParticle()

```
Particle ParticleZoo::IAEAphspFile::Reader::readBinaryParticle (   
    ByteBuffer & buffer ) [override], [protected], [virtual]
```

Read and decode a single particle from binary data.

Parameters

<i>buffer</i>	Binary buffer containing particle data
---------------	--

Returns

Decoded [Particle](#) object with all properties

Exceptions

<i>std::runtime_error</i>	if particle data is corrupted or invalid
---------------------------	--

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

The documentation for this class was generated from the following files:

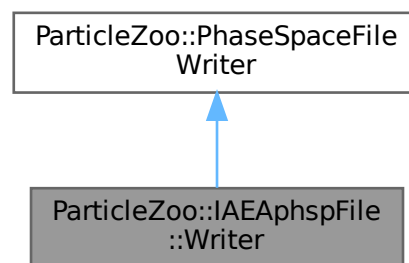
- include/particlezoo/IAEA/IAEAphspFile.h
- src/IAEA/IAEAphspFile.cc

7.9 ParticleZoo::IAEAphspFile::Writer Class Reference

[Writer](#) for IAEA format phase space files.

```
#include <particlezoo/IAEA/IAEAphspFile.h>
```

Inheritance diagram for ParticleZoo::IAEAphspFile::Writer:



Public Member Functions

- [Writer](#) (const std::string &filename, const [IAEAHeader](#) &templateHeader)
Construct writer using an existing header as template.
- [Writer](#) (const std::string &filename, const UserOptions &userOptions=UserOptions{}, const [FixedValues](#) &fixed←
Values=[FixedValues](#){})
Construct writer for new IAEA phase space file.
- [IAEAHeader](#) & [getHeader](#) ()
Get access to the IAEA header for configuration.
- std::uint64_t [getMaximumSupportedParticles](#) () const override
Get the maximum number of particles this format can store.
- void [setNumberOfOriginalHistories](#) (std::uint64_t numberOfHistories)
Set the number of original simulation histories.

Public Member Functions inherited from ParticleZoo::PhaseSpaceFileWriter

- **PhaseSpaceFileWriter** (const std::string &phspFormat, const std::string &fileName, const UserOptions &userOptions, **FormatType** formatType=**FormatType::BINARY**, const **FixedValues** fixedValues=**FixedValues()**, unsigned int bufferSize=**DEFAULT_BUFFER_SIZE**)
Construct a new Phase Space File Writer object.
- virtual **~PhaseSpaceFileWriter** ()
Destroy the Phase Space File Writer object.
- void **addAdditionalHistories** (std::uint64_t additionalHistories)
Add additional Monte Carlo histories to the count.
- void **close** ()
Close the phase space file and finalize writing.
- **ByteOrder** **getByteOrder** () const
Get the byte order used for binary data writing.
- float **getConstantPx** () const
Get the constant X-component of the direction unit vector (if constant).
- float **getConstantPy** () const
Get the constant Y-component of the direction unit vector (if constant).
- float **getConstantPz** () const
Get the constant Z-component of the direction unit vector (if constant).
- float **getConstantWeight** () const
Get the constant statistical weight value (if constant).
- float **getConstantX** () const
Get the constant X coordinate value (if constant).
- float **getConstantY** () const
Get the constant Y coordinate value (if constant).
- float **getConstantZ** () const
Get the constant Z coordinate value (if constant).
- const std::string **getFileName** () const
Get the filename where the phase space file is being written.
- const **FixedValues** **getFixedValues** () const
Get the fixed values configuration.
- virtual std::uint64_t **getHistoriesWritten** () const
Get the number of Monte Carlo histories that have been written.
- std::uint64_t **getParticlesWritten** () const
Get the number of particles that have been written to the file.
- const std::string **getPHSPFormat** () const
Get the phase space file format identifier.
- bool **isPxConstant** () const
Check if the X-component of the direction unit vector is set to a constant value for all particles.
- bool **isPyConstant** () const
Check if the Y-component of the direction unit vector is set to a constant value for all particles.

- bool [isPzConstant](#) () const
Check if the Z-component of the direction unit vector is set to a constant value for all particles.
- bool [isWeightConstant](#) () const
Check if the statistical weight is set to a constant value for all particles.
- bool [isXConstant](#) () const
Check if the X coordinate is set to a constant value for all particles.
- bool [isYConstant](#) () const
Check if the Y coordinate is set to a constant value for all particles.
- bool [isZConstant](#) () const
Check if the Z coordinate is set to a constant value for all particles.
- virtual void [writeParticle](#) ([Particle](#) particle)
Write a particle to the phase space file.

Static Public Member Functions

- static std::vector< [CLICCommand](#) > [getFormatSpecificCLICommands](#) ()
Get format-specific command-line options.

Static Public Member Functions inherited from [ParticleZoo::PhaseSpaceFileWriter](#)

- static std::vector< [CLICCommand](#) > [getCLICommands](#) ()
Get command line interface commands supported by this writer.

Protected Member Functions

- bool [canHaveConstantPx](#) () const override
Check if constant X momentum components are supported.
- bool [canHaveConstantPy](#) () const override
Check if constant Y momentum components are supported.
- bool [canHaveConstantPz](#) () const override
Check if constant Z momentum components are supported.
- bool [canHaveConstantWeight](#) () const override
Check if constant particle weights are supported.
- bool [canHaveConstantX](#) () const override
Check if constant X coordinates are supported.
- bool [canHaveConstantY](#) () const override
Check if constant Y coordinates are supported.
- bool [canHaveConstantZ](#) () const override
Check if constant Z coordinates are supported.

7.9 ParticleZoo::IAEAphspFile::Writer Class Reference

- void [fixedValuesHaveChanged](#) () override
Handle changes to fixed/constant values.
- std::size_t [getParticleRecordLength](#) () const override
Get the length of each particle record in bytes.
- void [writeBinaryParticle](#) (ByteBuffer &buffer, Particle &particle) override
Encode and write a single particle to binary data.
- void [writeHeaderData](#) (ByteBuffer &buffer) override
Write header data to the output buffer (not used for IAEA)

Protected Member Functions inherited from ParticleZoo::PhaseSpaceFileWriter

- virtual bool [accountForAdditionalHistories](#) (std::uint64_t additionalHistories)
Handle accounting for simulation histories that produced no particles.
- virtual bool [canWritePseudoParticlesExplicitly](#) () const
Check if this format can write pseudo-particles explicitly.
- virtual size_t [getMaximumASCIILineLength](#) () const
Get the maximum line length for ASCII format files.
- virtual std::size_t [getParticleRecordStartOffset](#) () const
Get the byte offset where particle records start in the file.
- virtual std::uint64_t [getPendingHistories](#) () const
Get the number of pending histories to account for.
- const UserOptions & [getUserOptions](#) () const
Get the user options that were passed to the constructor.
- void [setByteOrder](#) (ByteOrder byteOrder)
Set the byte order for binary data writing.
- void [setConstantPx](#) (float Px)
Set a constant X-component of the direction unit vector for all particles.
- void [setConstantPy](#) (float Py)
Set a constant Y-component of the direction unit vector for all particles.
- void [setConstantPz](#) (float Pz)
Set a constant Z-component of the direction unit vector for all particles.
- void [setConstantWeight](#) (float weight)
Set a constant statistical weight for all particles.
- void [setConstantX](#) (float X)
Set a constant X coordinate value for all particles.
- void [setConstantY](#) (float Y)
Set a constant Y coordinate value for all particles.
- void [setConstantZ](#) (float Z)
Set a constant Z coordinate value for all particles.
- virtual const std::string [writeASCIIParticle](#) (Particle &particle)
Write a particle in ASCII format as a string.
- virtual void [writeParticleManually](#) (Particle &particle)
Write a particle manually (for formats requiring third-party I/O).

7.9.1 Detailed Description

[Writer](#) for IAEA format phase space files.

Provides functionality to write phase space data to IAEA format files, handling header generation and binary particle data encoding. Supports all standard IAEA features and optional data types.

7.9.2 Constructor & Destructor Documentation

7.9.2.1 Writer() [1/2]

```
ParticleZoo::IAEAphspFile::Writer::Writer (
    const std::string & filename,
    const UserOptions & userOptions = UserOptions{},
    const FixedValues & fixedValues = FixedValues{} )
```

Construct writer for new IAEA phase space file.

Parameters

<i>filename</i>	Path for the new IAEA phase space data file (.IAEAphsp)
<i>userOptions</i>	User-specified options for writing behavior
<i>fixedValues</i>	Constant values to optimize storage

Exceptions

<i>std::runtime_error</i>	if file cannot be created
---------------------------	---------------------------

7.9.2.2 Writer() [2/2]

```
ParticleZoo::IAEAphspFile::Writer::Writer (
    const std::string & filename,
    const IAEAHeader & templateHeader )
```

Construct writer using an existing header as template.

Parameters

<i>filename</i>	Path for the new IAEA phase space data file (.IAEAphsp)
<i>templateHeader</i>	Existing header to copy configuration from

7.9.3 Member Function Documentation

7.9.3.1 canHaveConstantPx()

```
bool ParticleZoo::IAEAphspFile::Writer::canHaveConstantPx ( ) const [inline], [override], [protected], [virtual]
```

Check if constant X momentum components are supported.

Returns

true (IAEA format supports constant U direction cosines)

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.9.3.2 canHaveConstantPy()

```
bool ParticleZoo::IAEAphspFile::Writer::canHaveConstantPy ( ) const [inline], [override], [protected], [virtual]
```

Check if constant Y momentum components are supported.

Returns

true (IAEA format supports constant V direction cosines)

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.9.3.3 canHaveConstantPz()

```
bool ParticleZoo::IAEAphspFile::Writer::canHaveConstantPz ( ) const [inline], [override], [protected], [virtual]
```

Check if constant Z momentum components are supported.

Returns

true (IAEA format supports constant W direction cosines)

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.9.3.4 canHaveConstantWeight()

```
bool ParticleZoo::IAEAphspFile::Writer::canHaveConstantWeight ( ) const [inline], [override],  
[protected], [virtual]
```

Check if constant particle weights are supported.

Returns

true (IAEA format supports constant weight values)

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.9.3.5 canHaveConstantX()

```
bool ParticleZoo::IAEAphspFile::Writer::canHaveConstantX ( ) const [inline], [override], [protected],  
[virtual]
```

Check if constant X coordinates are supported.

Returns

true (IAEA format supports constant X values)

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.9.3.6 canHaveConstantY()

```
bool ParticleZoo::IAEAphspFile::Writer::canHaveConstantY ( ) const [inline], [override], [protected],  
[virtual]
```

Check if constant Y coordinates are supported.

Returns

true (IAEA format supports constant Y values)

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.9 ParticleZoo::IAEAphspFile::Writer Class Reference

7.9.3.7 canHaveConstantZ()

```
bool ParticleZoo::IAEAphspFile::Writer::canHaveConstantZ ( ) const [inline], [override], [protected], [virtual]
```

Check if constant Z coordinates are supported.

Returns

true (IAEA format supports constant Z values)

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.9.3.8 fixedValuesHaveChanged()

```
void ParticleZoo::IAEAphspFile::Writer::fixedValuesHaveChanged ( ) [inline], [override], [protected], [virtual]
```

Handle changes to fixed/constant values.

Updates the IAEA header when constant values are modified, ensuring the header reflects the current optimization settings.

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.9.3.9 getFormatSpecificCLICommands()

```
std::vector< CLICommand > ParticleZoo::IAEAphspFile::Writer::getFormatSpecificCLICommands ( ) [static]
```

Get format-specific command-line options.

Returns

Vector of CLI commands supported by IAEA writer

7.9.3.10 getHeader()

```
IAEAHeader & ParticleZoo::IAEAphspFile::Writer::getHeader ( ) [inline]
```

Get access to the IAEA header for configuration.

Returns

Reference to the header

7.9.3.11 `getMaximumSupportedParticles()`

```
std::uint64_t ParticleZoo::IAEAphspFile::Writer::getMaximumSupportedParticles ( ) const [inline],  
[override], [virtual]
```

Get the maximum number of particles this format can store.

Returns

Maximum particle count (effectively unlimited for IAEA)

Implements [ParticleZoo::PhaseSpaceFileWriter](#).

7.9.3.12 `getParticleRecordLength()`

```
std::size_t ParticleZoo::IAEAphspFile::Writer::getParticleRecordLength ( ) const [inline], [override],  
[protected], [virtual]
```

Get the length of each particle record in bytes.

Returns

Size of each particle record as configured in header

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.9.3.13 `setNumberOfOriginalHistories()`

```
void ParticleZoo::IAEAphspFile::Writer::setNumberOfOriginalHistories (   
    std::uint64_t numberOfHistories )
```

Set the number of original simulation histories.

Parameters

<i>numberOfHistories</i>	Number of original histories to record in header
--------------------------	--

7.9 ParticleZoo::IAEAphspFile::Writer Class Reference

7.9.3.14 writeBinaryParticle()

```
void ParticleZoo::IAEAphspFile::Writer::writeBinaryParticle (
    ByteBuffer & buffer,
    Particle & particle ) [override], [protected], [virtual]
```

Encode and write a single particle to binary data.

Parameters

<i>buffer</i>	Binary buffer to write particle data to
<i>particle</i>	Particle object to encode and store

Exceptions

<i>std::runtime_error</i>	if particle type is unsupported
---------------------------	---------------------------------

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.9.3.15 writeHeaderData()

```
void ParticleZoo::IAEAphspFile::Writer::writeHeaderData (
    ByteBuffer & buffer ) [override], [protected], [virtual]
```

Write header data to the output buffer (not used for IAEA)

Parameters

<i>buffer</i>	Binary buffer for header information (not used for IAEA)
---------------	--

Implements [ParticleZoo::PhaseSpaceFileWriter](#).

The documentation for this class was generated from the following files:

- include/particlezoo/IAEA/IAEAphspFile.h
- src/IAEA/IAEAphspFile.cc

7.10 ParticleZoo::Particle Class Reference

Represents a particle in phase space.

```
#include <particlezoo/Particle.h>
```

Public Member Functions

- [Particle](#) ()=default
Default constructor for [Particle](#).
- [Particle](#) ([ParticleType](#) type, double kineticEnergy, double x, double y, double z, double directionalCosineX, double directionalCosineY, double directionalCosineZ, bool [isNewHistory](#)=true, double weight=1.0)
Construct a [Particle](#) with specified properties.
- [Particle](#) ([ParticleType](#) type, float kineticEnergy, float x, float y, float z, float directionalCosineX, float directionalCosineY, float directionalCosineZ, bool [isNewHistory](#)=true, float weight=1.0f)
Construct a [Particle](#) with specified properties.
- bool [getBoolProperty](#) ([BoolPropertyType](#) type) const
Get the value of a boolean property.
- const std::vector< bool > & [getCustomBoolProperties](#) () const
Get a reference to all custom boolean properties.
- const std::vector< float > & [getCustomFloatProperties](#) () const
Get a reference to all custom float properties.
- const std::vector< std::int32_t > & [getCustomIntProperties](#) () const
Get a reference to all custom integer properties.
- const std::vector< std::string > & [getCustomStringProperties](#) () const
Get a reference to all custom string properties.
- float [getDirectionalCosineX](#) () const
Get the X component of the directional cosine (momentum unit vector).
- float [getDirectionalCosineY](#) () const
Get the Y component of the directional cosine (momentum unit vector).
- float [getDirectionalCosineZ](#) () const
Get the Z component of the directional cosine (momentum unit vector).
- float [getFloatProperty](#) ([FloatPropertyType](#) type) const
Get the value of a float property.
- std::uint32_t [getIncrementalHistories](#) () const
Convenience function to get the number of incremental histories regardless of whether the property is set. If the property is not set, it returns 1 if the particle is marked as a new history, otherwise 0.
- std::int32_t [getIntProperty](#) ([IntPropertyType](#) type) const
Get the value of an integer property.
- float [getKineticEnergy](#) () const
Get the kinetic energy of the particle.

7.10 ParticleZoo::Particle Class Reference

- int [getNumberOfBoolProperties](#) () const
Get the number of boolean properties currently stored.
- int [getNumberOfFloatProperties](#) () const
Get the number of float properties currently stored.
- int [getNumberOfIntProperties](#) () const
Get the number of integer properties currently stored.
- std::int32_t [getPDGCode](#) () const
Get the PDG identification code of the particle.
- [ParticleType](#) [getType](#) () const
Get the particle type.
- float [getWeight](#) () const
Get the statistical weight of the particle.
- float [getX](#) () const
Get the X coordinate position of the particle.
- float [getY](#) () const
Get the Y coordinate position of the particle.
- float [getZ](#) () const
Get the Z coordinate position of the particle.
- bool [hasBoolProperty](#) ([BoolPropertyType](#) type) const
Check if a boolean property of the specified type exists.
- bool [hasFloatProperty](#) ([FloatPropertyType](#) type) const
Check if a float property of the specified type exists.
- bool [hasIntProperty](#) ([IntPropertyType](#) type) const
Check if an integer property of the specified type exists.
- bool [isNewHistory](#) () const
Check if this particle starts a new Monte Carlo history.
- bool [isPrimary](#) () const
Check if this particle is a primary particle.
- bool [projectToXValue](#) (float X)
Project the particle's trajectory to a specific X coordinate.
- bool [projectToYValue](#) (float Y)
Project the particle's trajectory to a specific Y coordinate.
- bool [projectToZValue](#) (float Z)
Project the particle's trajectory to a specific Z coordinate.
- void [reserveBoolProperties](#) (unsigned int size)
Reserve memory for boolean properties.
- void [reserveFloatProperties](#) (unsigned int size)
Reserve memory for float properties.
- void [reserveIntProperties](#) (unsigned int size)
Reserve memory for integer properties.
- void [setBoolProperty](#) ([BoolPropertyType](#) type, bool value)

- Set the value of a boolean property.*
- void [setDirectionalCosineX](#) (float px)
 - Set the X component of the directional cosine (momentum unit vector).*
- void [setDirectionalCosineY](#) (float py)
 - Set the Y component of the directional cosine (momentum unit vector).*
- void [setDirectionalCosineZ](#) (float pz)
 - Set the Z component of the directional cosine (momentum unit vector).*
- void [setFloatProperty](#) ([FloatPropertyType](#) type, float value)
 - Set the value of a float property.*
- void [setGeneration](#) (std::int32_t generation)
 - Set the generation of the particle.*
- void [setIncrementalHistories](#) (std::uint32_t incrementalHistories)
 - Convenience function to set the number of incremental histories using the INCREMENTAL_HISTORY_NUMBER integer property.*
- void [setIntProperty](#) ([IntPropertyType](#) type, std::int32_t value)
 - Set the value of an integer property.*
- void [setKineticEnergy](#) (float energy)
 - Set the kinetic energy of the particle.*
- void [setNewHistory](#) (bool isNewHistory)
 - Set whether this particle starts a new Monte Carlo history.*
- void [setStringProperty](#) (std::string value)
 - Add a custom string property.*
- void [setWeight](#) (float weight)
 - Set the statistical weight of the particle.*
- void [setX](#) (float x)
 - Set the X coordinate position of the particle.*
- void [setY](#) (float y)
 - Set the Y coordinate position of the particle.*
- void [setZ](#) (float z)
 - Set the Z coordinate position of the particle.*

7.10.1 Detailed Description

Represents a particle in phase space.

The [Particle](#) class encapsulates all the information about a single particle including its position, momentum direction, kinetic energy, statistical weight, and additional properties specific to different simulation codes. It provides methods for manipulating particle properties, projecting particle trajectories, and storing format-specific metadata.

7.10.2 Constructor & Destructor Documentation

7.10.2.1 Particle() [1/3]

```
ParticleZoo::Particle::Particle ( ) [default]
```

Default constructor for [Particle](#).

Creates a particle with default values (unsupported type, zero energy, etc.).

7.10.2.2 Particle() [2/3]

```
ParticleZoo::Particle::Particle (
    ParticleType type,
    float kineticEnergy,
    float x,
    float y,
    float z,
    float directionalCosineX,
    float directionalCosineY,
    float directionalCosineZ,
    bool isNewHistory = true,
    float weight = 1.0f ) [inline]
```

Construct a [Particle](#) with specified properties.

Creates a particle with the given position, momentum direction, energy, and other properties. The directional cosines are automatically normalized to ensure they represent a unit vector.

Parameters

<i>type</i>	The particle type (electron, photon, proton, etc.)
<i>kineticEnergy</i>	The kinetic energy of the particle
<i>x</i>	The X coordinate position
<i>y</i>	The Y coordinate position
<i>z</i>	The Z coordinate position
<i>directionalCosineX</i>	The X component of the momentum unit vector
<i>directionalCosineY</i>	The Y component of the momentum unit vector
<i>directionalCosineZ</i>	The Z component of the momentum unit vector
<i>isNewHistory</i>	Whether this particle starts a new Monte Carlo history (default: true)
<i>weight</i>	The statistical weight of the particle (default: 1.0)

7.10.2.3 Particle() [3/3]

```
ParticleZoo::Particle::Particle (
    ParticleType type,
    double kineticEnergy,
    double x,
    double y,
    double z,
    double directionalCosineX,
    double directionalCosineY,
    double directionalCosineZ,
    bool isNewHistory = true,
    double weight = 1.0 ) [inline]
```

Construct a [Particle](#) with specified properties.

Creates a particle with the given position, momentum direction, energy, and other properties. The directional cosines are automatically normalized to ensure they represent a unit vector.

This overload accepts double precision inputs and converts them to float internally.

Parameters

<i>type</i>	The particle type (electron, photon, proton, etc.)
<i>kineticEnergy</i>	The kinetic energy of the particle
<i>x</i>	The X coordinate position
<i>y</i>	The Y coordinate position
<i>z</i>	The Z coordinate position
<i>directionalCosineX</i>	The X component of the momentum unit vector
<i>directionalCosineY</i>	The Y component of the momentum unit vector
<i>directionalCosineZ</i>	The Z component of the momentum unit vector
<i>isNewHistory</i>	Whether this particle starts a new Monte Carlo history (default: true)
<i>weight</i>	The statistical weight of the particle (default: 1.0)

7.10.3 Member Function Documentation

7.10.3.1 getBoolProperty()

```
bool ParticleZoo::Particle::getBoolProperty (
    BoolPropertyType type ) const [inline]
```

Get the value of a boolean property.

7.10 ParticleZoo::Particle Class Reference

Parameters

<i>type</i>	The boolean property type to retrieve
-------------	---------------------------------------

Returns

bool The value of the boolean property

Exceptions

<i>std::invalid_argument</i>	if the property type is invalid or not found
------------------------------	--

7.10.3.2 getCustomBoolProperties()

```
const std::vector< bool > & ParticleZoo::Particle::getCustomBoolProperties ( ) const [inline]
```

Get a reference to all custom boolean properties.

Returns

const std::vector<bool>& Reference to the vector of custom boolean properties

7.10.3.3 getCustomFloatProperties()

```
const std::vector< float > & ParticleZoo::Particle::getCustomFloatProperties ( ) const [inline]
```

Get a reference to all custom float properties.

Returns

const std::vector<float>& Reference to the vector of custom float properties

7.10.3.4 getCustomIntProperties()

```
const std::vector< std::int32_t > & ParticleZoo::Particle::getCustomIntProperties ( ) const [inline]
```

Get a reference to all custom integer properties.

Returns

const std::vector<std::int32_t>& Reference to the vector of custom integer properties

7.10.3.5 `getCustomStringProperties()`

```
const std::vector< std::string > & ParticleZoo::Particle::getCustomStringProperties ( ) const  
[inline]
```

Get a reference to all custom string properties.

Returns

const std::vector<std::string>& Reference to the vector of custom string properties

7.10.3.6 `getDirectionalCosineX()`

```
float ParticleZoo::Particle::getDirectionalCosineX ( ) const [inline]
```

Get the X component of the directional cosine (momentum unit vector).

Returns

float The X component of the directional cosine

7.10.3.7 `getDirectionalCosineY()`

```
float ParticleZoo::Particle::getDirectionalCosineY ( ) const [inline]
```

Get the Y component of the directional cosine (momentum unit vector).

Returns

float The Y component of the directional cosine

7.10.3.8 `getDirectionalCosineZ()`

```
float ParticleZoo::Particle::getDirectionalCosineZ ( ) const [inline]
```

Get the Z component of the directional cosine (momentum unit vector).

Returns

float The Z component of the directional cosine

7.10.3.9 `getFloatProperty()`

```
float ParticleZoo::Particle::getFloatProperty (   
    FloatPropertyType type ) const [inline]
```

Get the value of a float property.

7.10 ParticleZoo::Particle Class Reference

Parameters

<i>type</i>	The float property type to retrieve
-------------	-------------------------------------

Returns

float The value of the float property

Exceptions

<i>std::invalid_argument</i>	if the property type is invalid or not found
------------------------------	--

7.10.3.10 getIncrementalHistories()

```
std::uint32_t ParticleZoo::Particle::getIncrementalHistories ( ) const [inline]
```

Convenience function to get the number of incremental histories regardless of whether the property is set. If the property is not set, it returns 1 if the particle is marked as a new history, otherwise 0.

Returns

std::uint32_t The number of incremental histories

7.10.3.11 getIntProperty()

```
std::int32_t ParticleZoo::Particle::getIntProperty (
    IntPropertyType type ) const [inline]
```

Get the value of an integer property.

Parameters

<i>type</i>	The integer property type to retrieve
-------------	---------------------------------------

Returns

std::int32_t The value of the integer property

Exceptions

<code>std::invalid_argument</code>	if the property type is invalid or not found
------------------------------------	--

7.10.3.12 getKineticEnergy()

```
float ParticleZoo::Particle::getKineticEnergy ( ) const [inline]
```

Get the kinetic energy of the particle.

Returns

float The kinetic energy value

7.10.3.13 getNumberOfBoolProperties()

```
int ParticleZoo::Particle::getNumberOfBoolProperties ( ) const [inline]
```

Get the number of boolean properties currently stored.

Returns

int The number of boolean properties

7.10.3.14 getNumberOfFloatProperties()

```
int ParticleZoo::Particle::getNumberOfFloatProperties ( ) const [inline]
```

Get the number of float properties currently stored.

Returns

int The number of float properties

7.10 ParticleZoo::Particle Class Reference

7.10.3.15 getNumberOfIntProperties()

```
int ParticleZoo::Particle::getNumberOfIntProperties ( ) const [inline]
```

Get the number of integer properties currently stored.

Returns

int The number of integer properties

7.10.3.16 getPDGCode()

```
std::int32_t ParticleZoo::Particle::getPDGCode ( ) const [inline]
```

Get the PDG identification code of the particle.

Returns

std::int32_t The PDG code corresponding to the particle type

7.10.3.17 getType()

```
ParticleType ParticleZoo::Particle::getType ( ) const [inline]
```

Get the particle type.

Returns

ParticleType The type of particle (electron, photon, proton, etc.)

7.10.3.18 getWeight()

```
float ParticleZoo::Particle::getWeight ( ) const [inline]
```

Get the statistical weight of the particle.

Returns

float The statistical weight value

7.10.3.19 getX()

```
float ParticleZoo::Particle::getX ( ) const [inline]
```

Get the X coordinate position of the particle.

Returns

float The X coordinate value

7.10.3.20 getY()

```
float ParticleZoo::Particle::getY ( ) const [inline]
```

Get the Y coordinate position of the particle.

Returns

float The Y coordinate value

7.10.3.21 getZ()

```
float ParticleZoo::Particle::getZ ( ) const [inline]
```

Get the Z coordinate position of the particle.

Returns

float The Z coordinate value

7.10.3.22 hasBoolProperty()

```
bool ParticleZoo::Particle::hasBoolProperty (
    BoolPropertyType type ) const [inline]
```

Check if a boolean property of the specified type exists.

7.10 ParticleZoo::Particle Class Reference

Parameters

<i>type</i>	The boolean property type to check for
-------------	--

Returns

true if the property exists
false if the property does not exist

7.10.3.23 hasFloatProperty()

```
bool ParticleZoo::Particle::hasFloatProperty (
    FloatPropertyType type ) const [inline]
```

Check if a float property of the specified type exists.

Parameters

<i>type</i>	The float property type to check for
-------------	--------------------------------------

Returns

true if the property exists
false if the property does not exist

7.10.3.24 hasIntProperty()

```
bool ParticleZoo::Particle::hasIntProperty (
    IntPropertyType type ) const [inline]
```

Check if an integer property of the specified type exists.

Parameters

<i>type</i>	The integer property type to check for
-------------	--

Returns

true if the property exists
false if the property does not exist

7.10.3.25 isNewHistory()

```
bool ParticleZoo::Particle::isNewHistory ( ) const [inline]
```

Check if this particle starts a new Monte Carlo history.

Returns

true if this particle starts a new history
false if this particle continues an existing history

7.10.3.26 isPrimary()

```
bool ParticleZoo::Particle::isPrimary ( ) const [inline]
```

Check if this particle is a primary particle.

A primary particle is defined as a particle that is not generated by any other particle, i.e., it is the first particle in a given history.

Note

Be aware that if a format does not provide explicit information about primary/secondary status, then **no** particles will report as primary from files of that format by default.

Returns

true if this particle is a primary particle
false if this particle is a secondary particle or later generation, or if the primary/secondary status is not available

7.10.3.27 projectToXValue()

```
bool ParticleZoo::Particle::projectToXValue (
    float X ) [inline]
```

Project the particle's trajectory to a specific X coordinate.

Calculates where the particle would be when it reaches the specified X value, assuming it travels in a straight line. Updates the Y and Z coordinates accordingly.

7.10 ParticleZoo::Particle Class Reference

Parameters

X	The target X coordinate to project to
---	---------------------------------------

Returns

true if projection was successful
false if projection is impossible (particle has no movement in X direction)

7.10.3.28 projectToYValue()

```
bool ParticleZoo::Particle::projectToYValue (
    float Y ) [inline]
```

Project the particle's trajectory to a specific Y coordinate.

Calculates where the particle would be when it reaches the specified Y value, assuming it travels in a straight line. Updates the X and Z coordinates accordingly.

Parameters

Y	The target Y coordinate to project to
---	---------------------------------------

Returns

true if projection was successful
false if projection is impossible (particle has no movement in Y direction)

7.10.3.29 projectToZValue()

```
bool ParticleZoo::Particle::projectToZValue (
    float Z ) [inline]
```

Project the particle's trajectory to a specific Z coordinate.

Calculates where the particle would be when it reaches the specified Z value, assuming it travels in a straight line. Updates the X and Y coordinates accordingly.

Parameters

<i>Z</i>	The target Z coordinate to project to
----------	---------------------------------------

Returns

true if projection was successful

false if projection is impossible (particle has no movement in Z direction)

7.10.3.30 reserveBoolProperties()

```
void ParticleZoo::Particle::reserveBoolProperties (
    unsigned int size ) [inline]
```

Reserve memory for boolean properties.

Parameters

<i>size</i>	The number of boolean properties to reserve space for
-------------	---

7.10.3.31 reserveFloatProperties()

```
void ParticleZoo::Particle::reserveFloatProperties (
    unsigned int size ) [inline]
```

Reserve memory for float properties.

Parameters

<i>size</i>	The number of float properties to reserve space for
-------------	---

7.10.3.32 reserveIntProperties()

```
void ParticleZoo::Particle::reserveIntProperties (
    unsigned int size ) [inline]
```

Reserve memory for integer properties.

7.10 ParticleZoo::Particle Class Reference

Parameters

<i>size</i>	The number of integer properties to reserve space for
-------------	---

7.10.3.33 setBoolProperty()

```
void ParticleZoo::Particle::setBoolProperty (
    BoolPropertyType type,
    bool value ) [inline]
```

Set the value of a boolean property.

If the property doesn't exist, it will be created. If it exists, the value will be updated.

Parameters

<i>type</i>	The boolean property type to set
<i>value</i>	The value to set for the property

7.10.3.34 setDirectionalCosineX()

```
void ParticleZoo::Particle::setDirectionalCosineX (
    float px ) [inline]
```

Set the X component of the directional cosine (momentum unit vector).

Parameters

<i>px</i>	The X component of the directional cosine to set
-----------	--

7.10.3.35 setDirectionalCosineY()

```
void ParticleZoo::Particle::setDirectionalCosineY (
    float py ) [inline]
```

Set the Y component of the directional cosine (momentum unit vector).

Parameters

<i>py</i>	The Y component of the directional cosine to set
-----------	--

7.10.3.36 setDirectionalCosineZ()

```
void ParticleZoo::Particle::setDirectionalCosineZ (
    float pz ) [inline]
```

Set the Z component of the directional cosine (momentum unit vector).

Parameters

<i>pz</i>	The Z component of the directional cosine to set
-----------	--

7.10.3.37 setFloatProperty()

```
void ParticleZoo::Particle::setFloatProperty (
    FloatPropertyType type,
    float value ) [inline]
```

Set the value of a float property.

If the property doesn't exist, it will be created. If it exists, the value will be updated.

Parameters

<i>type</i>	The float property type to set
<i>value</i>	The value to set for the property

7.10.3.38 setGeneration()

```
void ParticleZoo::Particle::setGeneration (
    std::int32_t generation ) [inline]
```

Set the generation of the particle.

7.10 ParticleZoo::Particle Class Reference

Parameters

<i>generation</i>	The generation number (1 for primary, 2+ for secondary)
-------------------	---

7.10.3.39 setIncrementalHistories()

```
void ParticleZoo::Particle::setIncrementalHistories (
    std::uint32_t incrementalHistories ) [inline]
```

Convenience function to set the number of incremental histories using the INCREMENTAL_HISTORY_NUMBER integer property.

Parameters

<i>incrementalHistories</i>	The number of incremental histories to set (must be greater than 0)
-----------------------------	---

7.10.3.40 setIntProperty()

```
void ParticleZoo::Particle::setIntProperty (
    IntPropertyType type,
    std::int32_t value ) [inline]
```

Set the value of an integer property.

If the property doesn't exist, it will be created. If it exists, the value will be updated.

Parameters

<i>type</i>	The integer property type to set
<i>value</i>	The value to set for the property

7.10.3.41 setKineticEnergy()

```
void ParticleZoo::Particle::setKineticEnergy (
    float energy ) [inline]
```

Set the kinetic energy of the particle.

Parameters

<i>energy</i>	The kinetic energy value to set
---------------	---------------------------------

7.10.3.42 setNewHistory()

```
void ParticleZoo::Particle::setNewHistory (
    bool isNewHistory ) [inline]
```

Set whether this particle starts a new Monte Carlo history.

Parameters

<i>isNewHistory</i>	True if this particle starts a new history, false otherwise
---------------------	---

7.10.3.43 setStringProperty()

```
void ParticleZoo::Particle::setStringProperty (
    std::string value ) [inline]
```

Add a custom string property.

Associate a string value with this particle. Multiple string properties can be added.

Parameters

<i>value</i>	The string value to add as a property
--------------	---------------------------------------

7.10.3.44 setWeight()

```
void ParticleZoo::Particle::setWeight (
    float weight ) [inline]
```

Set the statistical weight of the particle.

Parameters

<i>weight</i>	The statistical weight value to set
---------------	-------------------------------------

7.10 ParticleZoo::Particle Class Reference

7.10.3.45 setX()

```
void ParticleZoo::Particle::setX (
    float x ) [inline]
```

Set the X coordinate position of the particle.

Parameters

<i>x</i>	The X coordinate value to set
----------	-------------------------------

7.10.3.46 setY()

```
void ParticleZoo::Particle::setY (
    float y ) [inline]
```

Set the Y coordinate position of the particle.

Parameters

<i>y</i>	The Y coordinate value to set
----------	-------------------------------

7.10.3.47 setZ()

```
void ParticleZoo::Particle::setZ (
    float z ) [inline]
```

Set the Z coordinate position of the particle.

Parameters

<i>z</i>	The Z coordinate value to set
----------	-------------------------------

The documentation for this class was generated from the following file:

- include/particlezoo/Particle.h

7.11 ParticleZoo::ParticleBalancedParallelReader Class Reference

Multi-threaded phase space file reader for parallel particle processing.

```
#include <particlezoo/parallel/ParticleBalancedParallelReader.h>
```

Public Member Functions

- [ParticleBalancedParallelReader](#) (const std::string &filename, const UserOptions &options={}, size_t numThreads=1)
Constructs a multi-threaded phase space reader.
- [~ParticleBalancedParallelReader](#) ()
Destructor that closes all underlying readers.
- void [close](#) ()
Closes all underlying phase space file readers.
- std::uint64_t [getHistoriesRead](#) (size_t threadIndex) const
Gets the number of original histories processed by a specific thread.
- [Particle](#) [getNextParticle](#) (size_t threadIndex)
Retrieves the next particle for a specific thread.
- std::uint64_t [getNumberOfOriginalHistories](#) () const
Gets the number of original histories in the phase space file.
- std::uint64_t [getNumberOfParticles](#) () const
Gets the total number of particles in the phase space file.
- std::uint64_t [getNumberOfRepresentedHistories](#) ()
Gets the number of represented histories in the file.
- std::size_t [getNumberOfThreads](#) () const
Gets the number of threads used by this reader.
- std::uint64_t [getParticlesRead](#) (size_t threadIndex) const
Gets the total number of particles processed by a specific thread.
- std::uint64_t [getTotalHistoriesRead](#) () const
Gets the total number of original histories read across all threads.
- std::uint64_t [getTotalParticlesRead](#) () const
Gets the total number of particles read across all threads.
- bool [hasMoreParticles](#) (size_t threadIndex)
Checks if more particles are available for a specific thread.
- bool [hasNativeIncrementalHistoryCounters](#) () const
Checks if the underlying phase space format provides native incremental history counters.
- bool [hasNativeRepresentedHistoryCount](#) () const
Checks if the underlying phase space format provides native represented history count.
- [Particle](#) [peekNextParticle](#) (size_t threadIndex)
Peeks at the next particle for a specific thread without consuming it.

7.11.1 Detailed Description

Multi-threaded phase space file reader for parallel particle processing.

[ParticleBalancedParallelReader](#) provides thread-safe parallel reading of phase space files by creating multiple independent [PhaseSpaceFileReader](#) instances and partitioning the particles across threads. Each thread maintains its own reader positioned at a specific section of the phase space file.

The important property of this class is that it distributes particles evenly across threads, rather than distributing histories evenly. This is for use-cases where individual particles are the fundamental unit of work and optimal load balancing by particle count is desired. This approach ensures that each thread processes approximately the same number of particles, regardless of the distribution of particles across histories. Borders between histories are still respected, but some threads may read a different number of histories than others.

The reader automatically handles:

- [Particle](#) partitioning across threads with proper history boundary alignment
- Thread-safe particle retrieval without locking (each thread has its own reader)
- Incremental history number tracking across partition boundaries

Note

Each thread must use its assigned thread index when calling methods.

7.11.2 Constructor & Destructor Documentation

7.11.2.1 ParticleBalancedParallelReader()

```
ParticleZoo::ParticleBalancedParallelReader::ParticleBalancedParallelReader (
    const std::string & filename,
    const UserOptions & options = {},
    size_t numThreads = 1 )
```

Constructs a multi-threaded phase space reader.

Creates multiple [PhaseSpaceFileReader](#) instances (one per thread) and positions each at its designated starting particle. The constructor partitions the total particles evenly across threads, with remainder particles distributed to the first N threads.

Parameters

<i>filename</i>	Path to the phase space file to read
<i>options</i>	User options for configuring the reader (format-specific settings)
<i>numThreads</i>	Number of parallel threads that will read from this file

Exceptions

<code>std::runtime_error</code>	If the file cannot be opened or contains zero particles
<code>std::runtime_error</code>	If a PhaseSpaceFileReader cannot be created

7.11.3 Member Function Documentation

7.11.3.1 close()

```
void ParticleZoo::ParticleBalancedParallelReader::close ( )
```

Closes all underlying phase space file readers.

Should be called when done reading to free file handles and resources. Automatically called by the destructor.

7.11.3.2 getHistoriesRead()

```
std::uint64_t ParticleZoo::ParticleBalancedParallelReader::getHistoriesRead (
    size_t threadIndex ) const
```

Gets the number of original histories processed by a specific thread.

Returns the cumulative count of original histories (including empty ones) covered by particles read via [getNextParticle\(\)](#) for this thread.

Uses one of two computation methods depending on format capabilities:

- **RATIO**: The represented history count is available from the header. Maps the per-thread represented count to original histories via the known O/R ratio.
- **INCREMENTAL**: Uses the direct sum of per-particle incremental history values. For formats with native per-particle data this gives exact original history counts. For formats without native data the default of 1 per new-history particle is used, so the result equals the number of represented (non-empty) histories read by this thread.

Parameters

<code>threadIndex</code>	The index of the thread (0 to numThreads-1)
--------------------------	---

7.11 ParticleZoo::ParticleBalancedParallelReader Class Reference

Returns

Number of original histories covered by this thread so far

Exceptions

<code>std::out_of_range</code>	If threadIndex is invalid
--------------------------------	---------------------------

7.11.3.3 getNextParticle()

```
Particle ParticleZoo::ParticleBalancedParallelReader::getNextParticle (
    size_t threadIndex )
```

Retrieves the next particle for a specific thread.

Reads and returns the next particle from the phase space file for the given thread. Updates internal counters for history and particle tracking.

Parameters

<code>threadIndex</code>	The index of the calling thread (0 to numThreads-1)
--------------------------	---

Returns

The next particle for this thread

Exceptions

<code>std::out_of_range</code>	If threadIndex is invalid
<code>std::runtime_error</code>	If no more particles are available for this thread

Note

Must call [hasMoreParticles\(\)](#) first to verify particles are available

Sets the INCREMENTAL_HISTORY_NUMBER property on particles

7.11.3.4 getNumberOfOriginalHistories()

```
std::uint64_t ParticleZoo::ParticleBalancedParallelReader::getNumberOfOriginalHistories ( ) const
[inline]
```

Gets the number of original histories in the phase space file.

Returns the total number of histories that are in the phase space file, including those that have no particles (empty histories).

Returns

Number of original histories (including empty ones)

7.11.3.5 `getNumberOfParticles()`

```
std::uint64_t ParticleZoo::ParticleBalancedParallelReader::getNumberOfParticles ( ) const [inline]
```

Gets the total number of particles in the phase space file.

Returns

Total particle count in the file

7.11.3.6 `getNumberOfRepresentedHistories()`

```
std::uint64_t ParticleZoo::ParticleBalancedParallelReader::getNumberOfRepresentedHistories ( )
```

Gets the number of represented histories in the file.

Returns the count of histories that actually produced particles and are represented in the phase space file. This is always less than or equal to the number of original histories.

Note

If the phase space format does not support direct retrieval of this value, it will be computed on the first call by scanning the file which may be time-consuming.

Returns

Number of histories with at least one particle

7.11 ParticleZoo::ParticleBalancedParallelReader Class Reference

7.11.3.7 getNumberOfThreads()

```
std::size_t ParticleZoo::ParticleBalancedParallelReader::getNumberOfThreads ( ) const [inline]
```

Gets the number of threads used by this reader.

Returns

Number of parallel threads

7.11.3.8 getParticlesRead()

```
std::uint64_t ParticleZoo::ParticleBalancedParallelReader::getParticlesRead (
    size_t threadIndex ) const
```

Gets the total number of particles processed by a specific thread.

Returns the cumulative count of particles that have been processed by this thread. This represents the contribution to the total particle count.

Parameters

<i>threadIndex</i>	The index of the thread (0 to numThreads-1)
--------------------	---

Returns

Total number of particles processed

Exceptions

<i>std::out_of_range</i>	If threadIndex is invalid
--------------------------	---------------------------

7.11.3.9 getTotalHistoriesRead()

```
std::uint64_t ParticleZoo::ParticleBalancedParallelReader::getTotalHistoriesRead ( ) const
```

Gets the total number of original histories read across all threads.

Returns

Total number of original histories read

7.11.3.10 getTotalParticlesRead()

```
std::uint64_t ParticleZoo::ParticleBalancedParallelReader::getTotalParticlesRead ( ) const
```

Gets the total number of particles read across all threads.

Returns

Total number of particles read

7.11.3.11 hasMoreParticles()

```
bool ParticleZoo::ParticleBalancedParallelReader::hasMoreParticles (
    size_t threadIndex )
```

Checks if more particles are available for a specific thread.

Determines whether the thread has reached its partition boundary or the end of the file.

Parameters

<i>threadIndex</i>	The index of the calling thread (0 to numThreads-1)
--------------------	---

Returns

true if more particles are available, false otherwise

Exceptions

<i>std::out_of_range</i>	If threadIndex is invalid
--------------------------	---------------------------

Note

Always call this before [getNextParticle\(\)](#) to avoid exceptions

7.11 ParticleZoo::ParticleBalancedParallelReader Class Reference

7.11.3.12 hasNativeIncrementalHistoryCounters()

```
bool ParticleZoo::ParticleBalancedParallelReader::hasNativeIncrementalHistoryCounters ( ) const
```

Checks if the underlying phase space format provides native incremental history counters.

Returns

True if native incremental history counters are available, false otherwise

7.11.3.13 hasNativeRepresentedHistoryCount()

```
bool ParticleZoo::ParticleBalancedParallelReader::hasNativeRepresentedHistoryCount ( ) const
```

Checks if the underlying phase space format provides native represented history count.

Returns

True if native represented history count is available, false otherwise

7.11.3.14 peekNextParticle()

```
Particle ParticleZoo::ParticleBalancedParallelReader::peekNextParticle (
    size_t threadIndex )
```

Peeks at the next particle for a specific thread without consuming it.

Allows inspection of the next particle that would be returned by [getNextParticle\(\)](#) without advancing the reader's position. Useful for checking history boundaries.

Parameters

<i>threadIndex</i>	The index of the calling thread (0 to numThreads-1)
--------------------	---

Returns

The next particle for this thread

Exceptions

<code>std::out_of_range</code>	If threadIndex is invalid
<code>std::runtime_error</code>	If no more particles are available for this thread

Note

Must call [hasMoreParticles\(\)](#) first to verify particles are available

The documentation for this class was generated from the following files:

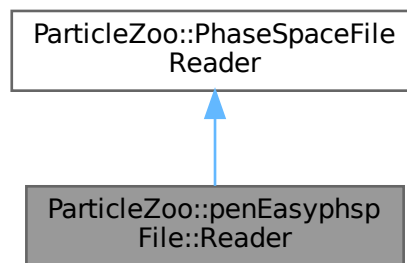
- include/particlezoo/parallel/ParticleBalancedParallelReader.h
- src/parallel/ParticleBalancedParallelReader.cc

7.12 ParticleZoo::penEasyphspFile::Reader Class Reference

[Reader](#) for penEasy format phase space files.

```
#include <particlezoo/peneasy/penEasyphspFile.h>
```

Inheritance diagram for ParticleZoo::penEasyphspFile::Reader:



Public Member Functions

- [Reader](#) (const std::string &fileName, const UserOptions &options=UserOptions{})
Construct reader for penEasy phase space file.
- std::uint64_t [getNumberOfOriginalHistories](#) () const override
Get the number of original simulation histories.
- std::uint64_t [getNumberOfParticles](#) () const override
Get the total number of particles in the phase space.
- bool [hasNativeIncrementalHistoryCounters](#) () const override
Check if this format stores incremental history numbers per-particle.

Public Member Functions inherited from [ParticleZoo::PhaseSpaceFileReader](#)

- [PhaseSpaceFileReader](#) (const std::string &phspFormat, const std::string &fileName, const UserOptions &userOptions, [FormatType](#) formatType=[FormatType::BINARY](#), const [FixedValues](#) fixedValues=[FixedValues\(\)](#), unsigned int bufferSize=[DEFAULT_BUFFER_SIZE](#))
Construct a new Phase Space File Reader object.
- virtual [~PhaseSpaceFileReader](#) ()

- Destroy the Phase Space File Reader object.*

 - void `close ()`
 - Close the phase space file and clean up resources.*
 - float `getConstantPx ()` const
 - Get the constant X-component of the direction unit vector (if constant).*
 - float `getConstantPy ()` const
 - Get the constant Y-component of the direction unit vector (if constant).*
 - float `getConstantPz ()` const
 - Get the constant Z-component of the direction unit vector (if constant).*
 - float `getConstantWeight ()` const
 - Get the constant statistical weight value (if constant).*
 - float `getConstantX ()` const
 - Get the constant X coordinate value (if constant).*
 - float `getConstantY ()` const
 - Get the constant Y coordinate value (if constant).*
 - float `getConstantZ ()` const
 - Get the constant Z coordinate value (if constant).*
 - const std::string `getFileName ()` const
 - Get the filename of the phase space file being read.*
 - std::uint64_t `getFileSize ()` const
 - Get the size of the phase space file in bytes.*
 - const FixedValues `getFixedValues ()` const
 - Get the fixed values configuration.*
 - virtual std::uint64_t `getHistoriesRead ()`
 - Get the number of Monte Carlo histories that have been read so far. If the end of the file has been reached, this will return the total number of original histories unless more histories than expected have already been read - in which case it returns the actual count.*
 - Particle `getNextParticle ()`
 - Get the next particle from the phase space file.*
 - virtual std::uint64_t `getNumberOfRepresentedHistories ()` const
 - Get the number of histories actually represented in this phase space.*
 - virtual std::uint64_t `getParticlesRead ()`
 - Get the number of particles that have been read so far.*
 - const std::string `getPHSPFormat ()` const
 - Get the phase space file format identifier.*
 - virtual bool `hasMoreParticles ()`
 - Check if there are more particles to read in the file.*
 - virtual bool `hasNativeRepresentedHistoryCount ()` const
 - Check if this format can provide the number of represented histories without scanning the file.*
 - bool `isPxConstant ()` const
 - Check if the X-component of momentum is constant for all particles.*
 - bool `isPyConstant ()` const

7.12 ParticleZoo::penEasyhspFile::Reader Class Reference

- Check if the Y-component of momentum is constant for all particles.*
- bool [isPzConstant](#) () const
- Check if the Z-component of momentum is constant for all particles.*
- bool [isWeightConstant](#) () const
- Check if the statistical weight is constant for all particles.*
- bool [isXConstant](#) () const
- Check if the X coordinate is constant for all particles.*
- bool [isYConstant](#) () const
- Check if the Y coordinate is constant for all particles.*
- bool [isZConstant](#) () const
- Check if the Z coordinate is constant for all particles.*
- void [moveToParticle](#) (std::uint64_t particleIndex)
- Move the file position to a specific particle index.*
- [Particle peekNextParticle](#) ()
- Peek at the next particle without advancing the file position.*
- void [setCommentMarkers](#) (const std::vector< std::string > &commentMarkers)
- Set comment markers for ASCII format files.*

Protected Member Functions

- size_t [getMaximumASCIILineLength](#) () const override
- Get the maximum length of ASCII particle lines, required for buffer sizing.*
- [Particle readASCIIParticle](#) (const std::string &line) override
- Parse a single ASCII line into a [Particle](#) object.*

Protected Member Functions inherited from [ParticleZoo::PhaseSpaceFileReader](#)

- double [calcThirdUnitComponent](#) (double &u, double &v) const
- Calculate the third component of a unit vector from two components (double precision).*
- float [calcThirdUnitComponent](#) (float &u, float &v) const
- Calculate the third component of a unit vector from two components (float precision).*
- const [ByteBuffer](#) [getHeaderData](#) ()
- Get the file header data as a byte buffer.*
- const [ByteBuffer](#) [getHeaderData](#) (std::size_t headerSize)
- Get a specific amount of header data as a byte buffer.*
- [Particle](#) [getNextParticle](#) (bool countParticleInStatistics)
- Get the next particle with optional statistics counting control.*
- std::size_t [getNumberOfEntriesInFile](#) () const
- Get the number of particle records that fit in the file.*
- virtual std::size_t [getParticleRecordLength](#) () const

- Get the length in bytes of each particle record.*
- virtual std::size_t [getParticleRecordStartOffset](#) () const
Get the byte offset where particle records start in the file.
- virtual std::uint64_t [getParticlesRead](#) (bool includeSkippedParticles)
Get the number of particles read with optional inclusion of skipped particles (including pseudo-particles).
- const UserOptions & [getUserOptions](#) () const
Get the user options that were passed to the constructor.
- virtual [Particle](#) [peekParticleManually](#) ()
Peek at a particle manually (for formats requiring third-party I/O).
- virtual [Particle](#) [readBinaryParticle](#) (ByteBuffer &buffer)
Read a particle from binary data.
- virtual [Particle](#) [readParticleManually](#) ()
Read a particle manually (for formats requiring third-party I/O).
- void [setByteOrder](#) (ByteOrder byteOrder)
Set the byte order for binary data interpretation.
- void [setConstantPx](#) (float Px)
Set a constant X-component of the direction unit vector for all particles.
- void [setConstantPy](#) (float Py)
Set a constant Y-component of the direction unit vector for all particles.
- void [setConstantPz](#) (float Pz)
Set a constant Z-component of the direction unit vector for all particles.
- void [setConstantWeight](#) (float weight)
Set a constant statistical weight for all particles.
- void [setConstantX](#) (float X)
Set a constant X coordinate value for all particles.
- void [setConstantY](#) (float Y)
Set a constant Y coordinate value for all particles.
- void [setConstantZ](#) (float Z)
Set a constant Z coordinate value for all particles.

Additional Inherited Members

Static Public Member Functions inherited from [ParticleZoo::PhaseSpaceFileReader](#)

- static std::vector< CLICommand > [getCLICommands](#) ()
Get command line interface commands supported by this reader.

7.12.1 Detailed Description

[Reader](#) for penEasy format phase space files.

Provides functionality to read phase space data from penEasy ASCII format files. Automatically counts total particles and total histories during construction by scanning the entire file (this may be slow for very large files).

7.12.2 Constructor & Destructor Documentation

7.12.2.1 Reader()

```
ParticleZoo::penEasyphspFile::Reader::Reader (
    const std::string & fileName,
    const UserOptions & options = UserOptions{} )
```

Construct reader for penEasy phase space file.

Scans the file during construction to count particles and sum delta-N values for determining the total number of original histories.

Parameters

<i>fileName</i>	Path to the penEasy phase space file to read
<i>options</i>	User-specified options for reading behavior

Exceptions

<i>std::runtime_error</i>	if file cannot be opened or parsed
---------------------------	------------------------------------

7.12.3 Member Function Documentation

7.12.3.1 getMaximumASCIILineLength()

```
size_t ParticleZoo::penEasyphspFile::Reader::getMaximumASCIILineLength ( ) const [inline], [override],
[protected], [virtual]
```

Get the maximum length of ASCII particle lines, required for buffer sizing.

Returns

Maximum line length

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

7.12.3.2 `getNumberOfOriginalHistories()`

```
std::uint64_t ParticleZoo::penEasyphspFile::Reader::getNumberOfOriginalHistories ( ) const [inline],  
[override], [virtual]
```

Get the number of original simulation histories.

Returns

Sum of all delta-N values from particle records

Implements [ParticleZoo::PhaseSpaceFileReader](#).

7.12.3.3 `getNumberOfParticles()`

```
std::uint64_t ParticleZoo::penEasyphspFile::Reader::getNumberOfParticles ( ) const [inline],  
[override], [virtual]
```

Get the total number of particles in the phase space.

Returns

Total particle count determined by file scanning

Implements [ParticleZoo::PhaseSpaceFileReader](#).

7.12.3.4 `hasNativeIncrementalHistoryCounters()`

```
bool ParticleZoo::penEasyphspFile::Reader::hasNativeIncrementalHistoryCounters ( ) const [inline],  
[override], [virtual]
```

Check if this format stores incremental history numbers per-particle.

Returns

true — penEasy always stores delta-N per particle

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

7.12 ParticleZoo::penEasyphspFile::Reader Class Reference

7.12.3.5 readASCIIParticle()

```
Particle ParticleZoo::penEasyphspFile::Reader::readASCIIParticle (
    const std::string & line ) [override], [protected], [virtual]
```

Parse a single ASCII line into a [Particle](#) object.

Parses penEasy format: KPAR E X Y Z U V W WGHT DeltaN ILB(1..5)

- KPAR: particle type code (1=electron, 2=photon, 3=positron, 4=proton)
- E: kinetic energy in eV
- X,Y,Z: position coordinates
- U,V,W: direction cosines
- WGHT: particle weight
- DeltaN: incremental history number
- ILB(1..5): PENELOPE ILB array values

Parameters

<i>line</i>	ASCII line containing particle data
-------------	-------------------------------------

Returns

Parsed [Particle](#) object with all properties set

Exceptions

<i>std::runtime_error</i>	if line cannot be parsed or contains invalid data
---------------------------	---

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

The documentation for this class was generated from the following files:

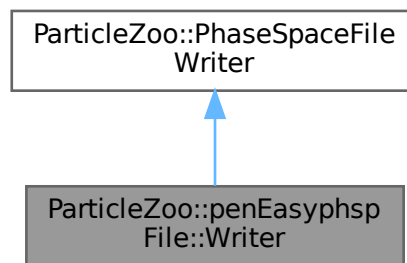
- include/particlezoo/peneasy/penEasyphspFile.h
- src/peneasy/penEasyphspFile.cc

7.13 ParticleZoo::penEasyphspFile::Writer Class Reference

[Writer](#) for penEasy format phase space files.

```
#include <particlezoo/peneasy/penEasyphspFile.h>
```

Inheritance diagram for ParticleZoo::penEasyphspFile::Writer:



Public Member Functions

- [Writer](#) (const std::string &fileName, const UserOptions &options=UserOptions{})
Construct writer for penEasy phase space file.
- std::uint64_t [getMaximumSupportedParticles](#) () const override
Get the maximum number of particles this format can store.

Public Member Functions inherited from [ParticleZoo::PhaseSpaceFileWriter](#)

- [PhaseSpaceFileWriter](#) (const std::string &phspFormat, const std::string &fileName, const UserOptions &userOptions, [FormatType](#) formatType=[FormatType::BINARY](#), const [FixedValues](#) fixedValues=[FixedValues\(\)](#), unsigned int bufferSize=[DEFAULT_BUFFER_SIZE](#))
Construct a new Phase Space File Writer object.
- virtual [~PhaseSpaceFileWriter](#) ()
Destroy the Phase Space File Writer object.
- void [addAdditionalHistories](#) (std::uint64_t additionalHistories)
Add additional Monte Carlo histories to the count.
- void [close](#) ()

7.13 ParticleZoo::penEasyhspFile::Writer Class Reference

- Close the phase space file and finalize writing.*
- `ByteOrder getByteOrder () const`
Get the byte order used for binary data writing.
- `float getConstantPx () const`
Get the constant X-component of the direction unit vector (if constant).
- `float getConstantPy () const`
Get the constant Y-component of the direction unit vector (if constant).
- `float getConstantPz () const`
Get the constant Z-component of the direction unit vector (if constant).
- `float getConstantWeight () const`
Get the constant statistical weight value (if constant).
- `float getConstantX () const`
Get the constant X coordinate value (if constant).
- `float getConstantY () const`
Get the constant Y coordinate value (if constant).
- `float getConstantZ () const`
Get the constant Z coordinate value (if constant).
- `const std::string getFileName () const`
Get the filename where the phase space file is being written.
- `const FixedValues getFixedValues () const`
Get the fixed values configuration.
- `virtual std::uint64_t getHistoriesWritten () const`
Get the number of Monte Carlo histories that have been written.
- `std::uint64_t getParticlesWritten () const`
Get the number of particles that have been written to the file.
- `const std::string getPHSPFormat () const`
Get the phase space file format identifier.
- `bool isPxConstant () const`
Check if the X-component of the direction unit vector is set to a constant value for all particles.
- `bool isPyConstant () const`
Check if the Y-component of the direction unit vector is set to a constant value for all particles.
- `bool isPzConstant () const`
Check if the Z-component of the direction unit vector is set to a constant value for all particles.
- `bool isWeightConstant () const`
Check if the statistical weight is set to a constant value for all particles.
- `bool isXConstant () const`
Check if the X coordinate is set to a constant value for all particles.
- `bool isYConstant () const`
Check if the Y coordinate is set to a constant value for all particles.
- `bool isZConstant () const`
Check if the Z coordinate is set to a constant value for all particles.
- `virtual void writeParticle (Particle particle)`
Write a particle to the phase space file.

Protected Member Functions

- `size_t` [getMaximumASCIILineLength](#) () const override
Get the maximum length of ASCII particle lines, required for buffer sizing.
- `std::size_t` [getParticleRecordStartOffset](#) () const override
Get the byte offset where particle records start.
- `const std::string` [writeASCIIParticle](#) ([Particle](#) &particle) override
Convert a particle to ASCII representation.
- `void` [writeHeaderData](#) ([ByteBuffer](#) &buffer) override
Write the file header to the output buffer.

Protected Member Functions inherited from [ParticleZoo::PhaseSpaceFileWriter](#)

- `virtual bool` [accountForAdditionalHistories](#) (`std::uint64_t` additionalHistories)
Handle accounting for simulation histories that produced no particles.
- `virtual bool` [canHaveConstantPx](#) () const
Check if this format supports constant X-component of the direction unit vector.
- `virtual bool` [canHaveConstantPy](#) () const
Check if this format supports constant Y-component of the direction unit vector.
- `virtual bool` [canHaveConstantPz](#) () const
Check if this format supports constant Z-component of the direction unit vector.
- `virtual bool` [canHaveConstantWeight](#) () const
Check if this format supports constant statistical weights.
- `virtual bool` [canHaveConstantX](#) () const
Check if this format supports constant X coordinates.
- `virtual bool` [canHaveConstantY](#) () const
Check if this format supports constant Y coordinates.
- `virtual bool` [canHaveConstantZ](#) () const
Check if this format supports constant Z coordinates.
- `virtual bool` [canWritePseudoParticlesExplicitly](#) () const
Check if this format can write pseudo-particles explicitly.
- `virtual void` [fixedValuesHaveChanged](#) ()
Called when fixed values have been changed.
- `virtual std::size_t` [getParticleRecordLength](#) () const
Get the length in bytes of each particle record.
- `virtual std::uint64_t` [getPendingHistories](#) () const
Get the number of pending histories to account for.
- `const UserOptions &` [getUserOptions](#) () const
Get the user options that were passed to the constructor.
- `void` [setByteOrder](#) ([ByteOrder](#) byteOrder)
Set the byte order for binary data writing.

7.13 ParticleZoo::penEasyphspFile::Writer Class Reference

- void [setConstantPx](#) (float Px)
Set a constant X-component of the direction unit vector for all particles.
- void [setConstantPy](#) (float Py)
Set a constant Y-component of the direction unit vector for all particles.
- void [setConstantPz](#) (float Pz)
Set a constant Z-component of the direction unit vector for all particles.
- void [setConstantWeight](#) (float weight)
Set a constant statistical weight for all particles.
- void [setConstantX](#) (float X)
Set a constant X coordinate value for all particles.
- void [setConstantY](#) (float Y)
Set a constant Y coordinate value for all particles.
- void [setConstantZ](#) (float Z)
Set a constant Z coordinate value for all particles.
- virtual void [writeBinaryParticle](#) (ByteBuffer &buffer, Particle &particle)
Write a particle in binary format to a byte buffer.
- virtual void [writeParticleManually](#) (Particle &particle)
Write a particle manually (for formats requiring third-party I/O).

Additional Inherited Members

Static Public Member Functions inherited from [ParticleZoo::PhaseSpaceFileWriter](#)

- static std::vector< CLICCommand > [getCLICommands](#) ()
Get command line interface commands supported by this writer.

7.13.1 Detailed Description

[Writer](#) for penEasy format phase space files.

Provides functionality to write phase space data in the penEasy ASCII format, which is compatible with the PENELOPE Monte Carlo transport code. The format includes particle type, energy, position, direction, weight, and PENELOPE-specific values (ILB1-ILB5) and delta-N values.

7.13.2 Constructor & Destructor Documentation

7.13.2.1 Writer()

```
ParticleZoo::penEasyphspFile::Writer::Writer (
    const std::string & fileName,
    const UserOptions & options = UserOptions{} )
```

Construct writer for penEasy phase space file.

Parameters

<i>fileName</i>	Path to the output penEasy phase space file
<i>options</i>	User-specified options for writing behavior

7.13.3 Member Function Documentation

7.13.3.1 `getMaximumASCIILineLength()`

```
size_t ParticleZoo::penEasyphspFile::Writer::getMaximumASCIILineLength ( ) const [inline], [override],  
[protected], [virtual]
```

Get the maximum length of ASCII particle lines, required for buffer sizing.

Returns

Maximum line length

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.13.3.2 `getMaximumSupportedParticles()`

```
std::uint64_t ParticleZoo::penEasyphspFile::Writer::getMaximumSupportedParticles ( ) const [inline],  
[override], [virtual]
```

Get the maximum number of particles this format can store.

Returns

Maximum particle count

Implements [ParticleZoo::PhaseSpaceFileWriter](#).

7.13.3.3 `getParticleRecordStartOffset()`

```
std::size_t ParticleZoo::penEasyphspFile::Writer::getParticleRecordStartOffset ( ) const [inline],  
[override], [protected], [virtual]
```

Get the byte offset where particle records start.

Returns

Header length (112 bytes for penEasy format)

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.13 ParticleZoo::penEasyphspFile::Writer Class Reference

7.13.3.4 writeASCIIParticle()

```
const std::string ParticleZoo::penEasyphspFile::Writer::writeASCIIParticle (
    Particle & particle ) [override], [protected], [virtual]
```

Convert a particle to ASCII representation.

Formats a particle according to the penEasy specification: KPAR E X Y Z U V W WGHT DeltaN ILB(1..5)

Parameters

<i>particle</i>	Particle object to convert to ASCII
-----------------	---

Returns

ASCII string representation of the particle

Exceptions

<i>std::runtime_error</i>	if particle type is unsupported or data is too long
---------------------------	---

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.13.3.5 writeHeaderData()

```
void ParticleZoo::penEasyphspFile::Writer::writeHeaderData (
    ByteBuffer & buffer ) [override], [protected], [virtual]
```

Write the file header to the output buffer.

Parameters

<i>buffer</i>	Byte buffer to write header data to
---------------	-------------------------------------

Implements [ParticleZoo::PhaseSpaceFileWriter](#).

The documentation for this class was generated from the following files:

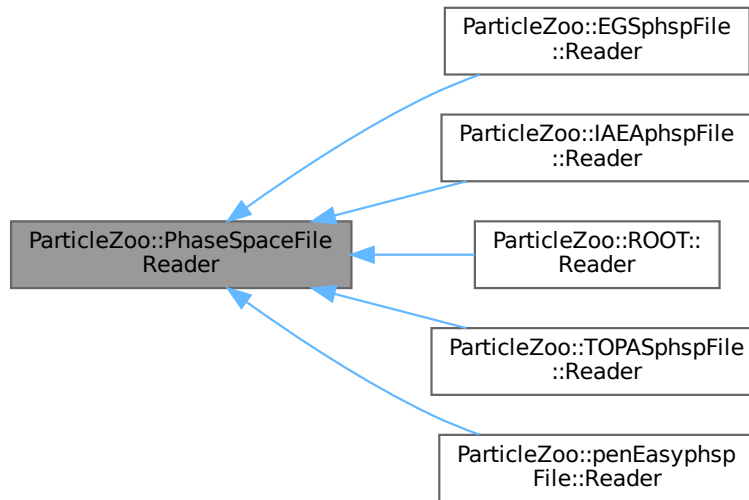
- include/particlezoo/peneasy/penEasyphspFile.h
- src/peneasy/penEasyphspFile.cc

7.14 ParticleZoo::PhaseSpaceFileReader Class Reference

Base class for reading phase space files.

```
#include <particlezoo/PhaseSpaceFileReader.h>
```

Inheritance diagram for ParticleZoo::PhaseSpaceFileReader:



Public Member Functions

- [PhaseSpaceFileReader](#) (const std::string &phspFormat, const std::string &fileName, const UserOptions &userOptions, [FormatType](#) formatType=[FormatType::BINARY](#), const [FixedValues](#) fixedValues=[FixedValues\(\)](#), unsigned int bufferSize=[DEFAULT_BUFFER_SIZE](#))
Construct a new Phase Space File Reader object.
- virtual [~PhaseSpaceFileReader](#) ()
Destroy the Phase Space File Reader object.
- void [close](#) ()
Close the phase space file and clean up resources.
- float [getConstantPx](#) () const
Get the constant X-component of the direction unit vector (if constant).
- float [getConstantPy](#) () const

7.14 ParticleZoo::PhaseSpaceFileReader Class Reference

- Get the constant Y-component of the direction unit vector (if constant).*
- float [getConstantPz](#) () const
- Get the constant Z-component of the direction unit vector (if constant).*
- float [getConstantWeight](#) () const
- Get the constant statistical weight value (if constant).*
- float [getConstantX](#) () const
- Get the constant X coordinate value (if constant).*
- float [getConstantY](#) () const
- Get the constant Y coordinate value (if constant).*
- float [getConstantZ](#) () const
- Get the constant Z coordinate value (if constant).*
- const std::string [getFileName](#) () const
- Get the filename of the phase space file being read.*
- std::uint64_t [getFileSize](#) () const
- Get the size of the phase space file in bytes.*
- const [FixedValues](#) [getFixedValues](#) () const
- Get the fixed values configuration.*
- virtual std::uint64_t [getHistoriesRead](#) ()
- Get the number of Monte Carlo histories that have been read so far. If the end of the file has been reached, this will return the total number of original histories unless more histories than expected have already been read - in which case it returns the actual count.*
- [Particle](#) [getNextParticle](#) ()
- Get the next particle from the phase space file.*
- virtual std::uint64_t [getNumberOfOriginalHistories](#) () const =0
- Get the number of original Monte Carlo histories that generated this phase space.*
- virtual std::uint64_t [getNumberOfParticles](#) () const =0
- Get the total number of particles in the phase space file.*
- virtual std::uint64_t [getNumberOfRepresentedHistories](#) () const
- Get the number of histories actually represented in this phase space.*
- virtual std::uint64_t [getParticlesRead](#) ()
- Get the number of particles that have been read so far.*
- const std::string [getPHSPFormat](#) () const
- Get the phase space file format identifier.*
- virtual bool [hasMoreParticles](#) ()
- Check if there are more particles to read in the file.*
- virtual bool [hasNativeIncrementalHistoryCounters](#) () const
- Check if this format directly stores incremental history numbers per-particle.*
- virtual bool [hasNativeRepresentedHistoryCount](#) () const
- Check if this format can provide the number of represented histories without scanning the file.*
- bool [isPxConstant](#) () const
- Check if the X-component of momentum is constant for all particles.*
- bool [isPyConstant](#) () const

- *Check if the Y-component of momentum is constant for all particles.*
• bool [isPzConstant](#) () const
- *Check if the Z-component of momentum is constant for all particles.*
• bool [isWeightConstant](#) () const
- *Check if the statistical weight is constant for all particles.*
• bool [isXConstant](#) () const
- *Check if the X coordinate is constant for all particles.*
• bool [isYConstant](#) () const
- *Check if the Y coordinate is constant for all particles.*
• bool [isZConstant](#) () const
- *Check if the Z coordinate is constant for all particles.*
• void [moveToParticle](#) (std::uint64_t particleIndex)
- *Move the file position to a specific particle index.*
• [Particle peekNextParticle](#) ()
- *Peek at the next particle without advancing the file position.*
• void [setCommentMarkers](#) (const std::vector< std::string > &commentMarkers)
- *Set comment markers for ASCII format files.*

Static Public Member Functions

- static std::vector< CLICommand > [getCLICommands](#) ()
Get command line interface commands supported by this reader.

Protected Member Functions

- double [calcThirdUnitComponent](#) (double &u, double &v) const
Calculate the third component of a unit vector from two components (double precision).
- float [calcThirdUnitComponent](#) (float &u, float &v) const
Calculate the third component of a unit vector from two components (float precision).
- const [ByteBuffer getHeaderData](#) ()
Get the file header data as a byte buffer.
- const [ByteBuffer getHeaderData](#) (std::size_t headerSize)
Get a specific amount of header data as a byte buffer.
- virtual std::size_t [getMaximumASCIILineLength](#) () const
Get the maximum line length for ASCII format files.
- [Particle getNextParticle](#) (bool countParticleInStatistics)
Get the next particle with optional statistics counting control.
- std::size_t [getNumberOfEntriesInFile](#) () const
Get the number of particle records that fit in the file.
- virtual std::size_t [getParticleRecordLength](#) () const

7.14 ParticleZoo::PhaseSpaceFileReader Class Reference

- Get the length in bytes of each particle record.*
- virtual std::size_t [getParticleRecordStartOffset](#) () const
Get the byte offset where particle records start in the file.
- virtual std::uint64_t [getParticlesRead](#) (bool includeSkippedParticles)
Get the number of particles read with optional inclusion of skipped particles (including pseudo-particles).
- const UserOptions & [getUserOptions](#) () const
Get the user options that were passed to the constructor.
- virtual [Particle](#) [peekParticleManually](#) ()
Peek at a particle manually (for formats requiring third-party I/O).
- virtual [Particle](#) [readASCIIParticle](#) (const std::string &line)
Read a particle from ASCII data.
- virtual [Particle](#) [readBinaryParticle](#) (ByteBuffer &buffer)
Read a particle from binary data.
- virtual [Particle](#) [readParticleManually](#) ()
Read a particle manually (for formats requiring third-party I/O).
- void [setByteOrder](#) (ByteOrder byteOrder)
Set the byte order for binary data interpretation.
- void [setConstantPx](#) (float Px)
Set a constant X-component of the direction unit vector for all particles.
- void [setConstantPy](#) (float Py)
Set a constant Y-component of the direction unit vector for all particles.
- void [setConstantPz](#) (float Pz)
Set a constant Z-component of the direction unit vector for all particles.
- void [setConstantWeight](#) (float weight)
Set a constant statistical weight for all particles.
- void [setConstantX](#) (float X)
Set a constant X coordinate value for all particles.
- void [setConstantY](#) (float Y)
Set a constant Y coordinate value for all particles.
- void [setConstantZ](#) (float Z)
Set a constant Z coordinate value for all particles.

7.14.1 Detailed Description

Base class for reading phase space files.

This abstract class provides a unified interface for reading particle phase space files from different simulation formats (EGS, IAEA, TOPAS, etc.). It handles both binary and ASCII file formats and provides functionality for particle iteration, statistics tracking, and format-specific optimizations. In cases where I/O must be handled by a third-party library (e.g., ROOT), this class also provides a framework for manually reading particles.

7.14.2 Constructor & Destructor Documentation

7.14.2.1 PhaseSpaceFileReader()

```
ParticleZoo::PhaseSpaceFileReader::PhaseSpaceFileReader (
    const std::string & phspFormat,
    const std::string & fileName,
    const UserOptions & userOptions,
    FormatType formatType = FormatType::BINARY,
    const FixedValues fixedValues = FixedValues(),
    unsigned int bufferSize = DEFAULT_BUFFER_SIZE )
```

Construct a new Phase Space File Reader object.

Parameters

<i>phspFormat</i>	The format identifier of the phase space file (e.g., "IAEA", "EGS", "TOPAS")
<i>fileName</i>	The path to the phase space file to read
<i>userOptions</i>	User-defined options for reading behavior
<i>formatType</i>	The format type (BINARY, ASCII, or NONE), defaults to BINARY
<i>fixedValues</i>	Pre-defined constant values for certain particle properties
<i>bufferSize</i>	Size of the internal buffer for reading, defaults to DEFAULT_BUFFER_SIZE

7.14.2.2 ~PhaseSpaceFileReader()

```
ParticleZoo::PhaseSpaceFileReader::~~PhaseSpaceFileReader ( ) [virtual]
```

Destroy the Phase Space File Reader object.

Ensures proper cleanup of file handles and allocated resources.

7.14.3 Member Function Documentation

7.14.3.1 calcThirdUnitComponent() [1/2]

```
double ParticleZoo::PhaseSpaceFileReader::calcThirdUnitComponent (
    double & u,
    double & v ) const [inline], [protected]
```

Calculate the third component of a unit vector from two components (double precision).

Given two components of a unit vector, calculates the third component. Handles normalization if the input components are not properly normalized.

7.14 ParticleZoo::PhaseSpaceFileReader Class Reference

Parameters

u	First component (may be modified for normalization)
v	Second component (may be modified for normalization)

Returns

double The calculated third component

7.14.3.2 calcThirdUnitComponent() [2/2]

```
float ParticleZoo::PhaseSpaceFileReader::calcThirdUnitComponent (
    float & u,
    float & v ) const [inline], [protected]
```

Calculate the third component of a unit vector from two components (float precision).

Given two components of a unit vector, calculates the third component. Handles normalization if the input components are not properly normalized.

Parameters

u	First component (may be modified for normalization)
v	Second component (may be modified for normalization)

Returns

float The calculated third component

7.14.3.3 close()

```
void ParticleZoo::PhaseSpaceFileReader::close ( )
```

Close the phase space file and clean up resources.

Explicitly closes the file handle and frees associated resources. The reader cannot be used after calling this method.

7.14.3.4 getCLICommands()

```
std::vector< CLICommand > ParticleZoo::PhaseSpaceFileReader::getCLICommands ( ) [static]
```

Get command line interface commands supported by this reader.

Returns a vector of CLI commands that can be used with this reader type.

Returns

std::vector<CLICommand> Vector of supported CLI commands

7.14.3.5 getConstantPx()

```
float ParticleZoo::PhaseSpaceFileReader::getConstantPx ( ) const [inline]
```

Get the constant X-component of the direction unit vector (if constant).

Returns

float The constant Px value

Exceptions

<code>std::runtime_error</code>	if Px is not constant
---------------------------------	-----------------------

7.14.3.6 getConstantPy()

```
float ParticleZoo::PhaseSpaceFileReader::getConstantPy ( ) const [inline]
```

Get the constant Y-component of the direction unit vector (if constant).

Returns

float The constant Py value

Exceptions

<code>std::runtime_error</code>	if Py is not constant
---------------------------------	-----------------------

7.14 ParticleZoo::PhaseSpaceFileReader Class Reference

7.14.3.7 getConstantPz()

```
float ParticleZoo::PhaseSpaceFileReader::getConstantPz ( ) const [inline]
```

Get the constant Z-component of the direction unit vector (if constant).

Returns

float The constant Pz value

Exceptions

<i>std::runtime_error</i>	if Pz is not constant
---------------------------	-----------------------

7.14.3.8 getConstantWeight()

```
float ParticleZoo::PhaseSpaceFileReader::getConstantWeight ( ) const [inline]
```

Get the constant statistical weight value (if constant).

Returns

float The constant weight value

Exceptions

<i>std::runtime_error</i>	if weight is not constant
---------------------------	---------------------------

7.14.3.9 getConstantX()

```
float ParticleZoo::PhaseSpaceFileReader::getConstantX ( ) const [inline]
```

Get the constant X coordinate value (if constant).

Returns

float The constant X coordinate value

Exceptions

<code>std::runtime_error</code>	if X is not constant
---------------------------------	----------------------

7.14.3.10 getConstantY()

```
float ParticleZoo::PhaseSpaceFileReader::getConstantY ( ) const [inline]
```

Get the constant Y coordinate value (if constant).

Returns

float The constant Y coordinate value

Exceptions

<code>std::runtime_error</code>	if Y is not constant
---------------------------------	----------------------

7.14.3.11 getConstantZ()

```
float ParticleZoo::PhaseSpaceFileReader::getConstantZ ( ) const [inline]
```

Get the constant Z coordinate value (if constant).

Returns

float The constant Z coordinate value

Exceptions

<code>std::runtime_error</code>	if Z is not constant
---------------------------------	----------------------

7.14.3.12 getFileName()

```
const std::string ParticleZoo::PhaseSpaceFileReader::getFileName ( ) const [inline]
```

Get the filename of the phase space file being read.

7.14 ParticleZoo::PhaseSpaceFileReader Class Reference

Returns

const std::string The filename/path of the file

7.14.3.13 getFileSize()

```
std::uint64_t ParticleZoo::PhaseSpaceFileReader::getFileSize ( ) const [inline]
```

Get the size of the phase space file in bytes.

Returns

std::uint64_t The file size in bytes

7.14.3.14 getFixedValues()

```
const FixedValues ParticleZoo::PhaseSpaceFileReader::getFixedValues ( ) const [inline]
```

Get the fixed values configuration.

Returns

const FixedValues The complete fixed values structure

7.14.3.15 getHeaderData() [1/2]

```
const ByteBuffer ParticleZoo::PhaseSpaceFileReader::getHeaderData ( ) [protected]
```

Get the file header data as a byte buffer.

Reads the entire header portion of the file into a [ByteBuffer](#). The header size is determined by [getParticleRecordStartOffset\(\)](#).

Returns

const ByteBuffer The header data

7.14.3.16 getHeaderData() [2/2]

```
const ByteBuffer ParticleZoo::PhaseSpaceFileReader::getHeaderData (
    std::size_t headerSize ) [protected]
```

Get a specific amount of header data as a byte buffer.

Parameters

<i>headerSize</i>	The number of bytes to read from the header
-------------------	---

Returns

const [ByteBuffer](#) The header data of specified size

7.14.3.17 getHistoriesRead()

```
std::uint64_t ParticleZoo::PhaseSpaceFileReader::getHistoriesRead ( ) [inline], [virtual]
```

Get the number of Monte Carlo histories that have been read so far. If the end of the file has been reached, this will return the total number of original histories unless more histories than expected have already been read - in which case it returns the actual count.

Returns

std::uint64_t The number of histories read

7.14.3.18 getMaximumASCIILineLength()

```
size_t ParticleZoo::PhaseSpaceFileReader::getMaximumASCIILineLength ( ) const [inline], [protected], [virtual]
```

Get the maximum line length for ASCII format files.

Must be implemented by derived classes that support ASCII format. Used for buffer allocation and parsing optimization.

Returns

std::size_t The maximum length of ASCII lines in number of characters

Exceptions

<i>std::runtime_error</i>	if not implemented for ASCII format
---------------------------	-------------------------------------

Reimplemented in [ParticleZoo::penEasyphspFile::Reader](#), and [ParticleZoo::TOPASphspFile::Reader](#).

7.14 ParticleZoo::PhaseSpaceFileReader Class Reference

7.14.3.19 getNextParticle() [1/2]

`Particle` ParticleZoo::PhaseSpaceFileReader::getNextParticle () [inline]

Get the next particle from the phase space file.

Reads and returns the next particle in the file. This method automatically handles buffering and format-specific parsing. The particle is counted in the read statistics.

Returns

`Particle` The next particle object containing position, momentum, energy, etc.

7.14.3.20 getNextParticle() [2/2]

`Particle` ParticleZoo::PhaseSpaceFileReader::getNextParticle (
 `bool countParticleInStatistics`) [protected]

Get the next particle with optional statistics counting control.

This protected version allows derived classes to control whether the particle should be counted in the read statistics.

Parameters

<code>countParticleInStatistics</code>	Whether to count this particle in statistics
--	--

Returns

`Particle` The next particle object

7.14.3.21 getNumberOfEntriesInFile()

`std::size_t` ParticleZoo::PhaseSpaceFileReader::getNumberOfEntriesInFile () const [inline], [protected]

Get the number of particle records that fit in the file.

For binary files, calculates how many complete records fit in the file. For other formats, returns `getNumberOfParticles()`.

Returns

`std::size_t` The number of particle entries in the file

7.14.3.22 `getNumberOfOriginalHistories()`

```
virtual std::uint64_t ParticleZoo::PhaseSpaceFileReader::getNumberOfOriginalHistories ( ) const  
[pure virtual]
```

Get the number of original Monte Carlo histories that generated this phase space.

This is a pure virtual method that must be implemented by derived classes as the method for determining history count varies by format.

Returns

`std::uint64_t` The number of original histories

Implemented in [ParticleZoo::EGSphspFile::Reader](#), [ParticleZoo::IAEaphspFile::Reader](#), [ParticleZoo::penEasyphspFile::Reader](#), [ParticleZoo::ROOT::Reader](#), and [ParticleZoo::TOPASphspFile::Reader](#).

7.14.3.23 `getNumberOfParticles()`

```
virtual std::uint64_t ParticleZoo::PhaseSpaceFileReader::getNumberOfParticles ( ) const [pure  
virtual]
```

Get the total number of particles in the phase space file.

This is a pure virtual method that must be implemented by derived classes as the method for determining particle count varies by format.

Returns

`std::uint64_t` The total number of particles in the file

Implemented in [ParticleZoo::EGSphspFile::Reader](#), [ParticleZoo::IAEaphspFile::Reader](#), [ParticleZoo::penEasyphspFile::Reader](#), [ParticleZoo::ROOT::Reader](#), and [ParticleZoo::TOPASphspFile::Reader](#).

7.14.3.24 `getNumberOfRepresentedHistories()`

```
std::uint64_t ParticleZoo::PhaseSpaceFileReader::getNumberOfRepresentedHistories ( ) const [inline],  
[virtual]
```

Get the number of histories actually represented in this phase space.

This is a virtual method that can be implemented by derived classes. The method for determining represented history count varies by format. If not implemented, this method throws an exception.

Returns

`std::uint64_t` The number of represented histories

7.14 ParticleZoo::PhaseSpaceFileReader Class Reference

Exceptions

<code>std::runtime_error</code>	if this information is not available in the file format
---------------------------------	---

Reimplemented in [ParticleZoo::TOPASphspFile::Reader](#).

7.14.3.25 getParticleRecordLength()

```
std::size_t ParticleZoo::PhaseSpaceFileReader::getParticleRecordLength ( ) const [inline], [protected], [virtual]
```

Get the length in bytes of each particle record.

Must be implemented by derived classes for binary formatted files.

Returns

`std::size_t` The length of each particle record in bytes

Exceptions

<code>std::runtime_error</code>	if not implemented for binary format
---------------------------------	--------------------------------------

Reimplemented in [ParticleZoo::EGSphspFile::Reader](#), [ParticleZoo::IAEAphspFile::Reader](#), and [ParticleZoo::TOPASphspFile::Reader](#).

7.14.3.26 getParticleRecordStartOffset()

```
std::size_t ParticleZoo::PhaseSpaceFileReader::getParticleRecordStartOffset ( ) const [inline], [protected], [virtual]
```

Get the byte offset where particle records start in the file.

This is typically after any file header. Default implementation returns 0.

Returns

`std::size_t` The byte offset of the first particle record

Reimplemented in [ParticleZoo::EGSphspFile::Reader](#), and [ParticleZoo::IAEAphspFile::Reader](#).

7.14.3.27 getParticlesRead() [1/2]

```
std::uint64_t ParticleZoo::PhaseSpaceFileReader::getParticlesRead ( ) [inline], [virtual]
```

Get the number of particles that have been read so far.

This excludes metadata particles and skipped particles.

Returns

std::uint64_t The number of particles read

7.14.3.28 getParticlesRead() [2/2]

```
std::uint64_t ParticleZoo::PhaseSpaceFileReader::getParticlesRead (
    bool includeSkippedParticles ) [inline], [protected], [virtual]
```

Get the number of particles read with optional inclusion of skipped particles (including pseudo-particles).

Parameters

<i>includeSkippedParticles</i>	Whether to include pseudo-particles and particles skipped by moveToParticle()
--------------------------------	---

Returns

std::uint64_t The number of particles read

7.14.3.29 getPHSPFormat()

```
const std::string ParticleZoo::PhaseSpaceFileReader::getPHSPFormat ( ) const [inline]
```

Get the phase space file format identifier.

Returns

const std::string The format identifier (e.g., "IAEA", "EGS", "TOPAS")

7.14 ParticleZoo::PhaseSpaceFileReader Class Reference

7.14.3.30 getUserOptions()

```
const UserOptions & ParticleZoo::PhaseSpaceFileReader::getUserOptions ( ) const [inline], [protected]
```

Get the user options that were passed to the constructor.

Returns

const UserOptions& Reference to the user options

7.14.3.31 hasMoreParticles()

```
bool ParticleZoo::PhaseSpaceFileReader::hasMoreParticles ( ) [virtual]
```

Check if there are more particles to read in the file.

Returns

true if there are more particles available to read

false if the end of file has been reached

7.14.3.32 hasNativeIncrementalHistoryCounters()

```
bool ParticleZoo::PhaseSpaceFileReader::hasNativeIncrementalHistoryCounters ( ) const [inline],  
[virtual]
```

Check if this format directly stores incremental history numbers per-particle.

When true, particles returned by [getNextParticle\(\)](#) carry the INCREMENTAL_HISTORY_NUMBER property with file-sourced values. Otherwise the incremental history numbers may be indirectly derived or otherwise determined.

Returns

true if incremental history data is directly stored per-particle in this format

Reimplemented in [ParticleZoo::IAEAphspFile::Reader](#), [ParticleZoo::penEasyphspFile::Reader](#), and [ParticleZoo::ROOT::Reader](#).

7.14.3.33 hasNativeRepresentedHistoryCount()

```
bool ParticleZoo::PhaseSpaceFileReader::hasNativeRepresentedHistoryCount ( ) const [inline],  
[virtual]
```

Check if this format can provide the number of represented histories without scanning the file.

Returns

true if [getNumberOfRepresentedHistories\(\)](#) is cheap (e.g. stored in header)

Reimplemented in [ParticleZoo::TOPASphspFile::Reader](#).

7.14.3.34 isPxConstant()

```
bool ParticleZoo::PhaseSpaceFileReader::isPxConstant ( ) const [inline]
```

Check if the X-component of momentum is constant for all particles.

Returns

true if Px is constant across all particles

false if Px varies between particles

7.14.3.35 isPyConstant()

```
bool ParticleZoo::PhaseSpaceFileReader::isPyConstant ( ) const [inline]
```

Check if the Y-component of momentum is constant for all particles.

Returns

true if Py is constant across all particles

false if Py varies between particles

7.14 ParticleZoo::PhaseSpaceFileReader Class Reference

7.14.3.36 isPzConstant()

```
bool ParticleZoo::PhaseSpaceFileReader::isPzConstant ( ) const [inline]
```

Check if the Z-component of momentum is constant for all particles.

Returns

true if Pz is constant across all particles
false if Pz varies between particles

7.14.3.37 isWeightConstant()

```
bool ParticleZoo::PhaseSpaceFileReader::isWeightConstant ( ) const [inline]
```

Check if the statistical weight is constant for all particles.

Returns

true if weight is constant across all particles
false if weight varies between particles

7.14.3.38 isXConstant()

```
bool ParticleZoo::PhaseSpaceFileReader::isXConstant ( ) const [inline]
```

Check if the X coordinate is constant for all particles.

Returns

true if X coordinate is constant across all particles
false if X coordinate varies between particles

7.14.3.39 isYConstant()

```
bool ParticleZoo::PhaseSpaceFileReader::isYConstant ( ) const [inline]
```

Check if the Y coordinate is constant for all particles.

Returns

true if Y coordinate is constant across all particles
false if Y coordinate varies between particles

7.14.3.40 isZConstant()

```
bool ParticleZoo::PhaseSpaceFileReader::isZConstant ( ) const [inline]
```

Check if the Z coordinate is constant for all particles.

Returns

true if Z coordinate is constant across all particles

false if Z coordinate varies between particles

7.14.3.41 moveToParticle()

```
void ParticleZoo::PhaseSpaceFileReader::moveToParticle (
    std::uint64_t particleIndex )
```

Move the file position to a specific particle index.

Allows random access to particles within the file. The next call to [getNextParticle\(\)](#) will return the particle at the specified index.

Parameters

<i>particleIndex</i>	Zero-based index of the particle to move to
----------------------	---

7.14.3.42 peekNextParticle()

```
Particle ParticleZoo::PhaseSpaceFileReader::peekNextParticle ( )
```

Peek at the next particle without advancing the file position.

Reads the next particle but does not move the internal file pointer forward. This allows for inspecting the upcoming particle without consuming it.

Returns

[Particle](#) The next particle object containing position, momentum, energy, etc.

7.14 ParticleZoo::PhaseSpaceFileReader Class Reference

7.14.3.43 peekParticleManually()

`Particle` ParticleZoo::PhaseSpaceFileReader::peekParticleManually () [inline], [protected], [virtual]

Peek at a particle manually (for formats requiring third-party I/O).

Can be implemented by derived classes to support manual file I/O, circumventing the internal file stream and buffer.

Must be implemented by derived classes that specify `FormatType::NONE`. The default implementation throws an exception.

Returns

`Particle` The manually entered particle object

Exceptions

<code>std::runtime_error</code>	if not implemented
---------------------------------	--------------------

Reimplemented in `ParticleZoo::ROOT::Reader`.

7.14.3.44 readASCIIParticle()

`Particle` ParticleZoo::PhaseSpaceFileReader::readASCIIParticle (
 const std::string & *line*) [inline], [protected], [virtual]

Read a particle from ASCII data.

Must be implemented by derived classes that support ASCII format. The default implementation throws an exception.

Parameters

<i>line</i>	The ASCII line containing the particle data
-------------	---

Returns

`Particle` The particle object parsed from ASCII data

Exceptions

<code>std::runtime_error</code>	if not implemented for ASCII format
---------------------------------	-------------------------------------

Reimplemented in [ParticleZoo::penEasyphspFile::Reader](#), and [ParticleZoo::TOPASphspFile::Reader](#).

7.14.3.45 readBinaryParticle()

```
Particle ParticleZoo::PhaseSpaceFileReader::readBinaryParticle (
    ByteBuffer & buffer ) [inline], [protected], [virtual]
```

Read a particle from binary data.

Must be implemented by derived classes that support binary format. The default implementation throws an exception.

Parameters

<i>buffer</i>	The byte buffer containing the particle data
---------------	--

Returns

[Particle](#) The particle object parsed from binary data

Exceptions

<code>std::runtime_error</code>	if not implemented for binary format
---------------------------------	--------------------------------------

Reimplemented in [ParticleZoo::EGSphspFile::Reader](#), [ParticleZoo::IAEaphspFile::Reader](#), and [ParticleZoo::TOPASphspFile::Reader](#).

7.14.3.46 readParticleManually()

```
Particle ParticleZoo::PhaseSpaceFileReader::readParticleManually ( ) [inline], [protected], [virtual]
```

Read a particle manually (for formats requiring third-party I/O).

Can be implemented by derived classes to support manual file I/O, circumventing the internal file stream and buffer.

Must be implemented by derived classes that specify [FormatType::NONE](#). The default implementation throws an exception.

7.14 ParticleZoo::PhaseSpaceFileReader Class Reference

Returns

[Particle](#) The manually entered particle object

Exceptions

<code>std::runtime_error</code>	if not implemented
---------------------------------	--------------------

Reimplemented in [ParticleZoo::ROOT::Reader](#).

7.14.3.47 setByteOrder()

```
void ParticleZoo::PhaseSpaceFileReader::setByteOrder (
    ByteOrder byteOrder ) [inline], [protected]
```

Set the byte order for binary data interpretation.

Parameters

<i>byteOrder</i>	The byte order to use (little-endian, big-endian, or PDP-endian)
------------------	--

7.14.3.48 setCommentMarkers()

```
void ParticleZoo::PhaseSpaceFileReader::setCommentMarkers (
    const std::vector< std::string > & commentMarkers ) [inline]
```

Set comment markers for ASCII format files.

Defines the strings that mark comment lines in ASCII format files. Lines beginning with these markers will be ignored during parsing.

Parameters

<i>commentMarkers</i>	Vector of strings that indicate comment lines
-----------------------	---

7.14.3.49 setConstantPx()

```
void ParticleZoo::PhaseSpaceFileReader::setConstantPx (
    float Px ) [inline], [protected]
```

Set a constant X-component of the direction unit vector for all particles.

Parameters

<i>Px</i>	The constant Px value to set
-----------	------------------------------

7.14.3.50 **setConstantPy()**

```
void ParticleZoo::PhaseSpaceFileReader::setConstantPy (
    float Py ) [inline], [protected]
```

Set a constant Y-component of the direction unit vector for all particles.

Parameters

<i>Py</i>	The constant Py value to set
-----------	------------------------------

7.14.3.51 **setConstantPz()**

```
void ParticleZoo::PhaseSpaceFileReader::setConstantPz (
    float Pz ) [inline], [protected]
```

Set a constant Z-component of the direction unit vector for all particles.

Parameters

<i>Pz</i>	The constant Pz value to set
-----------	------------------------------

7.14.3.52 **setConstantWeight()**

```
void ParticleZoo::PhaseSpaceFileReader::setConstantWeight (
    float weight ) [inline], [protected]
```

Set a constant statistical weight for all particles.

Parameters

<i>weight</i>	The constant weight value to set
---------------	----------------------------------

7.14 ParticleZoo::PhaseSpaceFileReader Class Reference

7.14.3.53 setConstantX()

```
void ParticleZoo::PhaseSpaceFileReader::setConstantX (
    float X ) [inline], [protected]
```

Set a constant X coordinate value for all particles.

Parameters

<i>X</i>	The constant X coordinate value to set
----------	--

7.14.3.54 setConstantY()

```
void ParticleZoo::PhaseSpaceFileReader::setConstantY (
    float Y ) [inline], [protected]
```

Set a constant Y coordinate value for all particles.

Parameters

<i>Y</i>	The constant Y coordinate value to set
----------	--

7.14.3.55 setConstantZ()

```
void ParticleZoo::PhaseSpaceFileReader::setConstantZ (
    float Z ) [inline], [protected]
```

Set a constant Z coordinate value for all particles.

Parameters

<i>Z</i>	The constant Z coordinate value to set
----------	--

The documentation for this class was generated from the following files:

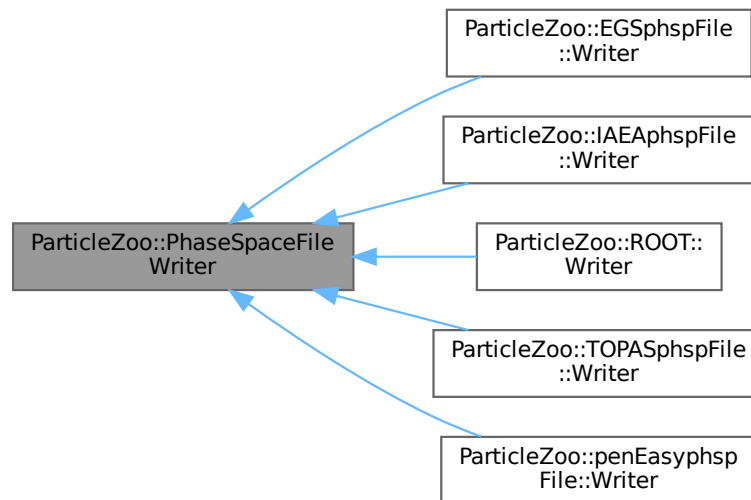
- include/particlezoo/PhaseSpaceFileReader.h
- src/PhaseSpaceFileReader.cc

7.15 ParticleZoo::PhaseSpaceFileWriter Class Reference

Base class for writing phase space files.

```
#include <particlezoo/PhaseSpaceFileWriter.h>
```

Inheritance diagram for ParticleZoo::PhaseSpaceFileWriter:



Public Member Functions

- [PhaseSpaceFileWriter](#) (const std::string &phspFormat, const std::string &fileName, const UserOptions &userOptions, [FormatType](#) formatType=[FormatType::BINARY](#), const [FixedValues](#) fixedValues=[FixedValues\(\)](#), unsigned int bufferSize=[DEFAULT_BUFFER_SIZE](#))
Construct a new Phase Space File Writer object.
- virtual [~PhaseSpaceFileWriter](#) ()
Destroy the Phase Space File Writer object.
- void [addAdditionalHistories](#) (std::uint64_t additionalHistories)
Add additional Monte Carlo histories to the count.
- void [close](#) ()
Close the phase space file and finalize writing.
- [ByteOrder](#) [getByteOrder](#) () const

7.15 ParticleZoo::PhaseSpaceFileWriter Class Reference

- Get the byte order used for binary data writing.*
- float [getConstantPx](#) () const
 - Get the constant X-component of the direction unit vector (if constant).*
- float [getConstantPy](#) () const
 - Get the constant Y-component of the direction unit vector (if constant).*
- float [getConstantPz](#) () const
 - Get the constant Z-component of the direction unit vector (if constant).*
- float [getConstantWeight](#) () const
 - Get the constant statistical weight value (if constant).*
- float [getConstantX](#) () const
 - Get the constant X coordinate value (if constant).*
- float [getConstantY](#) () const
 - Get the constant Y coordinate value (if constant).*
- float [getConstantZ](#) () const
 - Get the constant Z coordinate value (if constant).*
- const std::string [getFileName](#) () const
 - Get the filename where the phase space file is being written.*
- const [FixedValues](#) [getFixedValues](#) () const
 - Get the fixed values configuration.*
- virtual std::uint64_t [getHistoriesWritten](#) () const
 - Get the number of Monte Carlo histories that have been written.*
- virtual std::uint64_t [getMaximumSupportedParticles](#) () const =0
 - Get the maximum number of particles this writer can support.*
- std::uint64_t [getParticlesWritten](#) () const
 - Get the number of particles that have been written to the file.*
- const std::string [getPHSPFormat](#) () const
 - Get the phase space file format identifier.*
- bool [isPxConstant](#) () const
 - Check if the X-component of the direction unit vector is set to a constant value for all particles.*
- bool [isPyConstant](#) () const
 - Check if the Y-component of the direction unit vector is set to a constant value for all particles.*
- bool [isPzConstant](#) () const
 - Check if the Z-component of the direction unit vector is set to a constant value for all particles.*
- bool [isWeightConstant](#) () const
 - Check if the statistical weight is set to a constant value for all particles.*
- bool [isXConstant](#) () const
 - Check if the X coordinate is set to a constant value for all particles.*
- bool [isYConstant](#) () const
 - Check if the Y coordinate is set to a constant value for all particles.*
- bool [isZConstant](#) () const
 - Check if the Z coordinate is set to a constant value for all particles.*
- virtual void [writeParticle](#) ([Particle](#) particle)
 - Write a particle to the phase space file.*

Static Public Member Functions

- static std::vector< CLICommand > [getCLICommands](#) ()
Get command line interface commands supported by this writer.

Protected Member Functions

- virtual bool [accountForAdditionalHistories](#) (std::uint64_t additionalHistories)
Handle accounting for simulation histories that produced no particles.
- virtual bool [canHaveConstantPx](#) () const
Check if this format supports constant X-component of the direction unit vector.
- virtual bool [canHaveConstantPy](#) () const
Check if this format supports constant Y-component of the direction unit vector.
- virtual bool [canHaveConstantPz](#) () const
Check if this format supports constant Z-component of the direction unit vector.
- virtual bool [canHaveConstantWeight](#) () const
Check if this format supports constant statistical weights.
- virtual bool [canHaveConstantX](#) () const
Check if this format supports constant X coordinates.
- virtual bool [canHaveConstantY](#) () const
Check if this format supports constant Y coordinates.
- virtual bool [canHaveConstantZ](#) () const
Check if this format supports constant Z coordinates.
- virtual bool [canWritePseudoParticlesExplicitly](#) () const
Check if this format can write pseudo-particles explicitly.
- virtual void [fixedValuesHaveChanged](#) ()
Called when fixed values have been changed.
- virtual size_t [getMaximumASCIILineLength](#) () const
Get the maximum line length for ASCII format files.
- virtual std::size_t [getParticleRecordLength](#) () const
Get the length in bytes of each particle record.
- virtual std::size_t [getParticleRecordStartOffset](#) () const
Get the byte offset where particle records start in the file.
- virtual std::uint64_t [getPendingHistories](#) () const
Get the number of pending histories to account for.
- const UserOptions & [getUserOptions](#) () const
Get the user options that were passed to the constructor.
- void [setByteOrder](#) (ByteOrder byteOrder)
Set the byte order for binary data writing.
- void [setConstantPx](#) (float Px)
Set a constant X-component of the direction unit vector for all particles.

7.15 ParticleZoo::PhaseSpaceFileWriter Class Reference

- void [setConstantPy](#) (float Py)
Set a constant Y-component of the direction unit vector for all particles.
- void [setConstantPz](#) (float Pz)
Set a constant Z-component of the direction unit vector for all particles.
- void [setConstantWeight](#) (float weight)
Set a constant statistical weight for all particles.
- void [setConstantX](#) (float X)
Set a constant X coordinate value for all particles.
- void [setConstantY](#) (float Y)
Set a constant Y coordinate value for all particles.
- void [setConstantZ](#) (float Z)
Set a constant Z coordinate value for all particles.
- virtual const std::string [writeASCIIParticle](#) ([Particle](#) &particle)
Write a particle in ASCII format as a string.
- virtual void [writeBinaryParticle](#) ([ByteBuffer](#) &buffer, [Particle](#) &particle)
Write a particle in binary format to a byte buffer.
- virtual void [writeHeaderData](#) ([ByteBuffer](#) &buffer)=0
Write header data to a byte buffer.
- virtual void [writeParticleManually](#) ([Particle](#) &particle)
Write a particle manually (for formats requiring third-party I/O).

7.15.1 Detailed Description

Base class for writing phase space files.

This abstract class provides a unified interface for writing particle phase space files to different simulation formats (EGS, IAEA, TOPAS, etc.). It handles both binary and ASCII file formats, provides buffering for efficient I/O, and supports statistics tracking and format-specific optimizations. In cases where I/O must be handled by a third-party library (e.g., ROOT), this class also provides a framework for manually writing particles.

7.15.2 Constructor & Destructor Documentation

7.15.2.1 PhaseSpaceFileWriter()

```
ParticleZoo::PhaseSpaceFileWriter::PhaseSpaceFileWriter (  
    const std::string & phspFormat,  
    const std::string & fileName,  
    const UserOptions & userOptions,  
    FormatType formatType = FormatType::BINARY,  
    const FixedValues fixedValues = FixedValues(),  
    unsigned int bufferSize = DEFAULT_BUFFER_SIZE )
```

Construct a new Phase Space File Writer object.

Parameters

<i>phspFormat</i>	The format identifier of the phase space file (e.g., "IAEA", "EGS", "TOPAS")
<i>fileName</i>	The path where the phase space file will be written
<i>userOptions</i>	User-defined options for writing behavior
<i>formatType</i>	The format type (BINARY or ASCII), defaults to BINARY
<i>fixedValues</i>	Pre-defined constant values for certain particle properties
<i>bufferSize</i>	Size of the internal buffer for writing, defaults to DEFAULT_BUFFER_SIZE

7.15.2.2 ~PhaseSpaceFileWriter()

```
ParticleZoo::PhaseSpaceFileWriter::~~PhaseSpaceFileWriter ( ) [virtual]
```

Destroy the Phase Space File Writer object.

Ensures proper cleanup by closing the file and flushing any remaining buffered data.

7.15.3 Member Function Documentation**7.15.3.1 accountForAdditionalHistories()**

```
bool ParticleZoo::PhaseSpaceFileWriter::accountForAdditionalHistories (
    std::uint64_t additionalHistories ) [inline], [protected], [virtual]
```

Handle accounting for simulation histories that produced no particles.

Called by [addAdditionalHistories\(\)](#) to handle format-specific requirements for empty histories. Some formats need special handling such as writing pseudo-particles or updating header counters.

The default implementation returns true, indicating that the base class should automatically increment the history counter. Derived classes can override this to handle it manually (e.g., by writing additional pseudo-particles)

Parameters

<i>additionalHistories</i>	The number of additional (empty) histories
----------------------------	--

7.15 ParticleZoo::PhaseSpaceFileWriter Class Reference

Returns

- true if the base class should automatically increment the history counter
- false if the derived class handles it manually (e.g., by writing additional pseudo-particles)

Reimplemented in [ParticleZoo::TOPASphspFile::Writer](#).

7.15.3.2 addAdditionalHistories()

```
void ParticleZoo::PhaseSpaceFileWriter::addAdditionalHistories (
    std::uint64_t additionalHistories ) [inline]
```

Add additional Monte Carlo histories to the count.

Used to account for simulation histories that produced no particles to write. Some formats may need special handling for empty histories.

Parameters

<i>additionalHistories</i>	The number of additional (empty) histories to account for
----------------------------	---

7.15.3.3 canHaveConstantPx()

```
bool ParticleZoo::PhaseSpaceFileWriter::canHaveConstantPx ( ) const [inline], [protected], [virtual]
```

Check if this format supports constant X-component of the direction unit vector.

Derived classes can override this to indicate format-specific capabilities. Default implementation returns false.

Returns

- true if constant Px is supported by this format
- false if constant Px is not supported

Reimplemented in [ParticleZoo::IAEAphspFile::Writer](#).

7.15.3.4 canHaveConstantPy()

```
bool ParticleZoo::PhaseSpaceFileWriter::canHaveConstantPy ( ) const [inline], [protected], [virtual]
```

Check if this format supports constant Y-component of the direction unit vector.

Derived classes can override this to indicate format-specific capabilities. Default implementation returns false.

Returns

true if constant Py is supported by this format
false if constant Py is not supported

Reimplemented in [ParticleZoo::IAEAphspFile::Writer](#).

7.15.3.5 canHaveConstantPz()

```
bool ParticleZoo::PhaseSpaceFileWriter::canHaveConstantPz ( ) const [inline], [protected], [virtual]
```

Check if this format supports constant Z-component of the direction unit vector.

Derived classes can override this to indicate format-specific capabilities. Default implementation returns false.

Returns

true if constant Pz is supported by this format
false if constant Pz is not supported

Reimplemented in [ParticleZoo::IAEAphspFile::Writer](#).

7.15.3.6 canHaveConstantWeight()

```
bool ParticleZoo::PhaseSpaceFileWriter::canHaveConstantWeight ( ) const [inline], [protected], [virtual]
```

Check if this format supports constant statistical weights.

Derived classes can override this to indicate format-specific capabilities. Default implementation returns false.

Returns

true if constant weights are supported by this format
false if constant weights are not supported

Reimplemented in [ParticleZoo::IAEAphspFile::Writer](#).

7.15 ParticleZoo::PhaseSpaceFileWriter Class Reference

7.15.3.7 canHaveConstantX()

```
bool ParticleZoo::PhaseSpaceFileWriter::canHaveConstantX ( ) const [inline], [protected], [virtual]
```

Check if this format supports constant X coordinates.

Derived classes can override this to indicate format-specific capabilities. Default implementation returns false.

Returns

true if constant X coordinates are supported by this format
false if constant X coordinates are not supported

Reimplemented in [ParticleZoo::IAEAphspFile::Writer](#).

7.15.3.8 canHaveConstantY()

```
bool ParticleZoo::PhaseSpaceFileWriter::canHaveConstantY ( ) const [inline], [protected], [virtual]
```

Check if this format supports constant Y coordinates.

Derived classes can override this to indicate format-specific capabilities. Default implementation returns false.

Returns

true if constant Y coordinates are supported by this format
false if constant Y coordinates are not supported

Reimplemented in [ParticleZoo::IAEAphspFile::Writer](#).

7.15.3.9 canHaveConstantZ()

```
bool ParticleZoo::PhaseSpaceFileWriter::canHaveConstantZ ( ) const [inline], [protected], [virtual]
```

Check if this format supports constant Z coordinates.

Derived classes can override this to indicate format-specific capabilities. Default implementation returns false.

Returns

true if constant Z coordinates are supported by this format
false if constant Z coordinates are not supported

Reimplemented in [ParticleZoo::IAEAphspFile::Writer](#).

7.15.3.10 canWritePseudoParticlesExplicitly()

```
bool ParticleZoo::PhaseSpaceFileWriter::canWritePseudoParticlesExplicitly ( ) const [inline],  
[protected], [virtual]
```

Check if this format can write pseudo-particles explicitly.

Derived classes can override this to indicate if they support writing pseudo-particles (metadata particles) explicitly to the file. Default implementation returns false.

Returns

- true if pseudo-particles can be written explicitly
- false if explicit pseudo-particle writing is not supported

Reimplemented in [ParticleZoo::TOPASphspFile::Writer](#).

7.15.3.11 close()

```
void ParticleZoo::PhaseSpaceFileWriter::close ( )
```

Close the phase space file and finalize writing.

Flushes any remaining buffered data, writes the file header, and closes the file handle. The writer cannot be used after calling this method.

7.15.3.12 fixedValuesHaveChanged()

```
virtual void ParticleZoo::PhaseSpaceFileWriter::fixedValuesHaveChanged ( ) [inline], [protected],  
[virtual]
```

Called when fixed values have been changed.

Derived classes can override this to perform any necessary updates when constant values are modified. Default implementation does nothing.

Reimplemented in [ParticleZoo::IAEAphspFile::Writer](#).

7.15 ParticleZoo::PhaseSpaceFileWriter Class Reference

7.15.3.13 getByteOrder()

```
ByteOrder ParticleZoo::PhaseSpaceFileWriter::getByteOrder ( ) const [inline]
```

Get the byte order used for binary data writing.

Returns

ByteOrder The current byte order (little-endian, big-endian, or PDP-endian)

7.15.3.14 getCLICommands()

```
std::vector< CLICommand > ParticleZoo::PhaseSpaceFileWriter::getCLICommands ( ) [static]
```

Get command line interface commands supported by this writer.

Returns a vector of CLI commands that can be used with this writer type.

Returns

std::vector<CLICommand> Vector of supported CLI commands

7.15.3.15 getConstantPx()

```
float ParticleZoo::PhaseSpaceFileWriter::getConstantPx ( ) const [inline]
```

Get the constant X-component of the direction unit vector (if constant).

Returns

float The constant Px value

Exceptions

<code>std::runtime_error</code>	if Px is not set to constant
---------------------------------	------------------------------

7.15.3.16 getConstantPy()

```
float ParticleZoo::PhaseSpaceFileWriter::getConstantPy ( ) const [inline]
```

Get the constant Y-component of the direction unit vector (if constant).

Returns

float The constant Py value

Exceptions

<code>std::runtime_error</code>	if Py is not set to constant
---------------------------------	------------------------------

7.15.3.17 getConstantPz()

```
float ParticleZoo::PhaseSpaceFileWriter::getConstantPz ( ) const [inline]
```

Get the constant Z-component of the direction unit vector (if constant).

Returns

float The constant Pz value

Exceptions

<code>std::runtime_error</code>	if Pz is not set to constant
---------------------------------	------------------------------

7.15.3.18 getConstantWeight()

```
float ParticleZoo::PhaseSpaceFileWriter::getConstantWeight ( ) const [inline]
```

Get the constant statistical weight value (if constant).

Returns

float The constant weight value

7.15 ParticleZoo::PhaseSpaceFileWriter Class Reference

Exceptions

<code>std::runtime_error</code>	if weight is not set to constant
---------------------------------	----------------------------------

7.15.3.19 getConstantX()

```
float ParticleZoo::PhaseSpaceFileWriter::getConstantX ( ) const [inline]
```

Get the constant X coordinate value (if constant).

Returns

float The constant X coordinate value

Exceptions

<code>std::runtime_error</code>	if X is not set to constant
---------------------------------	-----------------------------

7.15.3.20 getConstantY()

```
float ParticleZoo::PhaseSpaceFileWriter::getConstantY ( ) const [inline]
```

Get the constant Y coordinate value (if constant).

Returns

float The constant Y coordinate value

Exceptions

<code>std::runtime_error</code>	if Y is not set to constant
---------------------------------	-----------------------------

7.15.3.21 getConstantZ()

```
float ParticleZoo::PhaseSpaceFileWriter::getConstantZ ( ) const [inline]
```

Get the constant Z coordinate value (if constant).

Returns

float The constant Z coordinate value

Exceptions

<code>std::runtime_error</code>	if Z is not set to constant
---------------------------------	-----------------------------

7.15.3.22 getFileName()

```
const std::string ParticleZoo::PhaseSpaceFileWriter::getFileName ( ) const [inline]
```

Get the filename where the phase space file is being written.

Returns

const std::string The filename/path of the output file

7.15.3.23 getFixedValues()

```
const FixedValues ParticleZoo::PhaseSpaceFileWriter::getFixedValues ( ) const [inline]
```

Get the fixed values configuration.

Returns

const FixedValues The complete fixed values structure

7.15.3.24 getHistoriesWritten()

```
std::uint64_t ParticleZoo::PhaseSpaceFileWriter::getHistoriesWritten ( ) const [inline], [virtual]
```

Get the number of Monte Carlo histories that have been written.

Returns

std::uint64_t The number of histories written to the file

7.15 ParticleZoo::PhaseSpaceFileWriter Class Reference

7.15.3.25 getMaximumASCIILineLength()

```
size_t ParticleZoo::PhaseSpaceFileWriter::getMaximumASCIILineLength ( ) const [inline], [protected],  
[virtual]
```

Get the maximum line length for ASCII format files.

Must be implemented by derived classes that support ASCII format. Used for buffer allocation and writing optimization.

Returns

std::size_t The maximum length of ASCII lines in characters

Exceptions

<i>std::runtime_error</i>	if not implemented for ASCII format
---------------------------	-------------------------------------

Reimplemented in [ParticleZoo::penEasyphspFile::Writer](#), and [ParticleZoo::TOPASphspFile::Writer](#).

7.15.3.26 getMaximumSupportedParticles()

```
virtual std::uint64_t ParticleZoo::PhaseSpaceFileWriter::getMaximumSupportedParticles ( ) const  
[pure virtual]
```

Get the maximum number of particles this writer can support.

This is a pure virtual method that must be implemented by derived classes as the maximum can vary by format.

Returns

std::uint64_t The maximum number of particles supported

Implemented in [ParticleZoo::EGSphspFile::Writer](#), [ParticleZoo::IAEphspFile::Writer](#), [ParticleZoo::penEasyphspFile::Writer](#), [ParticleZoo::ROOT::Writer](#), and [ParticleZoo::TOPASphspFile::Writer](#).

7.15.3.27 getParticleRecordLength()

```
std::size_t ParticleZoo::PhaseSpaceFileWriter::getParticleRecordLength ( ) const [inline], [protected],  
[virtual]
```

Get the length in bytes of each particle record.

Must be implemented by derived classes for binary formatted files.

Returns

std::size_t The length of each particle record in bytes

Exceptions

<code>std::runtime_error</code>	if not implemented for binary format
---------------------------------	--------------------------------------

Reimplemented in [ParticleZoo::EGSphspFile::Writer](#), [ParticleZoo::IAEphspFile::Writer](#), and [ParticleZoo::TOPASphspFile::Writer](#).

7.15.3.28 getParticleRecordStartOffset()

```
std::size_t ParticleZoo::PhaseSpaceFileWriter::getParticleRecordStartOffset ( ) const [inline],
[protected], [virtual]
```

Get the byte offset where particle records start in the file.

This is typically after any file header. Default implementation returns 0.

Returns

std::size_t The byte offset of the first particle record

Reimplemented in [ParticleZoo::EGSphspFile::Writer](#), and [ParticleZoo::penEasyphspFile::Writer](#).

7.15.3.29 getParticlesWritten()

```
std::uint64_t ParticleZoo::PhaseSpaceFileWriter::getParticlesWritten ( ) const [inline]
```

Get the number of particles that have been written to the file.

Returns

std::uint64_t The number of particles written (excludes pseudo-particles)

7.15.3.30 getPendingHistories()

```
std::uint64_t ParticleZoo::PhaseSpaceFileWriter::getPendingHistories ( ) const [inline], [protected],
[virtual]
```

Get the number of pending histories to account for.

Used internally to track histories that have not yet been written to the file (e.g., empty histories).

Override in derived classes if special handling is needed.

Returns

std::uint64_t The number of pending histories

Reimplemented in [ParticleZoo::TOPASphspFile::Writer](#).

7.15 ParticleZoo::PhaseSpaceFileWriter Class Reference

7.15.3.31 getPHSPFormat()

```
const std::string ParticleZoo::PhaseSpaceFileWriter::getPHSPFormat ( ) const [inline]
```

Get the phase space file format identifier.

Returns

const std::string The format identifier (e.g., "IAEA", "EGS", "TOPAS")

7.15.3.32 getUserOptions()

```
const UserOptions & ParticleZoo::PhaseSpaceFileWriter::getUserOptions ( ) const [inline], [protected]
```

Get the user options that were passed to the constructor.

Returns

const UserOptions& Reference to the user options

7.15.3.33 isPxConstant()

```
bool ParticleZoo::PhaseSpaceFileWriter::isPxConstant ( ) const [inline]
```

Check if the X-component of the direction unit vector is set to a constant value for all particles.

Returns

true if Px is constant across all particles

false if Px varies between particles

7.15.3.34 isPyConstant()

```
bool ParticleZoo::PhaseSpaceFileWriter::isPyConstant ( ) const [inline]
```

Check if the Y-component of the direction unit vector is set to a constant value for all particles.

Returns

true if Py is constant across all particles

false if Py varies between particles

7.15.3.35 isPzConstant()

```
bool ParticleZoo::PhaseSpaceFileWriter::isPzConstant ( ) const [inline]
```

Check if the Z-component of the direction unit vector is set to a constant value for all particles.

Returns

true if Pz is constant across all particles
false if Pz varies between particles

7.15.3.36 isWeightConstant()

```
bool ParticleZoo::PhaseSpaceFileWriter::isWeightConstant ( ) const [inline]
```

Check if the statistical weight is set to a constant value for all particles.

Returns

true if weight is constant across all particles
false if weight varies between particles

7.15.3.37 isXConstant()

```
bool ParticleZoo::PhaseSpaceFileWriter::isXConstant ( ) const [inline]
```

Check if the X coordinate is set to a constant value for all particles.

Returns

true if X coordinate is constant across all particles
false if X coordinate varies between particles

7.15.3.38 isYConstant()

```
bool ParticleZoo::PhaseSpaceFileWriter::isYConstant ( ) const [inline]
```

Check if the Y coordinate is set to a constant value for all particles.

Returns

true if Y coordinate is constant across all particles
false if Y coordinate varies between particles

7.15 ParticleZoo::PhaseSpaceFileWriter Class Reference

7.15.3.39 isZConstant()

```
bool ParticleZoo::PhaseSpaceFileWriter::isZConstant ( ) const [inline]
```

Check if the Z coordinate is set to a constant value for all particles.

Returns

true if Z coordinate is constant across all particles

false if Z coordinate varies between particles

7.15.3.40 setByteOrder()

```
void ParticleZoo::PhaseSpaceFileWriter::setByteOrder (
    ByteOrder byteOrder ) [inline], [protected]
```

Set the byte order for binary data writing.

Parameters

<i>byteOrder</i>	The byte order to use (little-endian, big-endian, or PDP-endian)
------------------	--

7.15.3.41 setConstantPx()

```
void ParticleZoo::PhaseSpaceFileWriter::setConstantPx (
    float Px ) [inline], [protected]
```

Set a constant X-component of the direction unit vector for all particles.

Parameters

<i>Px</i>	The constant Px value to set
-----------	------------------------------

7.15.3.42 setConstantPy()

```
void ParticleZoo::PhaseSpaceFileWriter::setConstantPy (
    float Py ) [inline], [protected]
```

Set a constant Y-component of the direction unit vector for all particles.

7.15 ParticleZoo::PhaseSpaceFileWriter Class Reference

Parameters

<i>Py</i>	The constant Py value to set
-----------	------------------------------

7.15.3.43 setConstantPz()

```
void ParticleZoo::PhaseSpaceFileWriter::setConstantPz (
    float Pz ) [inline], [protected]
```

Set a constant Z-component of the direction unit vector for all particles.

Parameters

<i>Pz</i>	The constant Pz value to set
-----------	------------------------------

7.15.3.44 setConstantWeight()

```
void ParticleZoo::PhaseSpaceFileWriter::setConstantWeight (
    float weight ) [inline], [protected]
```

Set a constant statistical weight for all particles.

Parameters

<i>weight</i>	The constant weight value to set
---------------	----------------------------------

7.15.3.45 setConstantX()

```
void ParticleZoo::PhaseSpaceFileWriter::setConstantX (
    float X ) [inline], [protected]
```

Set a constant X coordinate value for all particles.

Parameters

<i>X</i>	The constant X coordinate value to set
----------	--

7.15.3.46 setConstantY()

```
void ParticleZoo::PhaseSpaceFileWriter::setConstantY (
    float Y ) [inline], [protected]
```

Set a constant Y coordinate value for all particles.

Parameters

Y	The constant Y coordinate value to set
---	--

7.15.3.47 setConstantZ()

```
void ParticleZoo::PhaseSpaceFileWriter::setConstantZ (
    float Z ) [inline], [protected]
```

Set a constant Z coordinate value for all particles.

Parameters

Z	The constant Z coordinate value to set
---	--

7.15.3.48 writeASCIIParticle()

```
const std::string ParticleZoo::PhaseSpaceFileWriter::writeASCIIParticle (
    Particle & particle ) [inline], [protected], [virtual]
```

Write a particle in ASCII format as a string.

Must be implemented by derived classes that support ASCII format. The default implementation throws an exception.

Parameters

<i>particle</i>	The particle object to write
-----------------	------------------------------

Returns

const std::string The ASCII representation of the particle

7.15 ParticleZoo::PhaseSpaceFileWriter Class Reference

Exceptions

<i>std::runtime_error</i>	if not implemented for ASCII format
---------------------------	-------------------------------------

Reimplemented in [ParticleZoo::penEasyphspFile::Writer](#), and [ParticleZoo::TOPASphspFile::Writer](#).

7.15.3.49 writeBinaryParticle()

```
void ParticleZoo::PhaseSpaceFileWriter::writeBinaryParticle (
    ByteBuffer & buffer,
    Particle & particle ) [inline], [protected], [virtual]
```

Write a particle in binary format to a byte buffer.

Must be implemented by derived classes that support binary format. The default implementation throws an exception.

Parameters

<i>buffer</i>	The byte buffer to write particle data into
<i>particle</i>	The particle object to write

Exceptions

<i>std::runtime_error</i>	if not implemented for binary format
---------------------------	--------------------------------------

Reimplemented in [ParticleZoo::EGSphspFile::Writer](#), [ParticleZoo::IAEAphspFile::Writer](#), and [ParticleZoo::TOPASphspFile::Writer](#).

7.15.3.50 writeHeaderData()

```
virtual void ParticleZoo::PhaseSpaceFileWriter::writeHeaderData (
    ByteBuffer & buffer ) [protected], [pure virtual]
```

Write header data to a byte buffer.

This is a pure virtual method that must be implemented by derived classes to write format-specific header information.

Parameters

<i>buffer</i>	The byte buffer to write header data into
---------------	---

Implemented in [ParticleZoo::EGSphspFile::Writer](#), [ParticleZoo::IAEphspFile::Writer](#), [ParticleZoo::penEasyphspFile::Writer](#), [ParticleZoo::ROOT::Writer](#), and [ParticleZoo::TOPASphspFile::Writer](#).

7.15.3.51 writeParticle()

```
void ParticleZoo::PhaseSpaceFileWriter::writeParticle (
    Particle particle ) [virtual]
```

Write a particle to the phase space file.

Writes the given particle to the file using the appropriate format. For binary or ASCII files, the particle is automatically buffered and written when the buffer is full. Applies any constant values that have been set before writing.

Parameters

<i>particle</i>	The particle object to write to the file
-----------------	--

7.15.3.52 writeParticleManually()

```
void ParticleZoo::PhaseSpaceFileWriter::writeParticleManually (
    Particle & particle ) [inline], [protected], [virtual]
```

Write a particle manually (for formats requiring third-party I/O).

Can be implemented by derived classes to support manual file I/O, circumventing the internal file stream and buffer.

Must be implemented by derived classes that specify [FormatType::NONE](#). The default implementation throws an exception.

Parameters

<i>particle</i>	The particle object to write manually
-----------------	---------------------------------------

Exceptions

<i>std::runtime_error</i>	if not implemented
---------------------------	--------------------

Reimplemented in [ParticleZoo::ROOT::Writer](#).

The documentation for this class was generated from the following files:

7.15 ParticleZoo::PhaseSpaceFileWriter Class Reference

- `include/particlezoo/PhaseSpaceFileWriter.h`
- `src/PhaseSpaceFileWriter.cc`

7.16 ParticleZoo::ROOT::BranchInfo Struct Reference

Configuration for ROOT TTree branch mapping.

```
#include <particlezoo/ROOT/ROOTphsp.h>
```

Public Attributes

- `std::string branchName`
Name of the ROOT TTree branch.
- `double unitFactor`
Unit conversion factor to internal units.

7.16.1 Detailed Description

Configuration for ROOT TTree branch mapping.

Contains the branch name and unit conversion factor for mapping physical quantities to ROOT TTree branches.

The documentation for this struct was generated from the following file:

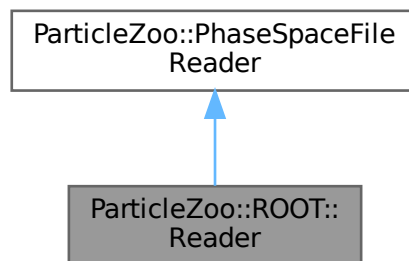
- `include/particlezoo/ROOT/ROOTphsp.h`

7.17 ParticleZoo::ROOT::Reader Class Reference

ROOT format phase space file reader.

```
#include <particlezoo/ROOT/ROOTphsp.h>
```

Inheritance diagram for ParticleZoo::ROOT::Reader:



Public Member Functions

- [Reader](#) (const std::string &fileName, const UserOptions &options=UserOptions{})
Construct a ROOT file reader with user options.
- [~Reader](#) () override
Destructor - closes ROOT file and cleans up resources.
- std::uint64_t [getNumberOfOriginalHistories](#) () const override
Get number of original Monte Carlo histories.
- std::uint64_t [getNumberOfParticles](#) () const override
Get total number of particles in the ROOT file.
- bool [hasNativeIncrementalHistoryCounters](#) () const override
Check if this format stores incremental history numbers per-particle.

Public Member Functions inherited from [ParticleZoo::PhaseSpaceFileReader](#)

- [PhaseSpaceFileReader](#) (const std::string &phspFormat, const std::string &fileName, const UserOptions &userOptions, [FormatType](#) formatType=[FormatType::BINARY](#), const [FixedValues](#) fixedValues=[FixedValues\(\)](#), unsigned int bufferSize=[DEFAULT_BUFFER_SIZE](#))
Construct a new Phase Space File Reader object.
- virtual [~PhaseSpaceFileReader](#) ()
Destroy the Phase Space File Reader object.
- void [close](#) ()
Close the phase space file and clean up resources.
- float [getConstantPx](#) () const
Get the constant X-component of the direction unit vector (if constant).
- float [getConstantPy](#) () const
Get the constant Y-component of the direction unit vector (if constant).
- float [getConstantPz](#) () const
Get the constant Z-component of the direction unit vector (if constant).
- float [getConstantWeight](#) () const
Get the constant statistical weight value (if constant).
- float [getConstantX](#) () const
Get the constant X coordinate value (if constant).
- float [getConstantY](#) () const
Get the constant Y coordinate value (if constant).
- float [getConstantZ](#) () const
Get the constant Z coordinate value (if constant).
- const std::string [getFileName](#) () const
Get the filename of the phase space file being read.
- std::uint64_t [getFileSize](#) () const
Get the size of the phase space file in bytes.
- const [FixedValues](#) [getFixedValues](#) () const
Get the fixed values configuration.
- virtual std::uint64_t [getHistoriesRead](#) ()
Get the number of Monte Carlo histories that have been read so far. If the end of the file has been reached, this will return the total number of original histories unless more histories than expected have already been read - in which case it returns the actual count.
- [Particle](#) [getNextParticle](#) ()
Get the next particle from the phase space file.
- virtual std::uint64_t [getNumberOfRepresentedHistories](#) () const
Get the number of histories actually represented in this phase space.
- virtual std::uint64_t [getParticlesRead](#) ()
Get the number of particles that have been read so far.
- const std::string [getPHSPFormat](#) () const
Get the phase space file format identifier.

7.17 ParticleZoo::ROOT::Reader Class Reference

- virtual bool [hasMoreParticles](#) ()
Check if there are more particles to read in the file.
- virtual bool [hasNativeRepresentedHistoryCount](#) () const
Check if this format can provide the number of represented histories without scanning the file.
- bool [isPxConstant](#) () const
Check if the X-component of momentum is constant for all particles.
- bool [isPyConstant](#) () const
Check if the Y-component of momentum is constant for all particles.
- bool [isPzConstant](#) () const
Check if the Z-component of momentum is constant for all particles.
- bool [isWeightConstant](#) () const
Check if the statistical weight is constant for all particles.
- bool [isXConstant](#) () const
Check if the X coordinate is constant for all particles.
- bool [isYConstant](#) () const
Check if the Y coordinate is constant for all particles.
- bool [isZConstant](#) () const
Check if the Z coordinate is constant for all particles.
- void [moveToParticle](#) (std::uint64_t particleIndex)
Move the file position to a specific particle index.
- [Particle peekNextParticle](#) ()
Peek at the next particle without advancing the file position.
- void [setCommentMarkers](#) (const std::vector< std::string > &commentMarkers)
Set comment markers for ASCII format files.

Static Public Member Functions

- static std::vector< CLICommand > [getFormatSpecificCLICommands](#) ()
Get format-specific CLI commands for ROOT configuration.

Static Public Member Functions inherited from [ParticleZoo::PhaseSpaceFileReader](#)

- static std::vector< CLICommand > [getCLICommands](#) ()
Get command line interface commands supported by this reader.

Protected Member Functions

- [Particle peekParticleManually](#) () override
Peek at the next particle from ROOT TTree without advancing the file position.
- [Particle readParticleManually](#) () override
Read next particle from ROOT TTree.

Protected Member Functions inherited from **ParticleZoo::PhaseSpaceFileReader**

- double [calcThirdUnitComponent](#) (double &u, double &v) const
Calculate the third component of a unit vector from two components (double precision).
- float [calcThirdUnitComponent](#) (float &u, float &v) const
Calculate the third component of a unit vector from two components (float precision).
- const [ByteBuffer](#) [getHeaderData](#) ()
Get the file header data as a byte buffer.
- const [ByteBuffer](#) [getHeaderData](#) (std::size_t headerSize)
Get a specific amount of header data as a byte buffer.
- virtual std::size_t [getMaximumASCIILineLength](#) () const
Get the maximum line length for ASCII format files.
- [Particle](#) [getNextParticle](#) (bool countParticleInStatistics)
Get the next particle with optional statistics counting control.
- std::size_t [getNumberOfEntriesInFile](#) () const
Get the number of particle records that fit in the file.
- virtual std::size_t [getParticleRecordLength](#) () const
Get the length in bytes of each particle record.
- virtual std::size_t [getParticleRecordStartOffset](#) () const
Get the byte offset where particle records start in the file.
- virtual std::uint64_t [getParticlesRead](#) (bool includeSkippedParticles)
Get the number of particles read with optional inclusion of skipped particles (including pseudo-particles).
- const UserOptions & [getUserOptions](#) () const
Get the user options that were passed to the constructor.
- virtual [Particle](#) [readASCIIParticle](#) (const std::string &line)
Read a particle from ASCII data.
- virtual [Particle](#) [readBinaryParticle](#) ([ByteBuffer](#) &buffer)
Read a particle from binary data.
- void [setByteOrder](#) ([ByteOrder](#) byteOrder)
Set the byte order for binary data interpretation.
- void [setConstantPx](#) (float Px)
Set a constant X-component of the direction unit vector for all particles.
- void [setConstantPy](#) (float Py)
Set a constant Y-component of the direction unit vector for all particles.
- void [setConstantPz](#) (float Pz)
Set a constant Z-component of the direction unit vector for all particles.
- void [setConstantWeight](#) (float weight)
Set a constant statistical weight for all particles.
- void [setConstantX](#) (float X)
Set a constant X coordinate value for all particles.
- void [setConstantY](#) (float Y)
Set a constant Y coordinate value for all particles.
- void [setConstantZ](#) (float Z)
Set a constant Z coordinate value for all particles.

7.17 ParticleZoo::ROOT::Reader Class Reference

7.17.1 Detailed Description

ROOT format phase space file reader.

Reads particle data from ROOT TTree structures with configurable branch mappings. Supports multiple format presets (TOPAS, OpenGATE) and custom branch configurations.

7.17.2 Constructor & Destructor Documentation

7.17.2.1 Reader()

```
ParticleZoo::ROOT::Reader::Reader (
    const std::string & fileName,
    const UserOptions & options = UserOptions{} )
```

Construct a ROOT file reader with user options.

Parameters

<i>fileName</i>	Path to the ROOT file
<i>options</i>	User configuration options including branch mappings

Exceptions

<i>std::runtime_error</i>	If file cannot be opened or required branches missing
---------------------------	---

7.17.3 Member Function Documentation

7.17.3.1 getFormatSpecificCLICommands()

```
std::vector< CLICommand > ParticleZoo::ROOT::Reader::getFormatSpecificCLICommands ( ) [static]
```

Get format-specific CLI commands for ROOT configuration.

Returns

Vector of ROOT-specific CLI commands

7.17.3.2 `getNumberOfOriginalHistories()`

```
std::uint64_t ParticleZoo::ROOT::Reader::getNumberOfOriginalHistories ( ) const [inline], [override], [virtual]
```

Get number of original Monte Carlo histories.

Returns

Number of original histories

Implements [ParticleZoo::PhaseSpaceFileReader](#).

7.17.3.3 `getNumberOfParticles()`

```
std::uint64_t ParticleZoo::ROOT::Reader::getNumberOfParticles ( ) const [inline], [override], [virtual]
```

Get total number of particles in the ROOT file.

Returns

Number of particles (TTree entries)

Implements [ParticleZoo::PhaseSpaceFileReader](#).

7.17.3.4 `hasNativeIncrementalHistoryCounters()`

```
bool ParticleZoo::ROOT::Reader::hasNativeIncrementalHistoryCounters ( ) const [inline], [override], [virtual]
```

Check if this format stores incremental history numbers per-particle.

Returns

true if the TTree has a history number branch

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

7.17 ParticleZoo::ROOT::Reader Class Reference

7.17.3.5 peekParticleManually()

`Particle` ParticleZoo::ROOT::Reader::peekParticleManually () [override], [protected], [virtual]

Peek at the next particle from ROOT TTree without advancing the file position.

Reads the next particle but does not move the internal file pointer forward. This allows for inspecting the upcoming particle without consuming it.

Returns

`Particle` object with data from next TTree entry

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

7.17.3.6 readParticleManually()

`Particle` ParticleZoo::ROOT::Reader::readParticleManually () [override], [protected], [virtual]

Read next particle from ROOT TTree.

Returns

`Particle` object with data from current TTree entry

Exceptions

<code>std::runtime_error</code>	If end of file reached or read error
---------------------------------	--------------------------------------

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

The documentation for this class was generated from the following files:

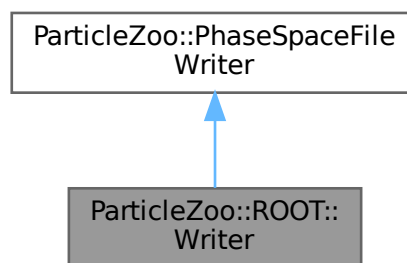
- include/particlezoo/ROOT/ROOTphsp.h
- src/ROOT/ROOTphsp.cc

7.18 ParticleZoo::ROOT::Writer Class Reference

ROOT format phase space file writer.

```
#include <particlezoo/ROOT/ROOTphsp.h>
```

Inheritance diagram for ParticleZoo::ROOT::Writer:



Public Member Functions

- [Writer](#) (const std::string &fileName, const UserOptions &options=UserOptions{})
Construct a ROOT file writer with user options.
- [~Writer](#) () override
Destructor - writes TTree and closes ROOT file.
- std::uint64_t [getMaximumSupportedParticles](#) () const override
Get maximum number of particles that can be stored.

Public Member Functions inherited from [ParticleZoo::PhaseSpaceFileWriter](#)

- [PhaseSpaceFileWriter](#) (const std::string &phspFormat, const std::string &fileName, const UserOptions &userOptions, [FormatType](#) formatType=[FormatType::BINARY](#), const [FixedValues](#) fixedValues=[FixedValues\(\)](#), unsigned int bufferSize=[DEFAULT_BUFFER_SIZE](#))
Construct a new Phase Space File Writer object.
- virtual [~PhaseSpaceFileWriter](#) ()
Destroy the Phase Space File Writer object.
- void [addAdditionalHistories](#) (std::uint64_t additionalHistories)

7.18 ParticleZoo::ROOT::Writer Class Reference

- Add additional Monte Carlo histories to the count.*
- void `close ()`
Close the phase space file and finalize writing.
- `ByteOrder getByteOrder () const`
Get the byte order used for binary data writing.
- float `getConstantPx () const`
Get the constant X-component of the direction unit vector (if constant).
- float `getConstantPy () const`
Get the constant Y-component of the direction unit vector (if constant).
- float `getConstantPz () const`
Get the constant Z-component of the direction unit vector (if constant).
- float `getConstantWeight () const`
Get the constant statistical weight value (if constant).
- float `getConstantX () const`
Get the constant X coordinate value (if constant).
- float `getConstantY () const`
Get the constant Y coordinate value (if constant).
- float `getConstantZ () const`
Get the constant Z coordinate value (if constant).
- const std::string `getFileName () const`
Get the filename where the phase space file is being written.
- const `FixedValues getFixedValues () const`
Get the fixed values configuration.
- virtual std::uint64_t `getHistoriesWritten () const`
Get the number of Monte Carlo histories that have been written.
- std::uint64_t `getParticlesWritten () const`
Get the number of particles that have been written to the file.
- const std::string `getPHSPFormat () const`
Get the phase space file format identifier.
- bool `isPxConstant () const`
Check if the X-component of the direction unit vector is set to a constant value for all particles.
- bool `isPyConstant () const`
Check if the Y-component of the direction unit vector is set to a constant value for all particles.
- bool `isPzConstant () const`
Check if the Z-component of the direction unit vector is set to a constant value for all particles.
- bool `isWeightConstant () const`
Check if the statistical weight is set to a constant value for all particles.
- bool `isXConstant () const`
Check if the X coordinate is set to a constant value for all particles.
- bool `isYConstant () const`
Check if the Y coordinate is set to a constant value for all particles.

- bool [isZConstant](#) () const
Check if the Z coordinate is set to a constant value for all particles.
- virtual void [writeParticle](#) ([Particle](#) particle)
Write a particle to the phase space file.

Static Public Member Functions

- static std::vector< CLICommand > [getFormatSpecificCLICommands](#) ()
Get format-specific CLI commands for ROOT configuration.

Static Public Member Functions inherited from [ParticleZoo::PhaseSpaceFileWriter](#)

- static std::vector< CLICommand > [getCLICommands](#) ()
Get command line interface commands supported by this writer.

Protected Member Functions

- void [writeHeaderData](#) ([ByteBuffer](#) &buffer) override
Write header data (not used for ROOT format).
- void [writeParticleManually](#) ([Particle](#) &particle) override
Write particle data to ROOT TTree.

Protected Member Functions inherited from [ParticleZoo::PhaseSpaceFileWriter](#)

- virtual bool [accountForAdditionalHistories](#) (std::uint64_t additionalHistories)
Handle accounting for simulation histories that produced no particles.
- virtual bool [canHaveConstantPx](#) () const
Check if this format supports constant X-component of the direction unit vector.
- virtual bool [canHaveConstantPy](#) () const
Check if this format supports constant Y-component of the direction unit vector.
- virtual bool [canHaveConstantPz](#) () const
Check if this format supports constant Z-component of the direction unit vector.
- virtual bool [canHaveConstantWeight](#) () const
Check if this format supports constant statistical weights.
- virtual bool [canHaveConstantX](#) () const
Check if this format supports constant X coordinates.
- virtual bool [canHaveConstantY](#) () const
Check if this format supports constant Y coordinates.
- virtual bool [canHaveConstantZ](#) () const

7.18 ParticleZoo::ROOT::Writer Class Reference

- Check if this format supports constant Z coordinates.*
- virtual bool `canWritePseudoParticlesExplicitly ()` const
Check if this format can write pseudo-particles explicitly.
- virtual void `fixedValuesHaveChanged ()`
Called when fixed values have been changed.
- virtual size_t `getMaximumASCIILineLength ()` const
Get the maximum line length for ASCII format files.
- virtual std::size_t `getParticleRecordLength ()` const
Get the length in bytes of each particle record.
- virtual std::size_t `getParticleRecordStartOffset ()` const
Get the byte offset where particle records start in the file.
- virtual std::uint64_t `getPendingHistories ()` const
Get the number of pending histories to account for.
- const UserOptions & `getUserOptions ()` const
Get the user options that were passed to the constructor.
- void `setByteOrder (ByteOrder byteOrder)`
Set the byte order for binary data writing.
- void `setConstantPx (float Px)`
Set a constant X-component of the direction unit vector for all particles.
- void `setConstantPy (float Py)`
Set a constant Y-component of the direction unit vector for all particles.
- void `setConstantPz (float Pz)`
Set a constant Z-component of the direction unit vector for all particles.
- void `setConstantWeight (float weight)`
Set a constant statistical weight for all particles.
- void `setConstantX (float X)`
Set a constant X coordinate value for all particles.
- void `setConstantY (float Y)`
Set a constant Y coordinate value for all particles.
- void `setConstantZ (float Z)`
Set a constant Z coordinate value for all particles.
- virtual const std::string `writeASCIIParticle (Particle &particle)`
Write a particle in ASCII format as a string.
- virtual void `writeBinaryParticle (ByteBuffer &buffer, Particle &particle)`
Write a particle in binary format to a byte buffer.

7.18.1 Detailed Description

ROOT format phase space file writer.

Writes particle data to ROOT TTree structures with configurable branch mappings. Supports multiple format presets (TOPAS, OpenGATE) and custom branch configurations.

7.18.2 Constructor & Destructor Documentation

7.18.2.1 Writer()

```
ParticleZoo::ROOT::Writer::Writer (
    const std::string & fileName,
    const UserOptions & options = UserOptions{} )
```

Construct a ROOT file writer with user options.

Parameters

<i>fileName</i>	Path for the output ROOT file
<i>options</i>	User configuration options including branch mappings

Exceptions

<i>std::runtime_error</i>	If file cannot be created or TTree setup fails
---------------------------	--

7.18.3 Member Function Documentation

7.18.3.1 getFormatSpecificCLICommands()

```
std::vector< CLICommand > ParticleZoo::ROOT::Writer::getFormatSpecificCLICommands ( ) [static]
```

Get format-specific CLI commands for ROOT configuration.

Returns

Vector of ROOT-specific CLI commands

7.18.3.2 getMaximumSupportedParticles()

```
std::uint64_t ParticleZoo::ROOT::Writer::getMaximumSupportedParticles ( ) const [inline], [override],
[virtual]
```

Get maximum number of particles that can be stored.

Returns

Maximum supported particles

Implements [ParticleZoo::PhaseSpaceFileWriter](#).

7.18 ParticleZoo::ROOT::Writer Class Reference

7.18.3.3 writeHeaderData()

```
void ParticleZoo::ROOT::Writer::writeHeaderData (  
    ByteBuffer & buffer ) [inline], [override], [protected], [virtual]
```

Write header data (not used for ROOT format).

Parameters

<i>buffer</i>	Unused buffer parameter
---------------	-------------------------

Implements [ParticleZoo::PhaseSpaceFileWriter](#).

7.18.3.4 writeParticleManually()

```
void ParticleZoo::ROOT::Writer::writeParticleManually (  
    Particle & particle ) [override], [protected], [virtual]
```

Write particle data to ROOT TTree.

Parameters

<i>particle</i>	Particle object to write to current TTree entry
-----------------	---

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

The documentation for this class was generated from the following files:

- include/particlezoo/ROOT/ROOTphsp.h
- src/ROOT/ROOTphsp.cc

7.19 ParticleZoo::SupportedFormat Struct Reference

Structure describing a supported phase space file format.

```
#include <particlezoo/utilities/formats.h>
```

Public Attributes

- const std::string **description**
Human-readable description of the format.
- const std::string **fileExtension**
Standard file extension (e.g., ".egsphsp", ".IAEAphsp")
- const bool **fileExtensionCanHaveSuffix** {false}
True if extension can have numeric suffixes (e.g., ".egsphsp1")
- const std::string **name**
Format name (e.g., "IAEA", "EGS", "TOPAS")

7.19.1 Detailed Description

Structure describing a supported phase space file format.

Contains metadata about a file format including its name, description, file extension, and whether the extension can have numeric suffixes.

The documentation for this struct was generated from the following file:

- include/particlezoo/utilities/formats.h

7.20 ParticleZoo::TOPASphspFile::Header Class Reference

[Header](#) for TOPAS phase space files.

```
#include <particlezoo/TOPAS/TOPASHeader.h>
```

Classes

- struct [DataColumn](#)
Column definition for TOPAS phase space files.
- struct [ParticleStats](#)
Statistics tracking for individual particle types for TOPAS phase space files.

Public Types

- enum class [ColumnType](#) {
POSITION_X , POSITION_Y , POSITION_Z , DIRECTION_COSINE_X ,
DIRECTION_COSINE_Y , ENERGY , WEIGHT , PARTICLE_TYPE ,
DIRECTION_COSINE_Z_SIGN , NEW_HISTORY_FLAG , TOPAS_TIME , TIME_OF_FLIGHT ,
RUN_ID , EVENT_ID , TRACK_ID , PARENT_ID ,
CHARGE , CREATOR_PROCESS , INITIAL_KINETIC_ENERGY , VERTEX_POSITION_X ,
VERTEX_POSITION_Y , VERTEX_POSITION_Z , INITIAL_DIRECTION_COSINE_X , INITIAL_DIRECTION_COSINE_Y
,
INITIAL_DIRECTION_COSINE_Z , SEED_PART_1 , SEED_PART_2 , SEED_PART_3 ,
SEED_PART_4 }
Column types supported in TOPAS phase space files.
- enum class [DataType](#) {
STRING , BOOLEAN , INT8 , INT32 ,
FLOAT32 , FLOAT64 }
Data types supported in TOPAS columns.
- using **ParticleStatsTable** = std::unordered_map< [ParticleType](#), [ParticleStats](#) >
Type alias for particle statistics table.

Public Member Functions

- [Header](#) (const std::string &fileName)
Construct header by reading from existing TOPAS file.
- [Header](#) (const std::string &fileName, TOPASFormat formatType)
Construct header for writing new TOPAS file.
- void [addColumnType](#) ([ColumnType](#) columnType)
Add a new column type to the phase space format.

- void `countParticleStats` (const `Particle` &particle)
Update particle statistics with a new particle.
- const std::vector< `DataColumn` > & `getColumnTypes` () const
Get the column definitions for this phase space.
- double `getMaxKineticEnergyOfType` (`ParticleType` type) const
Get the maximum kinetic energy for particles of a specific type.
- double `getMinKineticEnergyOfType` (`ParticleType` type) const
Get the minimum kinetic energy for particles of a specific type.
- std::uint64_t `getNumberOfOriginalHistories` () const
Get the number of original simulation histories.
- std::uint64_t `getNumberOfParticles` () const
Get the total number of particles in the phase space.
- std::uint64_t `getNumberOfParticlesOfType` (`ParticleType` type) const
Get the number of particles of a specific type.
- std::uint64_t `getNumberOfRepresentedHistories` () const
Get the number of histories explicitly represented by particles in the phase space.
- std::size_t `getRecordLength` () const
Get the length of each particle record in bytes.
- TOPASFormat `getTOPASFormat` () const
Get the TOPAS format type.
- std::string `getTOPASFormatName` () const
Get the human-readable format name.
- void `setNumberOfOriginalHistories` (std::uint64_t originalHistories)
Set the number of original simulation histories.
- void `writeHeader` ()
Write the complete header file.

Static Public Member Functions

- static std::string `getTOPASFormatName` (TOPASFormat format)
Get format name from enum value.

7.20.1 Detailed Description

[Header](#) for TOPAS phase space files.

This class handles reading, writing, and managing header information for TOPAS format phase space files. It manages file metadata, particle statistics, column definitions, and format-specific configurations. TOPAS files use separate .header and .phsp files.

7.20.2 Member Enumeration Documentation

7.20.2.1 ColumnType

```
enum class ParticleZoo::TOPASphspFile::Header::ColumnType [strong]
```

Column types supported in TOPAS phase space files.

Defines all possible column types that can appear in TOPAS files, from basic particle properties to extended tracking information.

Enumerator

POSITION_X	X coordinate position.
POSITION_Y	Y coordinate position.
POSITION_Z	Z coordinate position.
DIRECTION_COSINE_X	X direction cosine.
DIRECTION_COSINE_Y	Y direction cosine.
ENERGY	Kinetic energy.
WEIGHT	Particle statistical weight.
PARTICLE_TYPE	PDG particle type code.
DIRECTION_COSINE_Z_SIGN	Sign of Z direction cosine.
NEW_HISTORY_FLAG	First particle from history flag.
TOPAS_TIME	TOPAS simulation time.
TIME_OF_FLIGHT	Particle time of flight.
RUN_ID	Simulation run identifier.
EVENT_ID	Event identifier within run.
TRACK_ID	Track identifier within event.
PARENT_ID	Parent track identifier.
CHARGE	Particle charge.
CREATOR_PROCESS	Physics process that created particle.
INITIAL_KINETIC_ENERGY	Initial kinetic energy at creation.
VERTEX_POSITION_X	X coordinate of creation vertex.
VERTEX_POSITION_Y	Y coordinate of creation vertex.
VERTEX_POSITION_Z	Z coordinate of creation vertex.
INITIAL_DIRECTION_COSINE_X	Initial X direction cosine.
INITIAL_DIRECTION_COSINE_Y	Initial Y direction cosine.
INITIAL_DIRECTION_COSINE_Z	Initial Z direction cosine.
SEED_PART_1	Random seed component 1.
SEED_PART_2	Random seed component 2.
SEED_PART_3	Random seed component 3.
SEED_PART_4	Random seed component 4.

7.20.2.2 DataType

```
enum class ParticleZoo::TOPASphspFile::Header::DataType [strong]
```

Data types supported in TOPAS columns.

Defines the fundamental data types that can be stored in TOPAS phase space file columns.

Enumerator

STRING	Variable-length string data.
BOOLEAN	Boolean true/false values.
INT8	8-bit signed integer
INT32	32-bit signed integer
FLOAT32	32-bit floating-point
FLOAT64	64-bit floating-point

7.20.3 Constructor & Destructor Documentation

7.20.3.1 Header() [1/2]

```
ParticleZoo::TOPASphspFile::Header::Header (  
    const std::string & fileName )
```

Construct header by reading from existing TOPAS file.

Parameters

<i>fileName</i>	Path to TOPAS phase space file (.phsp or .header)
-----------------	---

Exceptions

<i>std::runtime_error</i>	if file cannot be read or is invalid
---------------------------	--------------------------------------

7.20.3.2 Header() [2/2]

```
ParticleZoo::TOPASphspFile::Header::Header (  
    const std::string & fileName,  
    TOPASFormat formatType )
```


7.20 ParticleZoo::TOPASphspFile::Header Class Reference

Construct header for writing new TOPAS file.

Parameters

<i>fileName</i>	Path for new TOPAS phase space file
<i>formatType</i>	Format to write (ASCII, BINARY, or LIMITED)

7.20.4 Member Function Documentation

7.20.4.1 addColumnType()

```
void ParticleZoo::TOPASphspFile::Header::addColumnType (
    ColumnType columnType )
```

Add a new column type to the phase space format.

Parameters

<i>columnType</i>	Type of column to add
-------------------	-----------------------

7.20.4.2 countParticleStats()

```
void ParticleZoo::TOPASphspFile::Header::countParticleStats (
    const Particle & particle ) [inline]
```

Update particle statistics with a new particle.

Parameters

<i>particle</i>	Particle to include in statistics calculations
-----------------	--

7.20.4.3 getColumnTypes()

```
const std::vector< Header::DataColumn > & ParticleZoo::TOPASphspFile::Header::getColumnTypes ( )
const [inline]
```

Get the column definitions for this phase space.

Returns

Vector of [DataColumn](#) objects describing the file structure

7.20.4.4 getMaxKineticEnergyOfType()

```
double ParticleZoo::TOPASphspFile::Header::getMaxKineticEnergyOfType (
    ParticleType type ) const [inline]
```

Get the maximum kinetic energy for particles of a specific type.

Parameters

<i>type</i>	Particle type to query
-------------	--

Returns

Maximum kinetic energy for the particle type

7.20.4.5 getMinKineticEnergyOfType()

```
double ParticleZoo::TOPASphspFile::Header::getMinKineticEnergyOfType (
    ParticleType type ) const [inline]
```

Get the minimum kinetic energy for particles of a specific type.

Parameters

<i>type</i>	Particle type to query
-------------	--

Returns

Minimum kinetic energy for the particle type

7.20.4.6 getNumberOfOriginalHistories()

```
std::uint64_t ParticleZoo::TOPASphspFile::Header::getNumberOfOriginalHistories ( ) const [inline]
```

Get the number of original simulation histories.

7.20 ParticleZoo::TOPASphspFile::Header Class Reference

Returns

Count of primary histories used in the simulation

7.20.4.7 getNumberOfParticles()

```
std::uint64_t ParticleZoo::TOPASphspFile::Header::getNumberOfParticles ( ) const [inline]
```

Get the total number of particles in the phase space.

Returns

Total particle count across all types

7.20.4.8 getNumberOfParticlesOfType()

```
std::uint64_t ParticleZoo::TOPASphspFile::Header::getNumberOfParticlesOfType (
    ParticleType type ) const [inline]
```

Get the number of particles of a specific type.

Parameters

<i>type</i>	Particle type to query
-------------	------------------------

Returns

Number of particles of the specified type

7.20.4.9 getNumberOfRepresentedHistories()

```
std::uint64_t ParticleZoo::TOPASphspFile::Header::getNumberOfRepresentedHistories ( ) const [inline]
```

Get the number of histories explicitly represented by particles in the phase space.

Returns

Count of histories that produced particles that reached the phase space

7.20.4.10 getRecordLength()

```
std::size_t ParticleZoo::TOPASphspFile::Header::getRecordLength ( ) const
```

Get the length of each particle record in bytes.

Returns

Record length based on format and column configuration

7.20.4.11 getTOPASFormat()

```
TOPASFormat ParticleZoo::TOPASphspFile::Header::getTOPASFormat ( ) const [inline]
```

Get the TOPAS format type.

Returns

TOPASFormat enum value

7.20.4.12 getTOPASFormatName() [1/2]

```
std::string ParticleZoo::TOPASphspFile::Header::getTOPASFormatName ( ) const [inline]
```

Get the human-readable format name.

Returns

Format name as string (e.g., "TOPAS BINARY")

7.20.4.13 getTOPASFormatName() [2/2]

```
std::string ParticleZoo::TOPASphspFile::Header::getTOPASFormatName (
    TOPASFormat format ) [inline], [static]
```

Get format name from enum value.

7.20 ParticleZoo::TOPASphspFile::Header Class Reference

Parameters

<i>format</i>	TOPAS format type
---------------	-------------------

Returns

Human-readable format name

7.20.4.14 setNumberOfOriginalHistories()

```
void ParticleZoo::TOPASphspFile::Header::setNumberOfOriginalHistories (
    std::uint64_t originalHistories ) [inline]
```

Set the number of original simulation histories.

Parameters

<i>originalHistories</i>	Count of primary histories
--------------------------	----------------------------

7.20.4.15 writeHeader()

```
void ParticleZoo::TOPASphspFile::Header::writeHeader ( )
```

Write the complete header file.

Writes the header information to the .header file with format-specific structure and particle statistics.

Exceptions

<i>std::runtime_error</i>	if file cannot be written
---------------------------	---------------------------

The documentation for this class was generated from the following files:

- include/particlezoo/TOPAS/TOPASHeader.h
- src/topas/TOPASHeader.cc

7.21 ParticleZoo::TOPASphspFile::Header::DataColumn Struct Reference

Column definition for TOPAS phase space files.

```
#include <particlezoo/TOPAS/TOPASHeader.h>
```

Public Member Functions

- [DataColumn](#) ([ColumnType](#) columnType)
Construct column from type (uses default name)
- [DataColumn](#) ([ColumnType](#) columnType, [DataType](#) valueType)
Construct column with specific data type.
- [DataColumn](#) ([ColumnType](#) columnType, [DataType](#) valueType, std::string_view name)
Construct column with all parameters specified.
- [DataColumn](#) (std::string_view name)
Construct column from name string.
- std::size_t [sizeOf](#) () const
Get the storage size of this column's data type.

Static Public Member Functions

- static constexpr std::string_view [getColumnName](#) ([ColumnType](#) columnType)
Get the string represented name for a column type.
- static constexpr [ColumnType](#) [getColumnType](#) (std::string_view name)
Parse column type from name.
- static constexpr [DataType](#) [getDataType](#) ([ColumnType](#) columnType)
Get the default data type for a column type.

Public Attributes

- [ColumnType](#) **columnType_**
Semantic type of the column.
- std::string **name_**
Human-readable column name.
- [DataType](#) **valueType_**
Data storage type.

7.21.1 Detailed Description

Column definition for TOPAS phase space files.

Describes a single column in the phase space file including its type, data format, and display name.

7.21.2 Constructor & Destructor Documentation

7.21.2.1 DataColumn() [1/4]

```
ParticleZoo::TOPASphspFile::Header::DataColumn::DataColumn (
    std::string_view name ) [inline]
```

Construct column from name string.

Parameters

<i>name</i>	Column name (determines type automatically)
-------------	---

7.21.2.2 DataColumn() [2/4]

```
ParticleZoo::TOPASphspFile::Header::DataColumn::DataColumn (
    ColumnType columnType ) [inline]
```

Construct column from type (uses default name)

Parameters

<i>columnType</i>	Column type
-------------------	-------------

7.21.2.3 DataColumn() [3/4]

```
ParticleZoo::TOPASphspFile::Header::DataColumn::DataColumn (
    ColumnType columnType,
    DataType valueType ) [inline]
```

Construct column with specific data type.

Parameters

<i>columnType</i>	Column type
<i>valueType</i>	Data storage type (overrides default)

7.21.2.4 DataColumn() [4/4]

```
ParticleZoo::TOPASphspFile::Header::DataColumn::DataColumn (
    ColumnType columnType,
    DataType valueType,
    std::string_view name ) [inline]
```

Construct column with all parameters specified.

Parameters

<i>columnType</i>	Column type
<i>valueType</i>	Data storage type
<i>name</i>	Custom column name

7.21.3 Member Function Documentation**7.21.3.1 getColumnName()**

```
static constexpr std::string_view ParticleZoo::TOPASphspFile::Header::DataColumn::getColumnName (
    ColumnType columnType ) [inline], [static], [constexpr]
```

Get the string represented name for a column type.

Parameters

<i>columnType</i>	Column type to query
-------------------	----------------------

Returns

Human-readable column name with units

7.21 ParticleZoo::TOPASphspFile::Header::DataColumn Struct Reference

Exceptions

<code>std::runtime_error</code>	if column type is unknown
---------------------------------	---------------------------

7.21.3.2 getColumnType()

```
static constexpr ColumnType ParticleZoo::TOPASphspFile::Header::DataColumn::getColumnType (
    std::string_view name ) [inline], [static], [constexpr]
```

Parse column type from name.

Parameters

<i>name</i>	Column name to parse
-------------	----------------------

Returns

Corresponding ColumnType

Exceptions

<code>std::runtime_error</code>	if name is not recognized
---------------------------------	---------------------------

7.21.3.3 getDataType()

```
static constexpr DataType ParticleZoo::TOPASphspFile::Header::DataColumn::getDataType (
    ColumnType columnType ) [inline], [static], [constexpr]
```

Get the default data type for a column type.

Parameters

<i>columnType</i>	Column type to query
-------------------	----------------------

Returns

Default DataType for storage

Exceptions

<i>std::runtime_error</i>	if column type is unknown
---------------------------	---------------------------

7.21.3.4 sizeof()

```
std::size_t ParticleZoo::TOPASphspFile::Header::DataColumn::sizeof ( ) const [inline]
```

Get the storage size of this column's data type.

Returns

Size in bytes (0 for variable-length strings)

The documentation for this struct was generated from the following file:

- include/particlezoo/TOPAS/TOPASHeader.h

7.22 ParticleZoo::TOPASphspFile::Header::ParticleStats Struct Reference

Statistics tracking for individual particle types for TOPAS phase space files.

```
#include <particlezoo/TOPAS/TOPASHeader.h>
```

Public Attributes

- `std::uint64_t count_ {}`
Number of particles of this type.
- `double maxKineticEnergy_ = 0`
Maximum kinetic energy encountered.
- `double minKineticEnergy_ = std::numeric_limits<double>::max()`
Minimum kinetic energy encountered.

7.22.1 Detailed Description

Statistics tracking for individual particle types for TOPAS phase space files.

The documentation for this struct was generated from the following file:

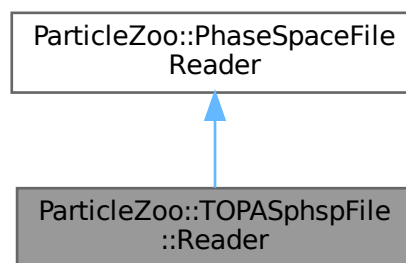
- `include/particlezoo/TOPAS/TOPASHeader.h`

7.23 ParticleZoo::TOPASphspFile::Reader Class Reference

[Reader](#) for TOPAS phase space files.

```
#include <particlezoo/TOPAS/TOPASphspFile.h>
```

Inheritance diagram for ParticleZoo::TOPASphspFile::Reader:



Public Member Functions

- [Reader](#) (const std::string &filename, const UserOptions &options=UserOptions{})
Construct reader for TOPAS phase space file.
- const [Header](#) & [getHeader](#) () const
Get access to the TOPAS header information.
- std::uint64_t [getNumberOfOriginalHistories](#) () const override
Get the number of original simulation histories.
- std::uint64_t [getNumberOfParticles](#) () const override
Get the total number of particles in the phase space.
- std::uint64_t [getNumberOfRepresentedHistories](#) () const override
Get the number of represented histories in the phase space.
- TOPASFormat [getTOPASFormat](#) () const
Get the TOPAS format type of this file.
- bool [hasNativeRepresentedHistoryCount](#) () const override
Check if this format can provide represented history count cheaply.
- void [setDetailedReading](#) (bool enable)
Enable or disable detailed reading of extended particle properties.

Public Member Functions inherited from ParticleZoo::PhaseSpaceFileReader

- **PhaseSpaceFileReader** (const std::string &phspFormat, const std::string &fileName, const UserOptions &userOptions, **FormatType** formatType=**FormatType::BINARY**, const **FixedValues** fixedValues=**FixedValues()**, unsigned int bufferSize=**DEFAULT_BUFFER_SIZE**)
Construct a new Phase Space File Reader object.
- virtual **~PhaseSpaceFileReader** ()
Destroy the Phase Space File Reader object.
- void **close** ()
Close the phase space file and clean up resources.
- float **getConstantPx** () const
Get the constant X-component of the direction unit vector (if constant).
- float **getConstantPy** () const
Get the constant Y-component of the direction unit vector (if constant).
- float **getConstantPz** () const
Get the constant Z-component of the direction unit vector (if constant).
- float **getConstantWeight** () const
Get the constant statistical weight value (if constant).
- float **getConstantX** () const
Get the constant X coordinate value (if constant).
- float **getConstantY** () const
Get the constant Y coordinate value (if constant).
- float **getConstantZ** () const
Get the constant Z coordinate value (if constant).
- const std::string **getFileName** () const
Get the filename of the phase space file being read.
- std::uint64_t **getFileSize** () const
Get the size of the phase space file in bytes.
- const **FixedValues** **getFixedValues** () const
Get the fixed values configuration.
- virtual std::uint64_t **getHistoriesRead** ()
Get the number of Monte Carlo histories that have been read so far. If the end of the file has been reached, this will return the total number of original histories unless more histories than expected have already been read - in which case it returns the actual count.
- **Particle** **getNextParticle** ()
Get the next particle from the phase space file.
- virtual std::uint64_t **getParticlesRead** ()
Get the number of particles that have been read so far.
- const std::string **getPHSPFormat** () const
Get the phase space file format identifier.
- virtual bool **hasMoreParticles** ()
Check if there are more particles to read in the file.

- virtual bool [hasNativeIncrementalHistoryCounters](#) () const
Check if this format directly stores incremental history numbers per-particle.
- bool [isPxConstant](#) () const
Check if the X-component of momentum is constant for all particles.
- bool [isPyConstant](#) () const
Check if the Y-component of momentum is constant for all particles.
- bool [isPzConstant](#) () const
Check if the Z-component of momentum is constant for all particles.
- bool [isWeightConstant](#) () const
Check if the statistical weight is constant for all particles.
- bool [isXConstant](#) () const
Check if the X coordinate is constant for all particles.
- bool [isYConstant](#) () const
Check if the Y coordinate is constant for all particles.
- bool [isZConstant](#) () const
Check if the Z coordinate is constant for all particles.
- void [moveToParticle](#) (std::uint64_t particleIndex)
Move the file position to a specific particle index.
- [Particle peekNextParticle](#) ()
Peek at the next particle without advancing the file position.
- void [setCommentMarkers](#) (const std::vector< std::string > &commentMarkers)
Set comment markers for ASCII format files.

Static Public Member Functions

- static std::vector< CLICommand > [getFormatSpecificCLICommands](#) ()
Get format-specific command-line options.

Static Public Member Functions inherited from [ParticleZoo::PhaseSpaceFileReader](#)

- static std::vector< CLICommand > [getCLICommands](#) ()
Get command line interface commands supported by this reader.

Protected Member Functions

- std::size_t [getMaximumASCIILineLength](#) () const override
Get the maximum length of ASCII particle lines.
- std::size_t [getParticleRecordLength](#) () const override
Get the length of each particle record in bytes.
- [Particle readASCIIParticle](#) (const std::string &line) override
Parse a single ASCII line into a [Particle](#) object.
- [Particle readBinaryParticle](#) (ByteBuffer &buffer) override
Read and decode a single particle from binary data.

Protected Member Functions inherited from ParticleZoo::PhaseSpaceFileReader

- double [calcThirdUnitComponent](#) (double &u, double &v) const
Calculate the third component of a unit vector from two components (double precision).
- float [calcThirdUnitComponent](#) (float &u, float &v) const
Calculate the third component of a unit vector from two components (float precision).
- const [ByteBuffer](#) [getHeaderData](#) ()
Get the file header data as a byte buffer.
- const [ByteBuffer](#) [getHeaderData](#) (std::size_t headerSize)
Get a specific amount of header data as a byte buffer.
- [Particle](#) [getNextParticle](#) (bool countParticleInStatistics)
Get the next particle with optional statistics counting control.
- std::size_t [getNumberOfEntriesInFile](#) () const
Get the number of particle records that fit in the file.
- virtual std::size_t [getParticleRecordStartOffset](#) () const
Get the byte offset where particle records start in the file.
- virtual std::uint64_t [getParticlesRead](#) (bool includeSkippedParticles)
Get the number of particles read with optional inclusion of skipped particles (including pseudo-particles).
- const UserOptions & [getUserOptions](#) () const
Get the user options that were passed to the constructor.
- virtual [Particle](#) [peekParticleManually](#) ()
Peek at a particle manually (for formats requiring third-party I/O).
- virtual [Particle](#) [readParticleManually](#) ()
Read a particle manually (for formats requiring third-party I/O).
- void [setByteOrder](#) ([ByteOrder](#) byteOrder)
Set the byte order for binary data interpretation.
- void [setConstantPx](#) (float Px)
Set a constant X-component of the direction unit vector for all particles.
- void [setConstantPy](#) (float Py)
Set a constant Y-component of the direction unit vector for all particles.
- void [setConstantPz](#) (float Pz)
Set a constant Z-component of the direction unit vector for all particles.
- void [setConstantWeight](#) (float weight)
Set a constant statistical weight for all particles.
- void [setConstantX](#) (float X)
Set a constant X coordinate value for all particles.
- void [setConstantY](#) (float Y)
Set a constant Y coordinate value for all particles.
- void [setConstantZ](#) (float Z)
Set a constant Z coordinate value for all particles.

7.23.1 Detailed Description

[Reader](#) for TOPAS phase space files.

Provides functionality to read phase space data from TOPAS format files, supporting ASCII, BINARY, and LIMITED formats. Handles TOPAS-specific features including pseudo-particles for empty history tracking and extensive metadata columns.

7.23.2 Constructor & Destructor Documentation

7.23.2.1 Reader()

```
ParticleZoo::TOPASphspFile::Reader::Reader (
    const std::string & filename,
    const UserOptions & options = UserOptions{} )
```

Construct reader for TOPAS phase space file.

Automatically detects the format (ASCII, BINARY, or LIMITED) by reading the header file and configures the reader accordingly.

Parameters

<i>filename</i>	Path to the TOPAS phase space file (.phsp or .header)
<i>options</i>	User-specified options for reading behavior

Exceptions

<i>std::runtime_error</i>	if file cannot be opened or format is invalid
---------------------------	---

7.23.3 Member Function Documentation

7.23.3.1 getFormatSpecificCLICommands()

```
std::vector< CLICommand > ParticleZoo::TOPASphspFile::Reader::getFormatSpecificCLICommands ( )
[static]
```

Get format-specific command-line options.

7.23 ParticleZoo::TOPASphspFile::Reader Class Reference

Returns

Vector of CLI commands supported by TOPAS reader (currently empty)

7.23.3.2 getHeader()

```
const Header & ParticleZoo::TOPASphspFile::Reader::getHeader ( ) const [inline]
```

Get access to the TOPAS header information.

Returns

Reference to the header containing file metadata and column definitions

7.23.3.3 getMaximumASCIILineLength()

```
std::size_t ParticleZoo::TOPASphspFile::Reader::getMaximumASCIILineLength ( ) const [inline],  
[override], [protected], [virtual]
```

Get the maximum length of ASCII particle lines.

Returns

Maximum line length (1024 characters for TOPAS format)

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

7.23.3.4 getNumberOfOriginalHistories()

```
std::uint64_t ParticleZoo::TOPASphspFile::Reader::getNumberOfOriginalHistories ( ) const [inline],  
[override], [virtual]
```

Get the number of original simulation histories.

Returns

Count of primary histories used in the simulation

Implements [ParticleZoo::PhaseSpaceFileReader](#).

7.23.3.5 `getNumberOfParticles()`

```
std::uint64_t ParticleZoo::TOPASphspFile::Reader::getNumberOfParticles ( ) const [inline], [override],  
[virtual]
```

Get the total number of particles in the phase space.

Returns

Total particle count as recorded in the header

Implements [ParticleZoo::PhaseSpaceFileReader](#).

7.23.3.6 `getNumberOfRepresentedHistories()`

```
std::uint64_t ParticleZoo::TOPASphspFile::Reader::getNumberOfRepresentedHistories ( ) const [inline],  
[override], [virtual]
```

Get the number of represented histories in the phase space.

Only available for ASCII and BINARY sub-formats (not LIMITED). For LIMITED, delegates to the base class which throws.

Returns

Number of represented histories from the header

Exceptions

<code>std::runtime_error</code>	if format is LIMITED
---------------------------------	----------------------

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

7.23.3.7 `getParticleRecordLength()`

```
std::size_t ParticleZoo::TOPASphspFile::Reader::getParticleRecordLength ( ) const [inline], [override],  
[protected], [virtual]
```

Get the length of each particle record in bytes.

7.23 ParticleZoo::TOPASphspFile::Reader Class Reference

Returns

Record length as defined by the TOPAS header

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

7.23.3.8 getTOPASFormat()

```
TOPASFormat ParticleZoo::TOPASphspFile::Reader::getTOPASFormat ( ) const [inline]
```

Get the TOPAS format type of this file.

Returns

TOPASFormat enum indicating ASCII, BINARY, or LIMITED

7.23.3.9 hasNativeRepresentedHistoryCount()

```
bool ParticleZoo::TOPASphspFile::Reader::hasNativeRepresentedHistoryCount ( ) const [inline],  
[override], [virtual]
```

Check if this format can provide represented history count cheaply.

Returns

true for ASCII and BINARY, false for LIMITED

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

7.23.3.10 readASCIIParticle()

```
Particle ParticleZoo::TOPASphspFile::Reader::readASCIIParticle (  
    const std::string & line ) [override], [protected], [virtual]
```

Parse a single ASCII line into a [Particle](#) object.

Parses TOPAS ASCII format with configurable columns. Supports both core particle data and extended properties based on header column definitions.

Parameters

<i>line</i>	ASCII line containing particle data
-------------	-------------------------------------

Returns

Parsed [Particle](#) object with properties set according to column types

Exceptions

<i>std::runtime_error</i>	if line cannot be parsed or contains invalid data
---------------------------	---

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

7.23.3.11 readBinaryParticle()

```
Particle ParticleZoo::TOPASphspFile::Reader::readBinaryParticle (  
    ByteBuffer & buffer ) [inline], [override], [protected], [virtual]
```

Read and decode a single particle from binary data.

Handles format-specific binary reading including pseudo-particle processing for empty history tracking in BINARY format.

Parameters

<i>buffer</i>	Binary buffer containing particle data
---------------	--

Returns

Decoded [Particle](#) object with all properties

Exceptions

<i>std::runtime_error</i>	if format is unsupported or data is corrupted
---------------------------	---

Reimplemented from [ParticleZoo::PhaseSpaceFileReader](#).

7.23 ParticleZoo::TOPASphspFile::Reader Class Reference

7.23.3.12 setDetailedReading()

```
void ParticleZoo::TOPASphspFile::Reader::setDetailedReading (
    bool enable ) [inline]
```

Enable or disable detailed reading of extended particle properties.

When enabled, reads all additional columns beyond the standard 10 columns. When disabled, only reads the core particle data for improved performance.

Parameters

<i>enable</i>	true to read all columns, false for core data only
---------------	--

The documentation for this class was generated from the following files:

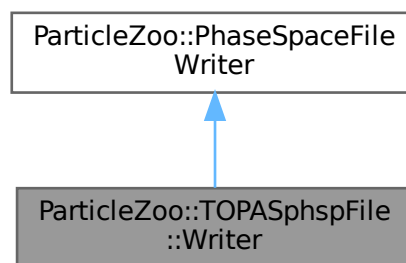
- include/particlezoo/TOPAS/TOPASphspFile.h
- src/topas/TOPASphspFile.cc

7.24 ParticleZoo::TOPASphspFile::Writer Class Reference

[Writer](#) for TOPAS phase space files.

```
#include <particlezoo/TOPAS/TOPASphspFile.h>
```

Inheritance diagram for ParticleZoo::TOPASphspFile::Writer:



Public Member Functions

- [Writer](#) (const std::string &filename, const UserOptions &options=UserOptions{})
Construct writer for TOPAS phase space file.
- [~Writer](#) () override
Destructor - closes the file if it has not been closed already.
- [Header](#) & [getHeader](#) ()
Get access to the TOPAS header for configuration.
- std::uint64_t [getMaximumSupportedParticles](#) () const override
Get the maximum number of particles this format can store.
- TOPASFormat [getTOPASFormat](#) () const
Get the TOPAS format type being written.

Public Member Functions inherited from ParticleZoo::PhaseSpaceFileWriter

- **PhaseSpaceFileWriter** (const std::string &phspFormat, const std::string &fileName, const UserOptions &userOptions, **FormatType** formatType=**FormatType::BINARY**, const **FixedValues** fixedValues=**FixedValues()**, unsigned int bufferSize=**DEFAULT_BUFFER_SIZE**)
Construct a new Phase Space File Writer object.
- virtual **~PhaseSpaceFileWriter** ()
Destroy the Phase Space File Writer object.
- void **addAdditionalHistories** (std::uint64_t additionalHistories)
Add additional Monte Carlo histories to the count.
- void **close** ()
Close the phase space file and finalize writing.
- **ByteOrder** **getByteOrder** () const
Get the byte order used for binary data writing.
- float **getConstantPx** () const
Get the constant X-component of the direction unit vector (if constant).
- float **getConstantPy** () const
Get the constant Y-component of the direction unit vector (if constant).
- float **getConstantPz** () const
Get the constant Z-component of the direction unit vector (if constant).
- float **getConstantWeight** () const
Get the constant statistical weight value (if constant).
- float **getConstantX** () const
Get the constant X coordinate value (if constant).
- float **getConstantY** () const
Get the constant Y coordinate value (if constant).
- float **getConstantZ** () const
Get the constant Z coordinate value (if constant).
- const std::string **getFileName** () const
Get the filename where the phase space file is being written.
- const **FixedValues** **getFixedValues** () const
Get the fixed values configuration.
- virtual std::uint64_t **getHistoriesWritten** () const
Get the number of Monte Carlo histories that have been written.
- std::uint64_t **getParticlesWritten** () const
Get the number of particles that have been written to the file.
- const std::string **getPHSPFormat** () const
Get the phase space file format identifier.
- bool **isPxConstant** () const
Check if the X-component of the direction unit vector is set to a constant value for all particles.
- bool **isPyConstant** () const
Check if the Y-component of the direction unit vector is set to a constant value for all particles.

- bool [isPzConstant](#) () const
Check if the Z-component of the direction unit vector is set to a constant value for all particles.
- bool [isWeightConstant](#) () const
Check if the statistical weight is set to a constant value for all particles.
- bool [isXConstant](#) () const
Check if the X coordinate is set to a constant value for all particles.
- bool [isYConstant](#) () const
Check if the Y coordinate is set to a constant value for all particles.
- bool [isZConstant](#) () const
Check if the Z coordinate is set to a constant value for all particles.
- virtual void [writeParticle](#) ([Particle](#) particle)
Write a particle to the phase space file.

Static Public Member Functions

- static std::vector< [CLICommand](#) > [getFormatSpecificCLICommands](#) ()
Get format-specific command-line options.

Static Public Member Functions inherited from [ParticleZoo::PhaseSpaceFileWriter](#)

- static std::vector< [CLICommand](#) > [getCLICommands](#) ()
Get command line interface commands supported by this writer.

Protected Member Functions

- bool [accountForAdditionalHistories](#) (std::uint64_t additionalHistories) override
Override base class method to handle additional histories.
- bool [canWritePseudoParticlesExplicitly](#) () const override
Check if pseudo-particles can be written explicitly.
- std::size_t [getMaximumASCIILineLength](#) () const override
Get the maximum length of ASCII particle lines.
- std::size_t [getParticleRecordLength](#) () const override
Get the length of each particle record in bytes.
- std::uint64_t [getPendingHistories](#) () const override
Get the number of pending histories to account for.
- const std::string [writeASCIIParticle](#) ([Particle](#) &particle) override
Convert a particle to ASCII representation.
- void [writeBinaryParticle](#) ([ByteBuffer](#) &buffer, [Particle](#) &particle) override
Encode and write a single particle to binary data.
- void [writeHeaderData](#) ([ByteBuffer](#) &buffer) override
Write header data to the output buffer.

Protected Member Functions inherited from [ParticleZoo::PhaseSpaceFileWriter](#)

- virtual bool [canHaveConstantPx](#) () const
Check if this format supports constant X-component of the direction unit vector.
- virtual bool [canHaveConstantPy](#) () const
Check if this format supports constant Y-component of the direction unit vector.
- virtual bool [canHaveConstantPz](#) () const
Check if this format supports constant Z-component of the direction unit vector.
- virtual bool [canHaveConstantWeight](#) () const
Check if this format supports constant statistical weights.
- virtual bool [canHaveConstantX](#) () const
Check if this format supports constant X coordinates.
- virtual bool [canHaveConstantY](#) () const
Check if this format supports constant Y coordinates.
- virtual bool [canHaveConstantZ](#) () const
Check if this format supports constant Z coordinates.
- virtual void [fixedValuesHaveChanged](#) ()
Called when fixed values have been changed.
- virtual std::size_t [getParticleRecordStartOffset](#) () const
Get the byte offset where particle records start in the file.
- const UserOptions & [getUserOptions](#) () const
Get the user options that were passed to the constructor.
- void [setByteOrder](#) (ByteOrder byteOrder)
Set the byte order for binary data writing.
- void [setConstantPx](#) (float Px)
Set a constant X-component of the direction unit vector for all particles.
- void [setConstantPy](#) (float Py)
Set a constant Y-component of the direction unit vector for all particles.
- void [setConstantPz](#) (float Pz)
Set a constant Z-component of the direction unit vector for all particles.
- void [setConstantWeight](#) (float weight)
Set a constant statistical weight for all particles.
- void [setConstantX](#) (float X)
Set a constant X coordinate value for all particles.
- void [setConstantY](#) (float Y)
Set a constant Y coordinate value for all particles.
- void [setConstantZ](#) (float Z)
Set a constant Z coordinate value for all particles.
- virtual void [writeParticleManually](#) (Particle &particle)
Write a particle manually (for formats requiring third-party I/O).

7.24.1 Detailed Description

[Writer](#) for TOPAS phase space files.

Provides functionality to write phase space data in TOPAS format files, supporting ASCII, BINARY, and LIMITED formats. Handles TOPAS-specific features including pseudo-particle generation for efficient empty history tracking and configurable column layouts.

7.24.2 Constructor & Destructor Documentation

7.24.2.1 Writer()

```
ParticleZoo::TOPASphspFile::Writer::Writer (
    const std::string & filename,
    const UserOptions & options = UserOptions{} )
```

Construct writer for TOPAS phase space file.

Creates a new TOPAS format writer with format determined by command-line options. Defaults to BINARY format if no format is specified.

Parameters

<i>filename</i>	Path for the output TOPAS phase space file (.phsp)
<i>options</i>	User-specified options including format selection

Exceptions

<i>std::runtime_error</i>	if file cannot be created or format is invalid
---------------------------	--

7.24.3 Member Function Documentation

7.24.3.1 accountForAdditionalHistories()

```
bool ParticleZoo::TOPASphspFile::Writer::accountForAdditionalHistories (
    std::uint64_t additionalHistories ) [inline], [override], [protected], [virtual]
```

Override base class method to handle additional histories.

For BINARY format, creates a pseudo-particle to represent multiple empty histories in a single record. For other formats, delegates to base class.

7.24 ParticleZoo::TOPASphspFile::Writer Class Reference

Parameters

<i>additionalHistories</i>	Number of additional empty histories to account for
----------------------------	---

Returns

false for BINARY (handled internally), true for others (use base class)

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.24.3.2 canWritePseudoParticlesExplicitly()

```
bool ParticleZoo::TOPASphspFile::Writer::canWritePseudoParticlesExplicitly ( ) const [inline],  
[override], [protected], [virtual]
```

Check if pseudo-particles can be written explicitly.

Returns

true for BINARY format, false for ASCII and LIMITED

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.24.3.3 getFormatSpecificCLICommands()

```
std::vector< CLICommand > ParticleZoo::TOPASphspFile::Writer::getFormatSpecificCLICommands ( )  
[static]
```

Get format-specific command-line options.

Returns

Vector of CLI commands supported by TOPAS writer

7.24.3.4 getHeader()

```
Header & ParticleZoo::TOPASphspFile::Writer::getHeader ( ) [inline]
```

Get access to the TOPAS header for configuration.

Returns

Reference to the header for modification and column management

7.24.3.5 `getMaximumASCIILineLength()`

```
std::size_t ParticleZoo::TOPASphspFile::Writer::getMaximumASCIILineLength ( ) const [inline],  
[override], [protected], [virtual]
```

Get the maximum length of ASCII particle lines.

Returns

Maximum line length (1024 characters for TOPAS format)

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.24.3.6 `getMaximumSupportedParticles()`

```
std::uint64_t ParticleZoo::TOPASphspFile::Writer::getMaximumSupportedParticles ( ) const [inline],  
[override], [virtual]
```

Get the maximum number of particles this format can store.

Returns

Maximum particle count

Implements [ParticleZoo::PhaseSpaceFileWriter](#).

7.24.3.7 `getParticleRecordLength()`

```
std::size_t ParticleZoo::TOPASphspFile::Writer::getParticleRecordLength ( ) const [inline], [override],  
[protected], [virtual]
```

Get the length of each particle record in bytes.

Returns

Record length as defined by the header and format type

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.24 ParticleZoo::TOPASphspFile::Writer Class Reference

7.24.3.8 getPendingHistories()

```
std::uint64_t ParticleZoo::TOPASphspFile::Writer::getPendingHistories ( ) const [inline], [override],  
[protected], [virtual]
```

Get the number of pending histories to account for.

In this override these are the empty histories tracked for pseudo-particle writing.

Returns

Count of empty histories pending writing

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.24.3.9 getTOPASFormat()

```
TOPASFormat ParticleZoo::TOPASphspFile::Writer::getTOPASFormat ( ) const [inline]
```

Get the TOPAS format type being written.

Returns

TOPASFormat enum indicating ASCII, BINARY, or LIMITED

7.24.3.10 writeASCIIParticle()

```
const std::string ParticleZoo::TOPASphspFile::Writer::writeASCIIParticle (   
    Particle & particle ) [override], [protected], [virtual]
```

Convert a particle to ASCII representation.

Formats a particle according to TOPAS ASCII specification with configurable columns.

Parameters

<i>particle</i>	Particle object to convert to ASCII
-----------------	---

Returns

ASCII string representation

Exceptions

<code>std::runtime_error</code>	if particle type is unsupported
---------------------------------	---------------------------------

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.24.3.11 writeBinaryParticle()

```
void ParticleZoo::TOPASphspFile::Writer::writeBinaryParticle (
    ByteBuffer & buffer,
    Particle & particle ) [inline], [override], [protected], [virtual]
```

Encode and write a single particle to binary data.

Handles format-specific binary encoding including pseudo-particle generation for empty history accounting in BINARY format.

Parameters

<i>buffer</i>	Binary buffer to write particle data to
<i>particle</i>	Particle object to encode and store

Exceptions

<code>std::runtime_error</code>	if format is unsupported
---------------------------------	--------------------------

Reimplemented from [ParticleZoo::PhaseSpaceFileWriter](#).

7.24.3.12 writeHeaderData()

```
void ParticleZoo::TOPASphspFile::Writer::writeHeaderData (
    ByteBuffer & buffer ) [override], [protected], [virtual]
```

Write header data to the output buffer.

Updates header statistics and writes the complete header file. The header is written to a separate .header file.

7.24 ParticleZoo::TOPASphspFile::Writer Class Reference

Parameters

<i>buffer</i>	Binary buffer (unused for TOPAS as header is separate)
---------------	--

Implements [ParticleZoo::PhaseSpaceFileWriter](#).

The documentation for this class was generated from the following files:

- include/particlezoo/TOPAS/TOPASphspFile.h
- src/topas/TOPASphspFile.cc

7.25 ParticleZoo::Version Struct Reference

[Version](#) information and metadata for the [ParticleZoo](#) library.

```
#include <particlezoo/utilities/version.h>
```

Static Public Member Functions

- static std::string [GetVersionString](#) ()
Generate a complete version string for display.

Static Public Attributes

- static constexpr const char * [CAVEAT](#) = ""
Development status indicator.
- static constexpr const int [MAJOR_VERSION](#) = 1
Major version number.
- static constexpr const int [MINOR_VERSION](#) = 1
Minor version number.
- static constexpr const int [PATCH_VERSION](#) = 2
Patch version number.
- static constexpr const char * [PROJECT_NAME](#) = "ParticleZoo"
Official project name.

7.25.1 Detailed Description

[Version](#) information and metadata for the [ParticleZoo](#) library.

This structure provides compile-time constants for version identification and runtime functions for generating version strings. It serves as the authoritative source for all version-related information used throughout the library, documentation generation, and user applications.

The version follows semantic versioning (SemVer) principles with MAJOR.MINOR.PATCH numbering plus development status indicators.

7.25.2 Member Function Documentation

7.25.2.1 GetVersionString()

```
static std::string ParticleZoo::Version::GetVersionString ( ) [inline], [static]
```

Generate a complete version string for display.

Creates a human-readable version string combining all version components in the standard format: "ProjectName vMAJOR.MINOR.PATCH STATUS"

Returns

Complete version string including project name, version numbers, and status

7.25.3 Member Data Documentation

7.25.3.1 CAVEAT

```
constexpr const char* ParticleZoo::Version::CAVEAT = "" [static], [constexpr]
```

Development status indicator.

Indicates the current development status of this version. Only used to indicate pre-release or special build states.

7.25.3.2 MAJOR_VERSION

```
constexpr const int ParticleZoo::Version::MAJOR_VERSION = 1 [static], [constexpr]
```

Major version number.

Incremented for backwards-incompatible API changes. Applications built against different major versions may require code changes.

7.25.3.3 MINOR_VERSION

```
constexpr const int ParticleZoo::Version::MINOR_VERSION = 1 [static], [constexpr]
```

Minor version number.

Incremented for backwards-compatible feature additions. Applications built against the same major version should work with higher minor versions without modification.

7.25.3.4 PATCH_VERSION

```
constexpr const int ParticleZoo::Version::PATCH_VERSION = 2 [static], [constexpr]
```

Patch version number.

Incremented for backwards-compatible bug fixes and minor improvements that do not add new features. Applications should always be able to upgrade to higher patch versions safely.

7.25.3.5 PROJECT_NAME

```
constexpr const char* ParticleZoo::Version::PROJECT_NAME = "ParticleZoo" [static], [constexpr]
```

Official project name.

The canonical name of the [ParticleZoo](#) library used in all user-facing output, documentation, and version strings.

The documentation for this struct was generated from the following file:

- `include/particlezoo/utilities/version.h`

Chapter 8

Python API Reference

This chapter documents the Python bindings for ParticleZoo.

8.1 Installation

The python module 'particlezoo' can be installed either in a virtual environment or in the user site-packages. In other case, you can do the following from the project root directory:

```
./configure
make install-python      # Standard install
# or
make install-python-dev  # Development mode (editable install)
```

Note: These commands will prompt before using `-break-system-packages` if no virtual environment is detected.

8.2 Quick Start

8.2.1 Reading a Phase Space File

```
import particlezoo as pz

# Read phase space file with automatic format detection
reader = pz.create_reader("input.egsphsp")
for p in reader:
    ke_mev = p.kinetic_energy / pz.MeV
    print(f"Energy: {ke_mev:.3f} MeV")
reader.close()
```

8.2.2 Writing a Phase Space File

```
import particlezoo as pz

# Create a writer with automatic format detection
writer = pz.create_writer("output.IAEAphsp")

# Create and write particles
for i in range(100):
    particle = pz.particle_from_pdg(
        pdg=11, # electron
        kineticEnergy=6.0 * pz.MeV,
        x=0.0 * pz.cm,
        y=0.0 * pz.cm,
        z=0.0 * pz.cm,
        px=0.0, py=0.0, pz=1.0,
        isNewHistory=(i == 0),
        weight=1.0
    )
    writer.write_particle(particle)

writer.close()
print(f"Wrote {writer.get_particles_written()} particles")
```

8.3 Classes

8.3.1 ArgParser

Singleton class for parsing command line arguments. Provides a centralized system for registering command line argument definitions and parsing command line input. Designed to provide a consistent interface for command line parsing across ParticleZoo tools.

Methods

parse_args()

Parse command line arguments and return UserOptions. Args should be a list of command line argument strings. Returns UserOptions object containing parsed arguments.

8.3 Classes

8.3.2 BoolPropertyType

Enumeration of boolean property types for particles. Defines standardized boolean flags that can be associated with particles from different Monte Carlo simulation codes.

Members:

INVALID : Invalid property type

IS_MULTIPLE_CROSSER : Flag indicating that the particle crossed the phase space plane multiple times (assuming the phase space is planar)

CUSTOM : Custom boolean property type, can be used for any user-defined purpose

8.3.3 EGSLATCHOPTION

Enumeration of LATCH interpretation options for EGS phase space files. Defines how the LATCH bits are interpreted when reading or writing particle data.

Members:

LATCH_OPTION_1 : Non-Inherited LATCH Setting: bits 1-23 store where the particle has been, secondaries do not inherit any bits from parent particles

LATCH_OPTION_2 : Comprehensive LATCH Setting (default): bits 1-23 store where the particle has been, bit settings are inherited from parent particles

LATCH_OPTION_3 : Comprehensive LATCH Setting 2: bits 1-23 store where the particle has interacted instead of where it has been, bit settings are inherited from parent particles

8.3.4 FixedValues

Structure defining constant (fixed) values for particle properties. Used to optimize phase space files by storing constant values once rather than repeating them for every particle. Useful when all particles share certain properties (e.g., all particles start from the same position).

Properties

- `constant_px` – Constant X directional cosine value (when `px_is_constant` is true)
- `constant_py` – Constant Y directional cosine value (when `py_is_constant` is true)
- `constant_pz` – Constant Z directional cosine value (when `pz_is_constant` is true)
- `constant_weight` – Constant statistical weight value (when `weight_is_constant` is true)
- `constant_x` – Constant X coordinate value (when `x_is_constant` is true)
- `constant_y` – Constant Y coordinate value (when `y_is_constant` is true)
- `constant_z` – Constant Z coordinate value (when `z_is_constant` is true)
- `px_is_constant` – True if X directional cosine is constant for all particles
- `py_is_constant` – True if Y directional cosine is constant for all particles
- `pz_is_constant` – True if Z directional cosine is constant for all particles
- `weight_is_constant` – True if statistical weight is constant for all particles
- `x_is_constant` – True if X coordinate is constant for all particles
- `y_is_constant` – True if Y coordinate is constant for all particles
- `z_is_constant` – True if Z coordinate is constant for all particles

8.3.5 FloatPropertyType

Enumeration of floating-point property types for particles. Defines standardized float properties that can be associated with particles from different Monte Carlo simulation codes.

Members:

`INVALID` : Invalid property type, used for error checking

`XLAST` : EGS-specific XLAST variable, for photons it is the X position of the last interaction, for electrons/positrons it is the X position it (or its ancestor) was created at by a photon

`YLAST` : EGS-specific YLAST variable, for photons it is the Y position of the last interaction, for electrons/positrons it is the Y position it (or its ancestor) was created at by a photon

`ZLAST` : EGS-specific ZLAST variable, for photons it is the Z position of the last interaction, for electrons/positrons it is the Z position it (or its ancestor) was created at by a photon

`CUSTOM` : Custom float property type, can be used for any user-defined purpose

8.3 Classes

8.3.6 FormatRegistry

Registry for phase space file format plugins. Provides factory functions to create readers and writers for different simulation formats (EGS, IAEA, TOPAS, ROOT, etc.).

Methods

extension_for_format()

Get the standard file extension for a given format name.

formats_for_extension()

Get list of formats that match a given file extension. Returns list of format names.

register_standard_formats()

Register all built-in phase space file formats. Must be called before creating readers/writers.

supported_formats()

Get list of all registered supported formats. Returns list of SupportedFormat objects.

8.3.7 HistoryBalancedParallelReader

Multi-threaded phase space file reader that distributes histories evenly across threads. Each thread maintains its own reader positioned at a specific section of the file. Histories are the fundamental unit of work; particle counts may vary between threads.

Methods

close()

Close all underlying readers and release resources.

get_histories_read()

Get the number of original histories processed by a thread (including empty ones).

get_next_particle()

Read and return the next particle for a thread.

get_number_of_original_histories()

Get the number of original histories in the file (including empty ones).

get_number_of_particles()

Get the total number of particles in the phase space file.

get_number_of_represented_histories()

Get the number of histories that produced at least one particle.

get_number_of_threads()

Get the number of threads used by this reader.

get_particles_read()

Get the number of particles processed by a thread.

get_total_histories_read()

Get the total number of original histories read across all threads.

has_more_particles()

Check if more particles are available for a thread.

peek_next_particle()

Peek at the next particle for a thread without consuming it.

8.3 Classes

8.3.8 IntPropertyType

Enumeration of integer property types for particles. Defines standardized integer properties that can be associated with particles from different Monte Carlo simulation codes.

Members:

INVALID : Invalid property type, used for error checking

INCREMENTAL_HISTORY_NUMBER : Sequential history number for tracking, tracks the number of new histories since the last particle was recorded

EGS_LATCH : EGS-specific latch variable (see BEAMnrc User Manual, Chapter 8 for details)

PENELOPE_ILB1 : PENELOPE ILB array value 1, corresponds to the generation of the particle (1 for primary, 2 for secondary, etc.)

PENELOPE_ILB2 : PENELOPE ILB array value 2, corresponds to the particle type of the particle's parent (applies only if ILB1 > 1)

PENELOPE_ILB3 : PENELOPE ILB array value 3, corresponds to the interaction type that created the particle (applies only if ILB1 > 1)

PENELOPE_ILB4 : PENELOPE ILB array value 4, is non-zero if the particle is created by atomic relaxation and corresponds to the atomic transition that created the particle

PENELOPE_ILB5 : PENELOPE ILB array value 5, a user-defined value which is passed on to all descendant particles created by this particle

GENERATION : Generation of the particle (1 for primary, 2 for secondary, etc.)

CUSTOM : Custom integer property type, can be used for any user-defined purpose

8.3.9 Particle

Represents a particle in phase space with position, momentum direction, kinetic energy, and additional properties. Used for Monte Carlo particle transport simulations.

Properties

- `is_new_history` – Whether this particle starts a new Monte Carlo history
- `is_primary` – Whether this particle is a primary particle (not generated by another particle)
- `kinetic_energy` – The kinetic energy of the particle
- `px` – The X component of the directional cosine (momentum unit vector)
- `py` – The Y component of the directional cosine (momentum unit vector)
- `pz` – The Z component of the directional cosine (momentum unit vector)
- `type` – The particle type (electron, photon, proton, etc.)
- `weight` – The statistical weight of the particle
- `x` – The X coordinate position
- `y` – The Y coordinate position
- `z` – The Z coordinate position

Methods

`get_bool_property()`

Get the value of a boolean property. Raises error if property is not set.

`get_custom_bool_properties()`

Get a list of all custom boolean properties.

`get_custom_float_properties()`

Get a list of all custom float properties.

`get_custom_int_properties()`

Get a list of all custom integer properties.

`get_custom_string_properties()`

8.3 Classes

Get a list of all custom string properties.

get_float_property()

Get the value of a float property. Raises error if property is not set.

get_incremental_histories()

Convenience function to get the number of incremental histories regardless of whether the property is set. If the property is not set, it returns 1 if the particle is marked as a new history, otherwise 0.

get_int_property()

Get the value of an integer property. Raises error if property is not set.

get_number_of_bool_properties()

Get the number of boolean properties currently stored.

get_number_of_float_properties()

Get the number of float properties currently stored.

get_number_of_int_properties()

Get the number of integer properties currently stored.

get_pdg_code()

Get the PDG identification code of the particle.

has_bool_property()

Check if the particle has a specific boolean property set.

has_float_property()

Check if the particle has a specific float property set.

has_int_property()

Check if the particle has a specific integer property set.

project_to_x()

Project the particle's trajectory to a specific X coordinate. Calculates where the particle would be when it reaches the specified X value, assuming it travels in a straight line. Updates the Y and Z coordinates accordingly. Returns True if projection was successful, False if impossible (particle has no movement in X direction).

project_to_y()

Project the particle's trajectory to a specific Y coordinate. Calculates where the particle would be when it reaches the specified Y value, assuming it travels in a straight line. Updates the X and Z coordinates accordingly. Returns True if projection was successful, False if impossible (particle has no movement in Y direction).

project_to_z()

Project the particle's trajectory to a specific Z coordinate. Calculates where the particle would be when it reaches the specified Z value, assuming it travels in a straight line. Updates the X and Y coordinates accordingly. Returns True if projection was successful, False if impossible (particle has no movement in Z direction).

reserve_bool_properties()

Reserve memory for boolean properties.

reserve_float_properties()

Reserve memory for float properties.

reserve_int_properties()

Reserve memory for integer properties.

set_bool_property()

8.3 Classes

Set the value of a boolean property.

set_float_property()

Set the value of a float property.

set_generation()

Set the generation of the particle (1 for primary, 2+ for secondary).

set_incremental_histories()

Convenience function to set the number of incremental histories using the INCREMENTAL_HISTORY_NUMBER integer property. The value must be greater than 0.

set_int_property()

Set the value of an integer property.

set_string_property()

Add a custom string property to the particle.

8.3.10 ParticleBalancedParallelReader

Multi-threaded phase space file reader that distributes particles evenly across threads. Each thread maintains its own reader positioned at a specific section of the file. Particles are the fundamental unit of work; history counts may vary between threads.

Methods

close()

Close all underlying readers and release resources.

get_histories_read()

Get the number of original histories covered by a thread.

get_next_particle()

Read and return the next particle for a thread.

get_number_of_original_histories()

Get the number of original histories in the file (including empty ones).

get_number_of_particles()

Get the total number of particles in the phase space file.

get_number_of_represented_histories()

Get the number of histories that produced at least one particle.

get_number_of_threads()

Get the number of threads used by this reader.

get_particles_read()

Get the number of particles processed by a thread.

get_total_histories_read()

Get the total number of original histories read across all threads.

has_more_particles()

Check if more particles are available for a thread.

peek_next_particle()

Peek at the next particle for a thread without consuming it.

8.3 Classes

8.3.11 PhaseSpaceFileReader

Reader for phase space files from various Monte Carlo simulation formats (EGS, IAEA, TOPAS, etc.). Provides unified interface for reading particle data from different file formats. Create using `create_reader()` or `create_reader_for_format()` factory functions.

Methods

close()

Close the phase space file and release associated resources.

get_constant_px()

Get the constant X directional cosine value (when `is_px_constant` returns True).

get_constant_py()

Get the constant Y directional cosine value (when `is_py_constant` returns True).

get_constant_pz()

Get the constant Z directional cosine value (when `is_pz_constant` returns True).

get_constant_weight()

Get the constant statistical weight value (when `is_weight_constant` returns True).

get_constant_x()

Get the constant X coordinate value (when `is_x_constant` returns True).

get_constant_y()

Get the constant Y coordinate value (when `is_y_constant` returns True).

get_constant_z()

Get the constant Z coordinate value (when is_z_constant returns True).

get_file_name()

Get the filename/path of the phase space file being read.

get_file_size()

Get the size of the phase space file in bytes.

get_histories_read()

Get the number of Monte Carlo histories read so far. Returns total original histories if end of file reached.

get_next_particle()

Read and return the next particle from the file. Automatically handles buffering and format-specific parsing.

get_number_of_original_histories()

Get the number of original Monte Carlo histories that generated this phase space.

get_number_of_particles()

Get the total number of particles in the phase space file.

get_particles_read()

Get the number of particles read so far (excludes metadata and skipped particles).

get_phsp_format()

Get the phase space file format identifier (e.g., 'IAEA', 'EGS', 'TOPAS').

8.3 Classes

has_more_particles()

Check if there are more particles available to read. Returns True if more particles remain, False at end of file.

is_px_constant()

Check if X directional cosine is constant for all particles.

is_py_constant()

Check if Y directional cosine is constant for all particles.

is_pz_constant()

Check if Z directional cosine is constant for all particles.

is_weight_constant()

Check if statistical weight is constant for all particles.

is_x_constant()

Check if X coordinate is constant for all particles.

is_y_constant()

Check if Y coordinate is constant for all particles.

is_z_constant()

Check if Z coordinate is constant for all particles.

move_to_particle()

Move the file position to a specific particle index (0-based). Useful for random access.

8.3.12 PhaseSpaceFileWriter

Writer for phase space files to various Monte Carlo simulation formats (EGS, IAEA, TOPAS, etc.). Provides unified interface for writing particle data to different file formats. Create using `create_writer()` or `create_writer_for_format()` factory functions.

Methods

add_additional_histories()

Add additional Monte Carlo histories to the count. Used for simulation histories that produced no particles.

close()

Close the phase space file, flush buffered data, and finalize writing.

get_file_name()

Get the filename/path where the phase space file is being written.

get_histories_written()

Get the number of Monte Carlo histories written to the file.

get_maximum_supported_particles()

Get the maximum number of particles this format can support.

get_particles_written()

Get the number of particles written to the file (excludes pseudo-particles).

get_phsp_format()

Get the phase space file format identifier (e.g., 'IAEA', 'EGS', 'TOPAS').

write_particle()

Write a particle to the phase space file. Automatically buffers and applies constant values.

8.3.13 SupportedFormat

Information about a supported phase space file format including name, description, and file extension.

Properties

- `description` – Human-readable description of the format
- `file_extension` – Standard file extension for this format (e.g., `'.phsp'`, `'.IAEAphsp'`)
- `file_extension_can_have_suffix` – True if the file extension can have an additional suffix (e.g., `'.egsphsp1'`)
- `name` – Format identifier name (e.g., `'IAEA'`, `'EGS'`, `'TOPAS'`)

8.3.14 UserOptions

Container for command-line options parsed by ArgParser. Stores format-specific and general options for phase space file readers and writers. Use `ArgParser.parse_args()` to create from command-line arguments, then `extract_*`() methods to retrieve values.

Methods

extract_bool_option()

Extract a boolean option value from a command

extract_float_option()

Extract a float option value from a command

extract_int_option()

Extract an integer option value from a command

extract_positional()

Extract a positional argument by index

extract_string_option()

Extract a string option value from a command

extract_uint_option()

Extract an unsigned integer option value from a command

8.4 Functions

8.4.1 `all_particle_types()`

`all_particle_types()` -> dict

Return a mapping of ParticleType names to enum values

8.4.2 `apply_ilb1_to_particle()`

`apply_ilb1_to_particle`

Apply PENELOPE ILB1 value to a particle. ILB1 represents particle generation: 1 for primary particles, 2+ for secondary particles. Also sets GENERATION flag. Raises RuntimeError if `ilb1 < 1`.

8.4.3 `apply_ilb2_to_particle()`

`apply_ilb2_to_particle`

Apply PENELOPE ILB2 value to a particle. ILB2 stores the parent particle type code. Only meaningful for secondary particles (`ILB1 > 1`).

8.4.4 `apply_ilb3_to_particle()`

`apply_ilb3_to_particle`

Apply PENELOPE ILB3 value to a particle. ILB3 stores the interaction type that created this particle. Only meaningful for secondary particles (`ILB1 > 1`).

8.4.5 `apply_ilb4_to_particle()`

`apply_ilb4_to_particle`

Apply PENELOPE ILB4 value to a particle. ILB4 indicates atomic relaxation transitions. Non-zero if particle was created by atomic relaxation.

8.4 Functions

8.4.6 `apply_ilb5_to_particle()`

`apply_ilb5_to_particle`

Apply PENELOPE ILB5 value to a particle. ILB5 is a user-defined tracking value passed on to all descendant particles.

8.4.7 `apply_ilb_array_to_particle()`

`apply_ilb_array_to_particle`

Apply all five PENELOPE ILB values to a particle from a list/array. Array must contain [ILB1, ILB2, ILB3, ILB4, ILB5] in order. ILB1: generation (1=primary, 2+=secondary), ILB2: parent type, ILB3: interaction type, ILB4: atomic transition, ILB5: user-defined.

8.4.8 `apply_latch_to_particle()`

`apply_latch_to_particle`

Apply EGS LATCH value to a particle. Sets the EGS_LATCH integer property, IS_MULTIPLE_CROSSER boolean property (from bit 32), and GENERATION integer property (from bits 24-28 for LATCH_OPTION_2 and LATCH_OPTION_3).

8.4.9 `create_reader()`

`create_reader`

Create a phase space file reader with automatic format detection based on file extension. Returns PhaseSpaceFileReader instance for reading particles from the file.

8.4.10 `create_reader_for_format()`

`create_reader_for_format`

Create a phase space file reader for a specific format (e.g., 'IAEA', 'EGS', 'TOPAS'). Returns PhaseSpaceFileReader instance configured for the specified format.

8.4.11 `create_writer()`

`create_writer`

Create a phase space file writer with automatic format detection based on file extension. Returns `PhaseSpaceFileWriter` instance for writing particles to the file. Use `fixed_values` to specify constant properties for all particles.

8.4.12 `create_writer_for_format()`

`create_writer_for_format`

Create a phase space file writer for a specific format (e.g., 'IAEA', 'EGS', 'TOPAS'). Returns `PhaseSpaceFileWriter` instance configured for the specified format. Use `fixed_values` to specify constant properties for all particles.

8.4.13 `does_particle_passLatch_filter()`

`does_particle_passLatch_filter`

Check if a particle passes the specified LATCH bitmask filter. Returns `True` if the particle has `EGS_LATCH` property and `(LATCH & filter) == filter`.

8.4.14 `extract_ilb1_from_particle()`

`extract_ilb1_from_particle(particle: particlezoo._pz.Particle) -> int`

Extract PENELOPE ILB1 value from a particle. Returns `PENELOPE_ILB1` property if set, otherwise infers from `GENERATION` flag.

8.4.15 `extract_ilb2_from_particle()`

`extract_ilb2_from_particle(particle: particlezoo._pz.Particle) -> int`

Extract PENELOPE ILB2 value from a particle. Returns `PENELOPE_ILB2` property if set, otherwise 0.

8.4 Functions

8.4.16 `extract_ilb3_from_particle()`

`extract_ilb3_from_particle(particle: particlezoo._pz.Particle) -> int`

Extract PENELOPE ILB3 value from a particle. Returns PENELOPE_ILB3 property if set, otherwise 0.

8.4.17 `extract_ilb4_from_particle()`

`extract_ilb4_from_particle(particle: particlezoo._pz.Particle) -> int`

Extract PENELOPE ILB4 value from a particle. Returns PENELOPE_ILB4 property if set, otherwise 0.

8.4.18 `extract_ilb5_from_particle()`

`extract_ilb5_from_particle(particle: particlezoo._pz.Particle) -> int`

Extract PENELOPE ILB5 value from a particle. Returns PENELOPE_ILB5 property if set, otherwise 0.

8.4.19 `extract_ilb_array_from_particle()`

`extract_ilb_array_from_particle`

Extract all five PENELOPE ILB values from a particle as a list. Returns [ILB1, ILB2, ILB3, ILB4, ILB5] extracted from particle properties. Missing properties return 0 (except ILB1 which may infer from GENERATION).

8.4.20 `extract_latch_from_particle()`

`extract_latch_from_particle`

Extract EGS LATCH value from a particle. If EGS_LATCH property exists, returns it directly. Otherwise, constructs LATCH from particle type (bits 29-30), IS_MULTIPLE_CROSSER flag (bit 32), and GENERATION flag (bits 24-28 for LATCH_OPTION_2 and LATCH_OPTION_3).

8.4.21 `get_particle_type_from_pdgid()`

`get_particle_type_from_pdgid`

Map PDG particle code to ParticleType enum. Returns ParticleType for recognized PDG codes.

8.4.22 `get_particle_type_name()`

`get_particle_type_name(type: particlezoo._pz.ParticleType) -> str`

Get human-readable name for ParticleType enum (e.g., 'electron', 'photon').

8.4.23 `get_pdgid()`

`get_pdgid(type: particlezoo._pz.ParticleType) -> int`

Get PDG particle code from ParticleType enum. Returns integer PDG code.

8.4.24 `get_version_string()`

`get_version_string() -> str`

Get the full ParticleZoo version string (e.g. 'ParticleZoo v1.1.1').

8.4.25 `particle_from_pdg()`

`particle_from_pdg`

Create a Particle using PDG particle code instead of ParticleType enum. Automatically converts PDG code to ParticleType. Directional cosines normalized automatically.